

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent
Kind Code
Date of Patent
Inventor(s)

12413784
B2
September 09, 2025
Deng; Zhipin et al.

Deblocking signaling in video coding

Abstract

Systems, methods and apparatus for video processing are described. The video processing may include video encoding, video decoding or video transcoding. One example method of video processing includes a method of video processing, including performing a conversion between a video region of a video and a bitstream of the video according to a format rule. The format rule specifies that an applicability of a deblocking filter for the video region is determined based on syntax elements at i) a picture parameter set level and ii) a picture level or a slice level.

Inventors: Deng; Zhipin (Beijing, CN), Zhang; Li (San Diego, CA), Wang; Ye-kui (San Diego, CA), Zhang; Kai (San Diego, CA), Xu; Jizheng (San Diego, CA)

Applicant: Beijing Bytedance Network Technology Co., Ltd. (Beijing, CN); Bytedance Inc. (Los Angeles, CA)

Family ID: 78022414

Assignee: BEIJING BYTEDANCE NETWORK TECHNOLOGY CO., LTD. (Beijing, CN); BYTEDANCE INC. (Los Angeles, CA)

Appl. No.: 18/325592

Filed: May 30, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20230319312 A1	Oct. 05, 2023

Foreign Application Priority Data

WO	PCT/CN2020/083967	Apr. 09, 2020
----	-------------------	---------------

Related U.S. Application Data

continuation parent-doc US 17962882 20221010 US 11743506 child-doc US 18325592
continuation parent-doc WO PCT/CN2021/086111 20210409 PENDING child-doc US 17962882

Publication Classification

Int. Cl.: H04N19/70 (20140101); H04N19/184 (20140101)

U.S. Cl.:

CPC H04N19/70 (20141101); H04N19/184 (20141101);

Field of Classification Search

CPC: H04N (19/70); H04N (19/184); H04N (19/172); H04N (19/117); H04N (19/174); H04N (19/82); H04N (19/86)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
7373009	12/2007	Winger	N/A	N/A
9001883	12/2014	Tsai	N/A	N/A
10972755	12/2020	Chen	N/A	N/A
11064206	12/2020	Zhang	N/A	N/A
11303914	12/2021	Li	N/A	N/A
11343516	12/2021	Li	N/A	N/A
11405648	12/2021	Yang	N/A	N/A
11432016	12/2021	Chen	N/A	N/A
11470348	12/2021	Su	N/A	N/A

11470358	12/2021	Samuelsson	N/A	N/A
11509913	12/2021	Lim	N/A	N/A
11553206	12/2022	He	N/A	N/A
11575890	12/2022	Li	N/A	N/A
11601657	12/2022	Hu	N/A	N/A
11743506	12/2022	Deng et al.	N/A	N/A
11825124	12/2022	Wang et al.	N/A	N/A
11924474	12/2023	Zhang et al.	N/A	N/A
12137252	12/2023	Deng et al.	N/A	N/A
12170766	12/2023	Deng et al.	N/A	N/A
2012/0121017	12/2011	Chen	N/A	N/A
2012/0189055	12/2011	Chien	N/A	N/A
2012/0189058	12/2011	Chen et al.	N/A	N/A
2012/0207227	12/2011	Tsai	N/A	N/A
2012/0250772	12/2011	Gao	N/A	N/A
2012/0269275	12/2011	Hannuksela	N/A	N/A
2013/0022104	12/2012	Chen	N/A	N/A
2013/0114742	12/2012	Hannuksela	N/A	N/A
2013/0156101	12/2012	Lu	N/A	N/A
2013/0182774	12/2012	Wang	N/A	N/A
2013/0194384	12/2012	Hannuksela	N/A	N/A
2014/0003489	12/2013	Hannuksela	N/A	N/A
2014/0016699	12/2013	Chen	N/A	N/A
2014/0064374	12/2013	Xiu	N/A	N/A
2014/0092991	12/2013	Sullivan	N/A	N/A
2014/0192885	12/2013	Seregin	N/A	N/A
2014/0254669	12/2013	Rapaka	N/A	N/A
2015/0016512	12/2014	Pu	N/A	N/A
2015/0016543	12/2014	Rapaka	N/A	N/A
2015/0043637	12/2014	Morigami	N/A	N/A
2015/0078456	12/2014	Hannuksela	N/A	N/A
2015/0103898	12/2014	Ye	N/A	N/A
2015/0195577	12/2014	Hannuksela	N/A	N/A
2015/0304665	12/2014	Hannuksela et al.	N/A	N/A
2016/0103898	12/2015	Zheng	N/A	N/A
2017/0324962	12/2016	Karczewicz	N/A	N/A
2017/0339415	12/2016	Wang	N/A	N/A
2018/0199051	12/2017	Hannuksela	N/A	N/A
2019/0020886	12/2018	Hannuksela	N/A	N/A
2019/0208207	12/2018	Misra	N/A	N/A
2019/0215517	12/2018	Ramasubramonian	N/A	N/A
2019/0335190	12/2018	Tabatabai	N/A	N/A
2020/0099924	12/2019	Seregin	N/A	N/A
2020/0296368	12/2019	Choi	N/A	N/A
2020/0314424	12/2019	Hu	N/A	H04N 19/463
2021/0092420	12/2020	Du	N/A	H04N 19/80
2021/0185330	12/2020	Kim	N/A	N/A
2022/0078460	12/2021	Misra	N/A	N/A
2022/0174287	12/2021	Lee	N/A	N/A
2022/0182681	12/2021	Paluri	N/A	N/A
2022/0321922	12/2021	Kim	N/A	N/A
2022/0329850	12/2021	Sullivan et al.	N/A	N/A
2022/0329851	12/2021	Sullivan	N/A	N/A
2022/0394301	12/2021	Deshpande	N/A	H04N 19/172
2023/0023220	12/2022	Wang et al.	N/A	N/A
2023/0027997	12/2022	Deshpande	N/A	N/A
2023/0028609	12/2022	Wang	N/A	N/A
2023/0038277	12/2022	Wang et al.	N/A	N/A
2023/0051471	12/2022	Kim	N/A	N/A
2023/0085304	12/2022	Naser	N/A	N/A
2023/0120726	12/2022	Deng et al.	N/A	N/A

FOREIGN PATENT DOCUMENTS

Patent No.	Application Date	Country	CPC
101288311	12/2007	CN	N/A
101317459	12/2007	CN	N/A
101356822	12/2008	CN	N/A
101558651	12/2008	CN	N/A
103210650	12/2012	CN	N/A
103370936	12/2012	CN	N/A
103918269	12/2013	CN	N/A
103947213	12/2013	CN	N/A
103999460	12/2013	CN	N/A
104025599	12/2013	CN	N/A
104054339	12/2013	CN	N/A

104137561	12/2013	CN	N/A
104303501	12/2014	CN	N/A
104662906	12/2014	CN	N/A
106664424	12/2016	CN	N/A
106797476	12/2016	CN	N/A
107079176	12/2016	CN	N/A
108028945	12/2017	CN	N/A
108141616	12/2017	CN	N/A
109076218	12/2017	CN	N/A
109196862	12/2018	CN	N/A
109691110	12/2018	CN	N/A
110115032	12/2018	CN	N/A
110710215	12/2019	CN	N/A
110771170	12/2019	CN	N/A
110809155	12/2019	CN	N/A
115443653	12/2021	CN	N/A
115428444	12/2024	CN	N/A
550618	12/2023	IN	N/A
2023511788	12/2022	JP	N/A
7506175	12/2023	JP	N/A
7583063	12/2023	JP	N/A
102629799	12/2023	KR	N/A
2012122364	12/2011	WO	N/A
2013067687	12/2012	WO	N/A
2013159335	12/2012	WO	N/A
2014052249	12/2013	WO	N/A
2015139010	12/2014	WO	N/A
2019089382	12/2018	WO	N/A
2020031059	12/2019	WO	N/A
2020039364	12/2019	WO	N/A
2020050752	12/2019	WO	N/A
2020057662	12/2019	WO	N/A
2020069058	12/2019	WO	N/A
2021207423	12/2020	WO	N/A

OTHER PUBLICATIONS

Foreign Communication From A Related Counterpart Application, Indian Application No. 202247059133, Office Action dated Dec. 29, 2022, 6 pages. cited by applicant

Notice of Allowance dated Apr. 25, 2023, 17 pages, U.S. Appl. No. 17/962,141, filed Oct. 7, 2022. cited by applicant

Foreign Communication From A Related Counterpart Application, European Application No. 21784701.1, Extended European Search Report dated Jul. 6, 2023, 14 pages. cited by applicant

Notice of Allowance dated Feb. 22, 2023, 36 pages, U.S. Appl. No. 17/962,882, filed Oct. 10, 2022. cited by applicant

Non-Final Office Action dated Mar. 30, 2023, 28 pages, U.S. Appl. No. 17/962,852, filed Oct. 10, 2022. cited by applicant

Non-Final Office Action dated Mar. 31, 2023, 19 pages, U.S. Appl. No. 17/967,382, filed Oct. 17, 2022. cited by applicant

Notice of Allowance from U.S. Appl. No. 18/450,647 dated Jun. 28, 2024, 12 pages. cited by applicant

Document: JVET-R0324, Chen, Y-W., et al., “AHG9: On syntax signaling conditions in picture header,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 5 pages. cited by applicant

Document: JVET-R0481-v1, Wang, Y.K., et al., “AHG2: Editorial input of integrated text for HLS adoptions,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 2 pages. cited by applicant

Document: JVET-R0173-r2, Naser, K., et al., “AhG 9: Cleanup of Constraint Flags,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 7 pages. cited by applicant

Document: JVET-S0105-v1, McCarthy, S., et al., “AHG9: Modification of general constraint information,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 19th Meeting: by teleconference, Jun. 22-Jul. 1, 2020, 16 pages. cited by applicant

Document: JVET-S0129-v1, Li, L., et al., “AHG9: cleanup on parameter sets and GCI,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 19th Meeting: by teleconference, Jun. 22-Jul. 1, 2020, 15 pages. cited by applicant

Document: JVET-S0085-v3, Chujoh, T., et al., “AHG9: On semantics related to reference picture lists,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 19th Meeting: by teleconference, Jun. 22-Jul. 1, 2020, 7 pages. cited by applicant

Document: JVET-R2001-vA, Bross, B., et al., “Versatile Video Coding (Draft 9),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 524 pages. cited by applicant

Chinese Office Action from Chinese Application No. 202180031080.5 dated Apr. 30, 2024, 14 pages. cited by applicant

Non-Final Office Action from U.S. Appl. No. 18/450,647 dated Feb. 29, 2024, 25 pages. cited by applicant

Extended European Search Report from European Application No. 21797509.3 dated Feb. 28, 2024, 20 pages. cited by applicant

Document: JVET-Q2001-vC, Bross, B., et al., “Versatile Video Coding (Draft 8),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 508 pages. cited by applicant

“Information technology—High efficiency coding and media delivery in heterogeneous environments—Part 2: High efficiency Video Coding,” ISO/IEC JTC 1/SC 29/WG 11 N 17661, Text of DIS ISO/IEC 23008-2:201x (4th ed.), Apr. 20, 2018, 8 pages. cited by applicant

Document: JVET-G1001-v1, Chen, J., et al., “Algorithm Description of Joint Exploration Test Model 7 (JEM 7),” Joint Video Exploration Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 7th Meeting: Torino, IT, Jul. 13-21, 2017, 50 pages. cited by applicant

Document: JVET-Q2002-v3, Chen, J., et al., “Algorithm description for Versatile Video Coding and Test Model 8 (VTM-8),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 97 pages. cited by applicant

Bossen, F., Retrieved from the internet: https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM.git, VTM software, Dec. 23, 2022, 3 pages. cited by applicant

Document: JVET-R0159-v2, Chen, J., et al., “AHG9: On high level syntax of deblocking filter,” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 4 pages. cited by applicant

Document: JVET-P2001-vE, Bross, B., et al., “Versatile Video Coding (Draft 7),” Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and

ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 491 pages. cited by applicant

Document: JVET-Q0505, Zhang, H., et al., "AHG15: Improvement for Quantization Matrix Signaling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 9 pages. cited by applicant

Document: JVET-Q0625-v3, Boyce, J., "Report of BoG on high level tool control, feature combinations, and Parameter sets," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 18 pages. cited by applicant

Document: JVET-Q0289-v1, Chen, L., et al., "AHG12/AHG9: Comments on miscellaneous HLS text," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 7 pages. cited by applicant

Document: JVET-P0410-v2, De Lagrange, P., et al., "AHG15: chroma QP mapping table syntax variant with less bits," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 10 pages. cited by applicant

Document: JVET-R0156-v1, Wang, B., et al., "Signaling cleanup on SPS," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 5 pages. cited by applicant

Document: JVET-Q0120-v1, Zhang, L., et al., "AHG12: Control of loop filtering across subpicture/tile/slice boundaries," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 11 pages. cited by applicant

Document: JVET-R0184-v1, Paluri, S., et al., "AHG9/AHG12: On reference picture wraparound for subpictures," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by Teleconference, Apr. 15-24, 2020, 9 pages. cited by applicant

Document: JVET-Q0212, He, Y., et al., "[AHG9/AHG12] On subpicture wrap around offset signaling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 8 pages. cited by applicant

Document: JVET-L0195-v2, Jung, J., et al., "CE6-related: MTS for non-square CUs," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 12th Meeting: Macao, CN, Oct. 3-12, 2018, 6 pages. cited by applicant

Document: JVET-R0324, Chen, Y., et al., "AHG9: On syntax signaling conditions in picture header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 2 pages. cited by applicant

Document: JVET-O0148-v1, Wang, Y., et al., "AHG17: On reference picture list signalling," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, 6 Pages. cited by applicant

Document: JVET-R0433, Naser, K., et al., "AhG 9: Combination of JVET-R0177/R0301 and JVET-R0074 on APS Signaling and Semantics Cleanup," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 9/ WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 5 pages. cited by applicant

Document: JVET-O0113-v1, Wang, Y., et al., "AHG17: On parsing dependency between parameter sets," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 15th Meeting: Gothenburg, SE, Jul. 3-12, 2019, 2 pages. cited by applicant

Document: JVET-Q0200-r2, Hendry, et al., "[AHG9]: On picture level and slice level tool parameters," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 9 pages. cited by applicant

Document: JVET-Q0379, Laroche, G., et al., "AhG9: On the position of APS Ids in Picture Header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 4 pages. cited by applicant

Document: JVET-P0180, Liu, H., et al., "Non-CE5: Cleanup of ALF syntax elements," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 8 Pages. cited by applicant

Document: JVET-P0293-v3, Hashimoto, T., et al., "AHG15: Signaling scaling matrix for LFNST case," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 6 pages. cited by applicant

Document: JVET-Q0117-v1, Wang, Y., et al., "AHG9: Cleanups on parameter sets," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 9 pages. cited by applicant

Document: JVET-Q0200-v1, Hendry, et al., "[AHG9]: On picture level and slice level tool parameters," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 6 pages. cited by applicant

Document: JVET-Q0200-r2 (v3), Hendry, et al., "[AHG9]: On picture level and slice level tool parameters," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 9 pages. cited by applicant

Document: JVET-Q0270, Pettersson, M., et al., "AHG9: On Picture Header Modifications," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 7 pages. cited by applicant

Document: JVET-Q0121-v1, Xu, J., et al., "AHG9: On deblocking control parameters," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 7 pages. cited by applicant

Document: JVET-R0324, Chen, Y., et al., "AHG9: On syntax signaling conditions in picture header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24 15-24, 2020, 2 pages. JVET-R0324.docx: pp. 1-2, JVET-R0324_JVET-Q2001-vE.docx: pp. 1,47-50. cited by applicant

Document: JVET-R0324, Chen, Y., et al., "AHG9: On syntax signaling conditions in picture header," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24 15-24, 2020, 2 pages. VET-R0324-v5.zip: JVET-R0324_r4.docx: pp. 1-5. cited by applicant

Document: JVET-Q2001-vE, Bross, B., et al., "Versatile Video Coding (Draft 8)," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 512 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, Indian Application No. 202247060915, Indian Office Action dated Dec. 30, 2022, 6 pages. cited by applicant

Non-Final Office Action dated Mar. 13, 2023, 25 pages, U.S. Appl. No. 17/973,810, filed Oct. 26, 2022. cited by applicant

Notice of Allowance dated Mar. 17, 2023, 19 pages, U.S. Appl. No. 17/962,141, filed Oct. 7, 2022. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085009, English Translation of International Search Report dated Jun. 28, 2021, 11 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085033, English Translation of International Search Report dated Jul. 1, 2021, 11 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085771, English Translation of International Search Report dated Jul. 7, 2021, 12 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085773, English Translation of International Search Report dated Jun. 29, 2021, 11 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085772, English Translation of International Search Report dated Jul. 9, 2021, 13 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/085772, Corrected English Translation of International Search Report dated Aug. 10, 2021, 8 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/086110, English Translation of International Search Report dated May 27, 2021, 10 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/086111, English Translation of International Search Report dated Jul. 7, 2021, 12 pages. cited by applicant

Foreign Communication From a Related Counterpart Application, Indian Application No. 202247058853, English Translation of Indian Office

Action dated Dec. 23, 2022, 5 pages. cited by applicant
Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/086178, English Translation of International Search Report dated Jul. 7, 2021, 9 pages. cited by applicant
Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/087962, English Translation of International Search Report dated Jul. 19, 2021, 13 pages. cited by applicant
Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/CN2021/088287, English Translation of International Search Report dated Jun. 30, 2021, 13 pages. cited by applicant
Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/US2021/028974, English Translation of International Search Report dated Sep. 8, 2021, 21 pages. cited by applicant
Foreign Communication From a Related Counterpart Application, PCT Application No. PCT/US2021/028977, International Search Report dated Jul. 20, 2021, 22 pages. cited by applicant
Document: JVET-Q0420-v1, Li, L., et al., "AHG12: Signaling of chroma presence in PPS and APS," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 3 pages. cited by applicant
Document: JVET-R0072-v2, Deng, Z., et al., "AHG9: On signalling of deblocking control," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 18th Meeting: by teleconference, Apr. 15-24, 2020, 7 pages. cited by applicant
Extended European Search Report from European Application No. 21797509.3 dated Oct. 20, 2023, 17 pages. cited by applicant
Decision to Grant a Patent for Japanese Application No. 20220563488, mailed Oct. 1, 2024, 6 pages. cited by applicant
Document: JVET-Q0416-v2, Chen, J., et al., "AHG8/AHG9: On horizontal wrap-around motion compensation," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-7, 2020, 9 pages. cited by applicant
Document: JVET-P0127, He, Y., et al., "On subpicture wraparound motion compensation," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 16th Meeting: Geneva, CH, Oct. 1-11, 2019, 4 pages. cited by applicant
Chinese Office Action from Chinese Patent Application No. 202180024977.5 dated Nov. 6, 2024, 49 pages. cited by applicant
Chinese Office Action from Chinese Patent Application No. 202180027492.1 dated Nov. 9, 2024, 5 pages. cited by applicant
Notice of Allowance from U.S. Appl. No. 18/476,823 dated Nov. 15, 2024, 37 pages. cited by applicant
Notice of Allowance from U.S. Appl. No. 18/521,244 dated Nov. 20, 2024, 38 pages. cited by applicant
Paluri S., et al., "AHG9/AHG12: On Reference Picture Wraparound for Subpictures," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11, 18th Meeting: by Teleconference, Apr. 15-24, 2020, Document: JVET-R0184, 8 Pages. cited by applicant
Wang Y-K., et al., "The High-Level Syntax of the Versatile Video Coding (VVC) Standard," IEEE Transactions on Circuits and Systems for Video Technology, Oct. 4, 2021, vol. 31, No. 10, pp. 3779-3800, Published on Apr. 5, 2021, XP011880991. cited by applicant
Document: JVET-Q0210_r2, Hendry, et al., "Miscellaneous HLS clean-ups," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and Iso/Iec Jtc 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 8 pages. cited by applicant
Document: JVET-Q0484-v1, Wan, W., et al., "AHG9: HLS control of chroma QP offset," Joint Video Experts Team (JVET) of ITU-T SG 16 WP 3 and ISO/IEC JTC 1/SC 29/WG 11 17th Meeting: Brussels, BE, Jan. 7-17, 2020, 2 pages. cited by applicant
Chinese Notice of Allowance from Chinese Patent Application No. 2021800310805 dated Mar. 5, 2025, 6 pages. cited by applicant
Chinese Office Action from Chinese Patent Application No. 2021800270808 dated Mar. 21, 2025, 547 pages. cited by applicant
Chinese Notice of Allowance from Chinese Patent Application No. 2021800315334 dated Apr. 9, 2025, 12 pages. cited by applicant

Primary Examiner: Torrente; Richard T

Attorney, Agent or Firm: Conley Rose, P.C.

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS (1) This application is a continuation of U.S. application Ser. No. 17/962,882, filed on Oct. 10, 2022, which is a continuation of International Patent Application No. PCT/CN2021/086111, filed on Apr. 9, 2021, which claims the priority to and benefit of International Patent Application No. PCT/CN2020/083967, filed on Apr. 9, 2020. All the aforementioned patent applications are hereby incorporated by reference in their entireties.

TECHNICAL FIELD

(1) The present disclosure relates to image and video coding and decoding.

BACKGROUND

(2) Digital video accounts for the largest bandwidth use on the internet and other digital communication networks. As the number of connected user devices capable of receiving and displaying video increases, it is expected that the bandwidth demand for digital video usage will continue to grow.

SUMMARY

(3) The present disclosure discloses techniques that can be used by video encoders and decoders for processing coded representation of video using control information useful for decoding of the coded representation.

(4) In one example aspect, a video processing method is disclosed. The method includes performing a conversion between a video having one or more chroma components, the video comprising one or more video pictures comprising one or more slices and a coded representation of the video, wherein the coded representation conforms to a format rule, wherein the format rule specifies that a chroma array type field controls a constraint on a conversion characteristic of chroma used during the conversion.

(5) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video comprising one or more video pictures comprising one or more video regions and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies the include a deblocking mode indicator for a video region indicative of applicability of a deblocking filter to the video region during the conversion.

(6) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video comprising one or more video pictures comprising one or more video slices and/or one or more video subpictures and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies that a flag indicating whether a single slice per subpicture mode is deemed to be enabled for a video picture in case that a picture partitioning is disabled for the video picture.

(7) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video comprising one or more video pictures comprising one or more video slices and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies that a picture or a slice level chroma quantization parameter offset is signaled in a picture header or a slice header.

(8) In another example aspect, another video processing method is disclosed. The method includes: performing a conversion between a video comprising one or more video pictures comprising one or more video slices and a coded representation of the video, wherein the coded

representation conforms to a format rule that specifies that a chroma quantization parameter (QP) table applicable for conversion of a video block of the video is derived as an XOR operation between $(\text{delta_qp_in_val_minus1}[i][j]+1)$ and $\text{delta_qp_diff_val}[i][j]$, wherein $\text{delta_qp_in_val_minus1}[i][j]$ specifies a delta value used to derive the input coordinate of the j-th pivot point of the i-th chroma mapping table and $\text{delta_qp_diff_val}[i][j]$ specifies a delta value used to derive the output coordinate of the j-th pivot point of the i-th chroma QP mapping table, where i and j are integers.

(9) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first field in an adaptation parameter set that is based on a second field is derived using information included in a picture header or a slice header if a picture or a slice referring to the adaptation parameter set.

(10) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video region of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on syntax elements at i) a picture parameter set level and ii) a picture level or a slice level.

(11) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a current video block of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies to omit a first syntax element indicating whether the current video block is coded in a skip mode based on at least one of a second syntax element indicating an applicability of a first prediction mode or a block size of the current video block, wherein the first prediction is derived from blocks of sample values of a same slice of the current video block as determined by block vectors.

(12) In another example aspect, another video processing method is disclosed. The method includes performing a conversion between a video comprising one or more pictures and a bitstream of the video according to a format rule, wherein the format rule specifies a presence and/or a value of an indication indicating whether to partition a picture into a video region based on a number of coding tree blocks in the picture.

(13) In yet another example aspect, a video encoder apparatus is disclosed. The video encoder comprises a processor configured to implement above-described methods.

(14) In yet another example aspect, a video decoder apparatus is disclosed. The video decoder comprises a processor configured to implement above-described methods.

(15) In yet another example aspect, a computer readable medium having code stored thereon is disclosed. The code embodies one of the methods described herein in the form of processor-executable code.

(16) These, and other, features are described throughout the present disclosure.

Description

BRIEF DESCRIPTION OF DRAWINGS

- (1) FIG. 1 is a block diagram of an example video processing system.
- (2) FIG. 2 is a block diagram of a video processing apparatus.
- (3) FIG. 3 is a flowchart for an example method of video processing.
- (4) FIG. 4 is a block diagram that illustrates a video coding system in accordance with some embodiments of the present disclosure.
- (5) FIG. 5 is a block diagram that illustrates an encoder in accordance with some embodiments of the present disclosure.
- (6) FIG. 6 is a block diagram that illustrates a decoder in accordance with some embodiments of the present disclosure.
- (7) FIG. 7 shows flowcharts for example methods of video processing based on some implementations of the disclosed technology.
- (8) FIGS. 8A to 8C show flowcharts for example methods of video processing based on some implementations of the disclosed technology.

DETAILED DESCRIPTION

(9) Section headings are used in the present disclosure for ease of understanding and do not limit the applicability of techniques and embodiments disclosed in each section only to that section. Furthermore, H.266 terminology is used in some description only for ease of understanding and not for limiting scope of the disclosed techniques. As such, the techniques described herein are applicable to other video codec protocols and designs also.

1. Introduction

(10) This disclosure is related to video coding technologies. Specifically, it is about the syntax design of adaptation parameter set (APS), deblocking, subpicture, and quantization parameter (QP) delta in video coding. The ideas may be applied individually or in various combination, to any video coding standard or non-standard video codec that supports multi-layer video coding, e.g., the being-developed Versatile Video Coding (VVC).

2. Abbreviations

(11) APS Adaptation Parameter Set AU Access Unit AUD Access Unit Delimiter AVC Advanced Video Coding CLVS Coded Layer Video Sequence CPB Coded Picture Buffer CRA Clean Random Access CTU Coding Tree Unit CVS Coded Video Sequence DPB Decoded Picture Buffer DPS Decoding Parameter Set EOB End Of Bitstream EOS End Of Sequence GDR Gradual Decoding Refresh HEVC High Efficiency Video Coding HRD Hypothetical Reference Decoder ID Identifier IDR Instantaneous Decoding Refresh JEM Joint Exploration Model MCTS Motion-Constrained Tile Sets NAL Network Abstraction Layer OLS Output Layer Set PH Picture Header PPS Picture Parameter Set PROF Prediction Refinement with Optical Flow PTL Profile, Tier and Level PU Picture Unit RBSP Raw Byte Sequence Payload SEI Supplemental Enhancement Information SH Slice Header SPS Sequence Parameter Set SVC Scalable Video Coding VCL Video Coding Layer VPS Video Parameter Set VTM VVC Test Model VUI Video Usability Information VVC Versatile Video Coding

3. Initial Discussion

(12) Video coding standards have evolved primarily through the development of the well-known International Telecommunication Union-Telecommunication Standardization Sector (ITU-T) and International Organization for Standardization (ISO)/International Electrotechnical Commission (IEC) standards. The ITU-T produced H.261 and H.263, ISO/IEC produced Moving Picture Experts Group (MPEG)-1 and MPEG-4 Visual, and the two organizations jointly produced the H.262/MPEG-2 Video and H.264/MPEG-4 Advanced Video Coding (AVC) and H.265/High Efficiency Video Coding (HEVC) standards. Since H.262, the video coding standards are based on the hybrid video coding structure wherein temporal prediction plus transform coding are utilized. To explore the future video coding technologies beyond HEVC, the Joint Video Exploration Team (JVET) was founded by Video Coding Experts Group (VCEG) and MPEG jointly in 2015. Since then, many new methods have been adopted by JVET and put into the reference software named Joint Exploration Model (JEM). The JVET meeting is concurrently held once every quarter, and the new coding standard is targeting at 50% bitrate reduction as compared to HEVC. The new video coding standard was officially named as Versatile Video Coding (VVC) in the April 2018 JVET meeting, and the first version of VVC test model (VTM) was released at that time. As there are continuous effort contributing to VVC standardization, new coding techniques are being adopted to the VVC standard in every JVET meeting. The VVC working draft and test model VTM are then updated after every meeting. The VVC project is now aiming for technical completion, Final Draft International Standard (FDIS), at the July 2020 meeting.

3.1. PPS Syntax and Semantics

(13) In the latest VVC draft text, the picture parameter set (PPS) syntax and semantics are as follows:

(14) TABLE-US-00001 Descriptor `pic_parameter_set_rbsp( )` { `pps_pic_parameter_set_id` ue(v) `pps_seq_parameter_set_id` u(4) `mixed_nalu_types_in_pic_flag` u(1) `pic_width_in_luma_samples` ue(v) `pic_height_in_luma_samples` ue(v) `pps_conformance_window_flag` u(1) `if( pps_conformance_window_flag )` { `pps_conf_win_left_offset` ue(v) `pps_conf_win_right_offset` ue(v) `pps_conf_win_top_offset` ue(v) `pps_conf_win_bottom_offset` ue(v) } `scaling_window_explicit_signalling_flag` u(1) `if`

```

scaling_win_left_offset_flag ) { scaling_win_left_offset ue(v) scaling_win_right_offset ue(v) scaling_win_top_offset
ue(v) scaling_win_bottom_offset ue(v) } output_flag_present_flag u(1) subpic_id_mapping_in_pps_flag u(1) if(
subpic_id_mapping_in_pps_flag ) { pps_num_subpics_minus1 ue(v) pps_subpic_id_len_minus1 ue(v) for( i = 0; i <=
pps_num_subpic_minus1; i++ ) pps_subpic_id[ i ] ue(v) } no_pic_partition_flag u(1) if( !no_pic_partition_flag ) {
pps_log2_ctu_size_minus5 u(2) num_exp_tile_columns_minus1 ue(v) num_exp_tile_rows_minus1 ue(v) for( i = 0; i <=
num_exp_tile_columns_minus1; i++ ) tile_column_width_minus1[ i ] ue(v) for( i = 0; i <= num_exp_tile_rows_minus1; i++ )
tile_row_height_minus1[ i ] ue(v) if( NumTilesInPic > 1 ) rect_slice_flag u(1) if( rect_slice_flag )
single_slice_per_subpic_flag u(1) if( rect_slice_flag && !single_slice_per_subpic_flag ) { num_slices_in_pic_minus1 ue(v) if(
num_slices_in_pic_minus1 > 0 ) tile_idx_delta_present_flag u(1) for( i = 0; i < num_slices_in_pic_minus1; i++ ) { if(
NumTileColumns > 1 ) slice_width_in_tiles_minus1[ i ] ue(v) if( NumTileRows > 1 && (
tile_idx_delta_present_flag || tileIdx % NumTileColumns = 0 ) ) slice_height_in_tiles_minus1[ i ] ue(v) if(
slice_width_in_tiles_minus1[ i ] = 0 && slice_height_in_tiles_minus1[ i ] = 0 && RowHeight[
SliceTopLeftTileIdx[ i ] / NumTileColumns ] > 1 ) { num_exp_slices_in_tile[ i ] ue(v) for( j = 0; j <
num_exp_slices_in_tile[ i ]; j++ ) exp_slice_height_in_ctus_minus1[ j ] ue(v) i += NumSlicesInTile[ i ] - 1 }
if( tile_idx_delta_present_flag && i < num_slices_in_pic_minus1 ) tile_idx_delta[ i ] se(v) } }
loop_filter_across_tiles_enabled_flag u(1) loop_filter_across_slices_enabled_flag u(1) } cabac_init_present_flag u(1) for( i = 0; i < 2;
i++ ) num_ref_idx_default_active_minus1[ i ] ue(v) rpl1_idx_present_flag u(1) init_qp_minus26 se(v) cu_qp_delta_enabled_flag u(1)
pps_chroma_tool_offsets_present_flag u(1) if( pps_chroma_tool_offsets_present_flag ) { pps_cb_qp_offset se(v) pps_cr_qp_offset
se(v) pps_joint_cbr_qp_offset_present_flag u(1) if( pps_joint_cbr_qp_offset_present_flag ) pps_joint_cbr_qp_offset_value se(v)
pps_slice_chroma_qp_offsets_present_flag u(1) pps_cu_chroma_qp_offset_list_enabled_flag u(1) } if(
pps_cu_chroma_qp_offset_list_enabled_flag ) { chroma_qp_offset_list_len_minus1 ue(v) for( i = 0; i <=
chroma_qp_offset_list_len_minus1; i++ ) { cb_qp_offset_list[ i ] se(v) cr_qp_offset_list[ i ] se(v) if(
pps_joint_cbr_qp_offset_present_flag ) joint_cbr_qp_offset_list[ i ] se(v) } } pps_weighted_pred_flag u(1)
pps_weighted_bipred_flag u(1) deblocking_filter_control_present_flag u(1) if( deblocking_filter_control_present_flag ) {
deblocking_filter_override_enabled_flag u(1) pps_deblocking_filter_disabled_flag u(1) if( !pps_deblocking_filter_disabled_flag ) {
pps_beta_offset_div2 se(v) pps_tc_offset_div2 se(v) pps_cb_beta_offset_div2 se(v) pps_cb_tc_offset_div2 se(v)
pps_cr_beta_offset_div2 se(v) pps_cr_tc_offset_div2 se(v) } } rpl_info_in_ph_flag u(1) if(
deblocking_filter_override_enabled_flag ) dbf_info_in_ph_flag u(1) sao_info_in_ph_flag u(1) alf_info_in_ph_flag u(1) if( (
pps_weighted_pred_flag || pps_weighted_bipred_flag ) && rpl_info_in_ph_flag ) wp_info_in_ph_flag u(1) qp_delta_info_in_ph_flag u(1)
pps_ref_wraparound_enabled_flag u(1) if( pps_ref_wraparound_enabled_flag ) pps_ref_wraparound_offset ue(v)
picture_header_extension_present_flag u(1) slice_header_extension_present_flag u(1) pps_extension_flag u(1) if( pps_extension_flag )
while( more_rbsp_data( ) ) pps_extension_data_flag u(1) rbsp_trailing_bits( ) }

```

(15) A PPS raw byte sequence payload (RBSP) shall be available to the decoding process prior to it being referenced, included in at least one AU with TemporalId less than or equal to the TemporalId of the PPS NAL unit or provided through external means.

(16) All PPS NAL units with a particular value of pps_pic_parameter_set_id within a picture unit (PU) shall have the same content.

(17) pps_pic_parameter_set_id identifies the PPS for reference by other syntax elements. The value of pps_pic_parameter_set_id shall be in the range of 0 to 63, inclusive.

(18) PPS NAL units, regardless of the nuh_layer_id values, share the same value space of pps_pic_parameter_set_id. Let ppsLayerId be the value of the nuh_layer_id of a particular PPS NAL unit, and vclLayerId be the value of the nuh_layer_id of a particular video coding layer (VCL) NAL unit. The particular VCL NAL unit shall not refer to the particular PPS NAL unit unless ppsLayerId is less than or equal to vclLayerId and the layer with nuh_layer_id equal to ppsLayerId is included in at least one OLS that includes the layer with nuh_layer_id equal to vclLayerId.

(19) pps_seq_parameter_set_id specifies the value of sps_seq_parameter_set_id for the sequence parameter set (SPS). The value of pps_seq_parameter_set_id shall be in the range of 0 to 15, inclusive. The value of pps_seq_parameter_set_id shall be the same in all PPSs that are referred to by coded pictures in a coded layer video sequence (CLVS).

(20) mixed_nalu_types_in_pic_flag equal to 1 specifies that each picture referring to the PPS has more than one VCL NAL unit, the VCL NAL units do not have the same value of nal_unit_type, and the picture is not an intra random access picture (IRAP) picture. mixed_nalu_types_in_pic_flag equal to 0 specifies that each picture referring to the PPS has one or more VCL NAL units and the VCL NAL units of each picture referring to the PPS have the same value of nal_unit_type.

(21) When no_mixed_nalu_types_in_pic_constraint_flag is equal to 1, the value of mixed_nalu_types_in_pic_flag shall be equal to 0.

(22) For each slice with a nal_unit_type value nalUnitTypeA in the range of instantaneous decoding refresh (IDR) with random access decodable leading (RADL) (IDR_W_RADL) to clean random access (CRA) network abstraction layer (NAL) unit type (CRA_NUT), inclusive, in a picture picA that also contains one or more slices with another value of nal_unit_type (i.e., the value of mixed_nalu_types_in_pic_flag for the picture picA is equal to 1), the following applies: The slice shall belong to a subpicture subpicA for which the value of the corresponding subpic_treated_as_pic_flag[i] is equal to 1. The slice shall not belong to a subpicture of picA containing VCL NAL units with nal_unit_type not equal to nalUnitTypeA. If nalUnitTypeA is equal to CRA, for all the following PUs following the current picture in the CLVS in decoding order and in output order, neither RefPicList[0] nor RefPicList[1] of a slice in subpicA in those PUs shall include any picture preceding picA in decoding order in an active entry. Otherwise (i.e., nalUnitTypeA is equal to IDR_W_RADL or IDR_N_LP), for all the PUs in the CLVS following the current picture in decoding order, neither RefPicList[0] nor RefPicList[1] of a slice in subpicA in those PUs shall include any picture preceding picA in decoding order in an active entry. NOTE 1-mixed_nalu_types_in_pic_flag equal to 1 indicates that pictures referring to the PPS contain slices with different NAL unit types, e.g., coded pictures originating from a subpicture bitstream merging operation for which encoders have to ensure matching bitstream structure and further alignment of parameters of the original bitstreams. One example of such alignments is as follows: When the value of sps_idr_rpl_flag is equal to 0 and mixed_nalu_types_in_pic_flag is equal to 1, a picture referring to the PPS cannot have slices with nal_unit_type equal to IDR_W_RADL or IDR_N_LP.

(23) pic_width_in_luma_samples specifies the width of each decoded picture referring to the PPS in units of luma samples.

pic_width_in_luma_samples shall not be equal to 0, shall be an integer multiple of Max(8, MinCbSizeY), and shall be less than or equal to pic_width_max_in_luma_samples.

(24) When res_change_in_clvs_allowed_flag equal to 0, the value of pic_width_in_luma_samples shall be equal to pic_width_max_in_luma_samples.

(25) pic_height_in_luma_samples specifies the height of each decoded picture referring to the PPS in units of luma samples.

pic_height_in_luma_samples shall not be equal to 0 and shall be an integer multiple of Max(8, MinCbSizeY), and shall be less than or equal to pic_height_max_in_luma_samples.

(26) When res_change_in_clvs_allowed_flag equal to 0, the value of pic_height_in_luma_samples shall be equal to pic_height_max_in_luma_samples.

(27) The variables PicWidthInCtbsY, PicHeightInCtbsY, PicSizeInCtbsY, PicWidthInMinCtbsY, PicHeightInMinCtbsY, PicSizeInMinCtbsY, PicSizeInSamplesY, PicWidthInSamplesC and PicHeightInSamplesC are derived as follows:

$\text{PicWidthInCtbsY} = \text{Ceil}(\text{pic_width_in_luma_samples} - \text{CtbSizeY})$ (69)
 $\text{PicHeightInCtbsY} = \text{Ceil}(\text{pic_height_in_luma_samples} - \text{CtbSizeY})$ (70)
 $\text{PicSizeInCtbsY} = \text{PicWidthInCtbsY} * \text{PicHeightInCtbsY}$ (71)
 $\text{PicWidthInMinCtbsY} = \text{pic_width_in_luma_samples} / \text{MinCbSizeY}$ (72)
 $\text{PicHeightInMinCtbsY} = \text{pic_height_in_luma_samples} / \text{MinCbSizeY}$ (73)
 $\text{PicSizeInMinCtbsY} = \text{PicWidthInMinCtbsY} * \text{PicHeightInMinCtbsY}$ (74)
 $\text{PicSizeInSamplesY} = \text{pic_width_in_luma_samples} * \text{pic_height_in_luma_samples}$ (75)
 $\text{PicWidthInSamplesC} = \text{pic_width_in_luma_samples} / \text{SubWidthC}$ (76)
 $\text{PicHeightInSamplesC} = \text{pic_height_in_luma_samples} / \text{SubHeightC}$ (77)

pps_conformance_window_flag equal to 1 indicates that the conformance cropping window offset parameters follow next in the PPS.
 pps_conformance_window_flag equal to 0 indicates that the conformance cropping window offset parameters are not present in the PPS.

(28) pps_conf_win_left_offset, pps_conf_win_right_offset, pps_conf_win_top_offset, and pps_conf_win_bottom_offset specify the samples of the pictures in the CLVS that are output from the decoding process, in terms of a rectangular region specified in picture coordinates for output. When pps_conformance_window_flag is equal to 0, the values of pps_conf_win_left_offset, pps_conf_win_right_offset, pps_conf_win_top_offset, and pps_conf_win_bottom_offset are inferred to be equal to 0. The conformance cropping window contains the luma samples with horizontal picture coordinates from $\text{SubWidthC} * \text{pps_conf_win_left_offset}$ to $\text{pic_width_in_luma_samples} - (\text{SubWidthC} * \text{pps_conf_win_right_offset} + 1)$ and vertical picture coordinates from $\text{SubHeightC} * \text{pps_conf_win_top_offset}$ to $\text{pic_height_in_luma_samples} - (\text{SubHeightC} * \text{pps_conf_win_bottom_offset} + 1)$, inclusive.

(29) The value of $\text{SubWidthC} * (\text{pps_conf_win_left_offset} + \text{pps_conf_win_right_offset})$ shall be less than $\text{pic_width_in_luma_samples}$, and the value of $\text{SubHeightC} * (\text{pps_conf_win_top_offset} + \text{pps_conf_win_bottom_offset})$ shall be less than $\text{pic_height_in_luma_samples}$.

(30) When ChromaArrayType is not equal to 0, the corresponding specified samples of the two chroma arrays are the samples having picture coordinates $(x/\text{SubWidthC}, y/\text{SubHeightC})$, where (x, y) are the picture coordinates of the specified luma samples. NOTE 2—The conformance cropping window offset parameters are only applied at the output. All internal decoding processes are applied to the uncropped picture size.

(31) Let ppsA and ppsB be any two PPSs referring to the same SPS. It is a requirement of bitstream conformance that, when ppsA and ppsB have the same the values of $\text{pic_width_in_luma_samples}$ and $\text{pic_height_in_luma_samples}$, respectively, ppsA and ppsB shall have the same values of pps_conf_win_left_offset, pps_conf_win_right_offset, pps_conf_win_top_offset, and pps_conf_win_bottom_offset, respectively.

(32) When $\text{pic_width_in_luma_samples}$ is equal to $\text{pic_width_max_in_luma_samples}$ and $\text{pic_height_in_luma_samples}$ is equal to $\text{pic_height_max_in_luma_samples}$, it is a requirement of bitstream conformance that pps_conf_win_left_offset, pps_conf_win_right_offset, pps_conf_win_top_offset, and pps_conf_win_bottom_offset, are equal to sps_conf_win_left_offset, sps_conf_win_right_offset, sps_conf_win_top_offset, and sps_conf_win_bottom_offset, respectively.

(33) scaling_window_explicit_signalling_flag equal to 1 specifies that the scaling window offset parameters are present in the PPS.
 scaling_window_explicit_signalling_flag equal to 0 specifies that the scaling window offset parameters are not present in the PPS. When res_change_in_clvs_allowed_flag is equal to 0, the value of scaling_window_explicit_signalling_flag shall be equal to 0.

(34) scaling_win_left_offset, scaling_win_right_offset, scaling_win_top_offset, and scaling_win_bottom_offset specify the offsets that are applied to the picture size for scaling ratio calculation. When not present, the values of scaling_win_left_offset, scaling_win_right_offset, scaling_win_top_offset, and scaling_win_bottom_offset are inferred to be equal to pps_conf_win_left_offset, pps_conf_win_right_offset, pps_conf_win_top_offset, and pps_conf_win_bottom_offset, respectively.

(35) The value of $\text{SubWidthC} * (\text{scaling_win_left_offset} + \text{scaling_win_right_offset})$ shall be less than $\text{pic_width_in_luma_samples}$, and the value of $\text{SubHeightC} * (\text{scaling_win_top_offset} + \text{scaling_win_bottom_offset})$ shall be less than $\text{pic_height_in_luma_samples}$.

(36) The variables PicOutputWidthL and PicOutputHeightL are derived as follows:
 $\text{PicOutputWidthL} = \text{pic_width_in_luma_samples} - \text{SubWidthC} * (\text{scaling_win_right_offset} + \text{scaling_win_left_offset})$ (78)
 $\text{PicOutputHeightL} = \text{pic_height_in_luma_samples} - \text{SubHeightC} * (\text{scaling_win_bottom_offset} + \text{scaling_win_top_offset})$ (79)

(37) Let refPicOutputWidthL and refPicOutputHeightL be the PicOutputWidthL and PicOutputHeightL, respectively, of a reference picture of a current picture referring to this PPS. Is a requirement of bitstream conformance that all of the following conditions are satisfied: $\text{PicOutputWidthL} * 2$ shall be greater than or equal to $\text{refPicWidthInLumaSamples}$. $\text{PicOutputHeightL} * 2$ shall be greater than or equal to $\text{refPicHeightInLumaSamples}$. PicOutputWidthL shall be less than or equal to $\text{refPicWidthInLumaSamples} * 8$. PicOutputHeightL shall be less than or equal to $\text{refPicHeightInLumaSamples} * 8$. $\text{PicOutputWidthL} * \text{pic_width_max_in_luma_samples}$ shall be greater than or equal to $\text{refPicOutputWidthL} * (\text{pic_width_in_luma_samples} - \text{Max}(8, \text{MinCbSizeY}))$. $\text{PicOutputHeightL} * \text{pic_height_max_in_luma_samples}$ shall be greater than or equal to $\text{refPicOutputHeightL} * (\text{pic_height_in_luma_samples} - \text{Max}(8, \text{MinCbSizeY}))$.

output_flag_present_flag equal to 1 indicates that the pic_output_flag syntax element is present in slice headers referring to the PPS.
 output_flag_present_flag equal to 0 indicates that the pic_output_flag syntax element is not present in slice headers referring to the PPS.

(38) subpic_id_mapping_in_pps_flag equal to 1 specifies that the subpicture ID mapping is signalled in the PPS. subpic_id_mapping_in_pps_flag equal to 0 specifies that the subpicture ID mapping is not signalled in the PPS. If subpic_id_mapping_explicitly_signalled_flag is 0 or subpic_id_mapping_in_sps_flag is equal to 1, the value of subpic_id_mapping_in_pps_flag shall be equal to 0. Otherwise (subpic_id_mapping_explicitly_signalled_flag is equal to 1 and subpic_id_mapping_in_sps_flag is equal to 0), the value of subpic_id_mapping_in_pps_flag shall be equal to 1.

(39) pps_num_subpics_minus1 shall be equal to sps_num_subpics_minus1.

(40) pps_subpic_id_len_minus1 shall be equal to sps_subpic_id_len_minus1.

(41) pps_subpic_id[i] specifies the subpicture ID of the i-th subpicture. The length of the pps_subpic_id[i] syntax element is $\text{pps_subpic_id_len_minus1} + 1$ bits.

(42) The variable SubpicIdVal[i], for each value of i in the range of 0 to $\text{sps_num_subpics_minus1}$, inclusive, is derived as follows:
 for $(i=0; i \leq \text{sps_num_subpics_minus1}; i++)$ if (subpic_id_mapping_explicitly_signalled_flag) SubpicIdVal[i] = subpic_id_mapping_in_pps_flag?
 $\text{pps_subpic_id}[i] : \text{sps_subpic_id}[i]$ else SubpicIdVal[i] = i (80)

(43) It is a requirement of bitstream conformance that both of the following constraints apply: For any two differently values of i and j in the range of 0 to $\text{sps_num_subpics_minus1}$, inclusive, SubpicIdVal[i] shall not be equal to SubpicIdVal[j]. When the current picture is not the first picture of the CLVS, for each value of i in the range of 0 to $\text{sps_num_subpics_minus1}$, inclusive, if the value of SubpicIdVal[i] is not equal to the value of SubpicIdVal[i] of the previous picture in decoding order in the same layer, the nal_unit_type for all coded slice NAL units of the subpicture in the current picture with subpicture index i shall be equal to a particular value in the range of IDR_W_RADL to CRA_NUT, inclusive.

(44) no_pic_partition_flag equal to 1 specifies that no picture partitioning is applied to each picture referring to the PPS. no_pic_partition_flag equal to 0 specifies each picture referring to the PPS may be partitioned into more than one tile or slice.

(45) It is a requirement of bitstream conformance that the value of no_pic_partition_flag shall be the same for all PPSs that are referred to by coded pictures within a CLVS.

(46) It is a requirement of bitstream conformance that the value of no_pic_partition_flag shall not be equal to 1 when the value of $\text{sps_num_subpics_minus1} + 1$ is greater than 1.

(47) pps_log2_ctu_size_minus5 plus 5 specifies the luma coding tree block size of each CTU. pps_log2_ctu_size_minus5 shall be equal to sps_log

2_ctu_size_minus5.

(48) num_exp_tile_columns_minus1 plus 1 specifies the number of explicitly provided tile column widths. The value of num_exp_tile_columns_minus1 shall be in the range of 0 to PicWidthInCtbsY-1, inclusive. When no_pic_partition_flag is equal to 1, the value of num_exp_tile_columns_minus1 is inferred to be equal to 0.

(49) num_exp_tile_rows_minus1 plus 1 specifies the number of explicitly provided tile row heights. The value of num_exp_tile_rows_minus1 shall be in the range of 0 to PicHeightInCtbsY-1, inclusive. When no_pic_partition_flag is equal to 1, the value of num_exp_tile_rows_minus1 is inferred to be equal to 0.

(50) tile_column_width_minus1[i] plus 1 specifies the width of the i-th tile column in units of coding tree blocks (CTBs) for i in the range of 0 to num_exp_tile_columns_minus1-1, inclusive. tile_column_width_minus1[num_exp_tile_columns_minus1] is used to derive the width of the tile columns with index greater than or equal to num_exp_tile_columns_minus1 as specified in clause 6.5.1. The value of tile_column_width_minus1[i] shall be in the range of 0 to PicWidthInCtbsY-1, inclusive. When not present, the value of tile_column_width_minus1[0] is inferred to be equal to PicWidthInCtbsY-1.

(51) tile_row_height_minus1[i] plus 1 specifies the height of the i-th tile row in units of CTBs for i in the range of 0 to num_exp_tile_rows_minus1-1, inclusive. tile_row_height_minus1[num_exp_tile_rows_minus1] is used to derive the height of the tile rows with index greater than or equal to num_exp_tile_rows_minus1 as specified in clause 6.5.1. The value of tile_row_height_minus1[i] shall be in the range of 0 to PicHeightInCtbsY-1, inclusive. When not present, the value of tile_row_height_minus1[0] is inferred to be equal to PicHeightInCtbsY-1.

(52) rect_slice_flag equal to 0 specifies that tiles within each slice are in raster scan order and the slice information is not signalled in PPS.

rect_slice_flag equal to 1 specifies that tiles within each slice cover a rectangular region of the picture and the slice information is signalled in the PPS. When not present, rect_slice_flag is inferred to be equal to 1. When subpic_info_present_flag is equal to 1, the value of rect_slice_flag shall be equal to 1.

(53) single_slice_per_subpic_flag equal to 1 specifies that each subpicture consists of one and only one rectangular slice.

single_slice_per_subpic_flag equal to 0 specifies that each subpicture may consist of one or more rectangular slices. When single_slice_per_subpic_flag is equal to 1, num_slices_in_pic_minus1 is inferred to be equal to sps_num_subpics_minus1. When not present, the value of single_slice_per_subpic_flag is inferred to be equal to 0.

(54) num_slices_in_pic_minus1 plus 1 specifies the number of rectangular slices in each picture referring to the PPS. The value of num_slices_in_pic_minus1 shall be in the range of 0 to MaxSlicesPerPicture-1, inclusive, where MaxSlicesPerPicture is specified in Annex A. When no_pic_partition_flag is equal to 1, the value of num_slices_in_pic_minus1 is inferred to be equal to 0.

(55) tile_idx_delta_present_flag equal to 0 specifies that tile_idx_delta values are not present in the PPS and all rectangular slices in pictures referring to the PPS are specified in raster order according to the process defined in clause 6.5.1. tile_idx_delta_present_flag equal to 1 specifies that tile_idx_delta values may be present in the PPS and all rectangular slices in pictures referring to the PPS are specified in the order indicated by the values of tile_idx_delta. When not present, the value of tile_idx_delta_present_flag is inferred to be equal to 0.

(56) slice_width_in_tiles_minus1[i] plus 1 specifies the width of the i-th rectangular slice in units of tile columns. The value of slice_width_in_tiles_minus1[i] shall be in the range of 0 to NumTileColumns-1, inclusive.

(57) When slice_width_in_tiles_minus1[i] is not present, the following applies: If NumTileColumns is equal to 1, the value of slice_width_in_tiles_minus1[i] is inferred to be equal to 0. Otherwise, the value of slice_width_in_tiles_minus1[i] is inferred as specified in clause 6.5.1.

(58) slice_height_in_tiles_minus1[i] plus 1 specifies the height of the i-th rectangular slice in units of tile rows. The value of slice_height_in_tiles_minus1[i] shall be in the range of 0 to NumTileRows-1, inclusive.

(59) When slice_height_in_tiles_minus1[i] is not present, the following applies: If NumTileRows is equal to 1, or tile_idx_delta_present_flag is equal to 0 and tileIdx % NumTileColumns is greater than 0, the value of slice_height_in_tiles_minus1[i] is inferred to be equal to 0. Otherwise (NumTileRows is not equal to 1, and tile_idx_delta_present_flag is equal to 1 or tileIdx % NumTileColumns is equal to 0), when tile_idx_delta_present_flag is equal to 1 or tileIdx % NumTileColumns is equal to 0, the value of slice_height_in_tiles_minus1[i] is inferred to be equal to slice_height_in_tiles_minus1[i-1].

(60) num_exp_slices_in_tile[i] specifies the number of explicitly provided slice heights in the current tile that contains more than one rectangular slices. The value of num_exp_slices_in_tile[i] shall be in the range of 0 to RowHeight[tileY]-1, inclusive, where tileY is the tile row index containing the i-th slice. When not present, the value of num_exp_slices_in_tile[i] is inferred to be equal to 0. When num_exp_slices_in_tile[i] is equal to 0, the value of the variable NumSlicesInTile[i] is derived to be equal to 1.

(61) exp_slice_height_in_ctus_minus1[j] plus 1 specifies the height of the j-th rectangular slice in the current tile in units of CTU rows. The value of exp_slice_height_in_ctus_minus1[j] shall be in the range of 0 to RowHeight[tileY]-1, inclusive, where tile Y is the tile row index of the current tile.

(62) When num_exp_slices_in_tile[i] is greater than 0, the variable NumSlicesInTile[i] and SliceHeightInCtusMinus1[i+k] for k in the range of 0 to NumSlicesInTile[i]-1 are derived as follows:

```
remainingHeightInCtbsY=RowHeight[SliceTopLeftTileIdx[i]/NumTileColumns]numExpSliceInTile=num_exp_slices_in_tile[i]
for(j=0;j<numExpSliceInTile-1;j++){SliceHeightInCtusMinus1[i+j]=exp_slice_height_in_ctu_minus1[j]remainingHeightInCtbsY-
=SliceHeightInCtusMinus1[j]}uniformSliceHeightMinus1=SliceHeightInCtusMinus1[i-1]while(remainingHeightInCtbsY>=
(uniformSliceHeightMinus1+1)){SliceHeightInCtusMinus1[i+j]=uniformSliceHeightMinus1 remainingHeightInCtbsY=
(uniformSliceHeightMinus1+1)j++} if(remainingHeightInCtbsY>0){SliceHeightInCtusMinus1[i+j]=remainingHeightInCtbsY
j++}NumSlicesInTile[i]=j      (81)
```

(63) tile_idx_delta[i] specifies the difference between the tile index of the first tile in the i-th rectangular slice and the tile index of the first tile in the (i+1)-th rectangular slice. The value of tile_idx_delta[i] shall be in the range of -NumTilesInPic+1 to NumTilesInPic-1, inclusive. When not present, the value of tile_idx_delta[i] is inferred to be equal to 0. When present, the value of tile_idx_delta[i] shall not be equal to 0.

(64) loop_filter_across_tiles_enabled_flag equal to 1 specifies that in-loop filtering operations may be performed across tile boundaries in pictures referring to the PPS. loop_filter_across_tiles_enabled_flag equal to 0 specifies that in-loop filtering operations are not performed across tile boundaries in pictures referring to the PPS. The in-loop filtering operations include the deblocking filter, sample adaptive offset filter, and adaptive loop filter operations. When not present, the value of loop_filter_across_tiles_enabled_flag is inferred to be equal to 1.

(65) loop_filter_across_slices_enabled_flag equal to 1 specifies that in-loop filtering operations may be performed across slice boundaries in pictures referring to the PPS. loop_filter_across_slice_enabled_flag equal to 0 specifies that in-loop filtering operations are not performed across slice boundaries in pictures referring to the PPS. The in-loop filtering operations include the deblocking filter, sample adaptive offset filter, and adaptive loop filter operations. When not present, the value of loop_filter_across_slices_enabled_flag is inferred to be equal to 0.

(66) cabac_init_present_flag equal to 1 specifies that cabac_init_flag is present in slice headers referring to the PPS. cabac_init_present_flag equal to 0 specifies that cabac_init_flag is not present in slice headers referring to the PPS. num_ref_idx_default_active_minus1[i] plus 1, when i is equal to 0, specifies the inferred value of the variable NumRefIdxActive[0] for P or B slices with num_ref_idx_active_override_flag equal to 0, and, when i is equal to 1, specifies the inferred value of NumRefIdxActive[1] for B slices with num_ref_idx_active_override_flag equal to 0. The value of num_ref_idx_default_active_minus1[i] shall be in the range of 0 to 14, inclusive.

(67) rpl1_idx_present_flag equal to 0 specifies that ref_pic_list_sps_flag[1] and ref_pic_list_idx[1] are not present in the PH syntax structures or the slice headers for pictures referring to the PPS. rpl1_idx_present_flag equal to 1 specifies that ref_pic_list_sps_flag[1] and ref_pic_list_idx[1] may be

present in the PH syntax structures or the slice headers for pictures referring to the PPS.

(68) `init_qp_minus26` plus 26 specifies the initial value of `SliceQp.sub.Y` for each slice referring to the PPS. The initial value of `SliceQp.sub.Y` is modified at the picture level when a non-zero value of `ph_qp_delta` is decoded or at the slice level when a non-zero value of `slice_qp_delta` is decoded. The value of `init_qp_minus26` shall be in the range of $-(26+QpBdOffset)$ to +37, inclusive.

(69) `cu_qp_delta_enabled_flag` equal 1 specifies that the `ph_cu_qp_delta_subdiv_intra_slice` and to `ph_cu_qp_delta_subdiv_inter_slice` syntax elements are present in PHs referring to the PPS and `cu_qp_delta_abs` may be present in the transform unit syntax. `cu_qp_delta_enabled_flag` equal to 0 specifies that the `ph_cu_qp_delta_subdiv_intra_slice` and `ph_cu_qp_delta_subdiv_inter_slice` syntax elements are not present in PHs referring to the PPS and `cu_qp_delta_abs` is not present in the transform unit syntax.

(70) `pps_chroma_tool_offsets_present_flag` equal to 1 specifies that chroma tool offsets related syntax elements are present in the PPS RBSP syntax structure. `pps_chroma_tool_offsets_present_flag` equal to 0 specifies that chroma tool offsets related syntax elements are not present in the PPS RBSP syntax structure. When `ChromaArrayType` is equal to 0, the value of `pps_chroma_tool_offsets_present_flag` shall be equal to 0.

(71) `pps_cb_qp_offset` and `pps_cr_qp_offset` specify the offsets to the luma quantization parameter `Qp'.sub.Y` used for deriving `Qp'.sub.Cb` and `Qp'.sub.Cr`, respectively. The values of `pps_cb_qp_offset` and `pps_cr_qp_offset` shall be in the range of -12 to +12, inclusive. When `ChromaArrayType` is equal to 0, `pps_cb_qp_offset` and `pps_cr_qp_offset` are not used in the decoding process and decoders shall ignore their value.

When not present, the values of `pps_cb_qp_offset` and `pps_cr_qp_offset` are inferred to be equal to 0.

(72) `pps_joint_cbr_qp_offset_present_flag` equal to 1 specifies that `pps_joint_cbr_qp_offset_value` and `joint_cbr_qp_offset_list[i]` are present in the PPS RBSP syntax structure. `pps_joint_cbr_qp_offset_present_flag` equal to 0 specifies that `pps_joint_cbr_qp_offset_value` and `joint_cbr_qp_offset_list[i]` are not present in the PPS RBSP syntax structure. When `ChromaArrayType` is equal to 0 or `sps_joint_cbr_enabled_flag` is equal to 0, the value of `pps_joint_cbr_qp_offset_present_flag` shall be equal to 0. When not present, the value of `pps_joint_cbr_qp_offset_present_flag` is inferred to be equal to 0. `pps_joint_cbr_qp_offset_value` specifies the offset to the luma quantization parameter `Qp'.sub.Y` used for deriving `Qp'.sub.CbCr`. The value of `pps_joint_cbr_qp_offset_value` shall be in the range of -12 to +12, inclusive. When `ChromaArrayType` is equal to 0 or `sps_joint_cbr_enabled_flag` is equal to 0, `pps_joint_cbr_qp_offset_value` is not used in the decoding process and decoders shall ignore its value. When `pps_joint_cbr_qp_offset_present_flag` is equal to 0, `pps_joint_cbr_qp_offset_value` is not present and is inferred to be equal to 0.

(73) `pps_slice_chroma_qp_offsets_present_flag` equal to 1 specifies that the `slice_cb_qp_offset` and `slice_cr_qp_offset` syntax elements are present in the associated slice headers. `pps_slice_chroma_qp_offsets_present_flag` equal to 0 specifies that the `slice_cb_qp_offset` and `slice_cr_qp_offset` syntax elements are not present in the associated slice headers. When not present, the value of `pps_slice_chroma_qp_offsets_present_flag` is inferred to be equal to 0.

(74) `pps_cu_chroma_qp_offset_list_enabled_flag` equal to 1 specifies that the `ph_cu_chroma_qp_offset_subdiv_intra_slice` and `ph_cu_chroma_qp_offset_subdiv_inter_slice` syntax elements are present in PHs referring to the PPS and `cu_chroma_qp_offset_flag` may be present in the transform unit syntax and the palette coding syntax. `pps_cu_chroma_qp_offset_list_enabled_flag` equal to 0 specifies that the `ph_cu_chroma_qp_offset_subdiv_intra_slice` and `ph_cu_chroma_qp_offset_subdiv_inter_slice` syntax elements are not present in PHs referring to the PPS and the `cu_chroma_qp_offset_flag` is not present in the transform unit syntax and the palette coding syntax. When not present, the value of `pps_cu_chroma_qp_offset_list_enabled_flag` is inferred to be equal to 0.

(75) `chroma_qp_offset_list_len_minus1` plus 1 specifies the number of `cb_qp_offset_list[i]`, `cr_qp_offset_list[i]`, and `joint_cbr_qp_offset_list[i]`, syntax elements that are present in the PPS RBSP syntax structure. The value of `chroma_qp_offset_list_len_minus1` shall be in the range of 0 to 5, inclusive.

(76) `cb_qp_offset_list[i]`, `cr_qp_offset_list[i]`, and `joint_cbr_qp_offset_list[i]`, specify offsets used in the derivation of `Qp'.sub.Cb`, `Qp'.sub.Cr`, and `Qp'.sub.CbCr`, respectively. The values of `cb_qp_offset_list[i]`, `cr_qp_offset_list[i]`, and `joint_cbr_qp_offset_list[i]` shall be in the range -12 to +12, inclusive. When `pps_joint_cbr_qp_offset_present_flag` is equal to 0, `joint_cbr_qp_offset_list[i]` is not present and it is inferred to be equal to 0.

(77) `pps_weighted_pred_flag` equal to 0 specifies that weighted prediction is not applied to P slices referring to the PPS. `pps_weighted_pred_flag` equal to 1 specifies that weighted prediction is applied to P slices referring to the PPS. When `sps_weighted_pred_flag` is equal to 0, the value of `pps_weighted_pred_flag` shall be equal to 0.

(78) `pps_weighted_bipred_flag` equal to 0 specifies that explicit weighted prediction is not applied to B slices referring to the PPS.

`pps_weighted_bipred_flag` equal to 1 specifies that explicit weighted prediction is applied to B slices referring to the PPS. When `sps_weighted_bipred_flag` is equal to 0, the value of `pps_weighted_bipred_flag` shall be equal to 0.

(79) `deblocking_filter_control_present_flag` equal to 1 specifies the presence of deblocking filter control syntax elements in the PPS.

`deblocking_filter_control_present_flag` equal to 0 specifies the absence of deblocking filter control syntax elements in the PPS.

(80) `deblocking_filter_override_enabled_flag` equal to 1 specifies the presence of `ph_deblocking_filter_override_flag` in the PHs referring to the PPS or `slice_deblocking_filter_override_flag` in the slice headers referring to the PPS. `deblocking_filter_override_enabled_flag` equal to 0 specifies the absence of `ph_deblocking_filter_override_flag` in PHs referring to the PPS or `slice_deblocking_filter_override_flag` in slice headers referring to the PPS. When not present, the value of `deblocking_filter_override_enabled_flag` is inferred to be equal to 0.

(81) `pps_deblocking_filter_disabled_flag` equal to 1 specifies that the operation of deblocking filter is not applied for slices referring the PPS to in which `slice_deblocking_filter_disabled_flag` is not present. `pps_deblocking_filter_disabled_flag` equal to 0 specifies that the operation of the deblocking filter is applied for slices referring to the PPS in which `slice_deblocking_filter_disabled_flag` is not present. When not present, the value of `pps_deblocking_filter_disabled_flag` is inferred to be equal to 0.

(82) `pps_beta_offset_div2` and `pps_tc_offset_div2` specify the default deblocking parameter offsets for β and tC (divided by 2) that are applied to the luma component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the picture headers or the slice headers of the slices referring to the PPS. The values of `pps_beta_offset_div2` and `pps_tc_offset_div2` shall both be in the range of -12 to 12, inclusive. When not present, the values of `pps_beta_offset_div2` and `pps_tc_offset_div2` are both inferred to be equal to 0.

(83) `pps_cb_beta_offset_div2` and `pps_cb_tc_offset_div2` specify the default deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cb component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the picture headers or the slice headers of the slices referring to the PPS. The values of `pps_cb_beta_offset_div2` and `pps_cb_tc_offset_div2` shall both be in the range of -12 to 12, inclusive. When not present, the values of `pps_cb_beta_offset_div2` and `pps_cb_tc_offset_div2` are both inferred to be equal to 0.

(84) `pps_cr_beta_offset_div2` and `pps_cr_tc_offset_div2` specify the default deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cr component for slices referring to the PPS, unless the default deblocking parameter offsets are overridden by the deblocking parameter offsets present in the picture headers or the slice headers of the slices referring to the PPS. The values of `pps_cr_beta_offset_div2` and `pps_cr_tc_offset_div2` shall both be in the range of -12 to 12, inclusive. When not present, the values of `pps_cr_beta_offset_div2` and `pps_cr_tc_offset_div2` are both inferred to be equal to 0.

(85) `rpl_info_in_ph_flag` equal to 1 specifies that reference picture list information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. `rpl_info_in_ph_flag` equal to 0 specifies that reference picture list information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure.

(86) dbf_info_in_ph_flag equal to 1 specifies that deblocking filter information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. dbf_info_in_ph_flag equal to 0 specifies that deblocking filter information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure. When not present, the value of dbf_info_in_ph_flag is inferred to be equal to 0.

(87) sao_info_in_ph_flag equal to 1 specifies that sample adaptive offset (SAO) filter information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. sao_info_in_ph_flag equal to 0 specifies that SAO filter information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure.

(88) alf_info_in_ph_flag equal to 1 specifies that adaptive loop filter (ALF) information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. alf_info_in_ph_flag equal to 0 specifies that ALF information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure.

(89) wp_info_in_ph_flag equal to 1 specifies that weighted prediction information may be present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. wp_info_in_ph_flag equal to 0 specifies that weighted prediction information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure. When not present, the value of wp_info_in_ph_flag is inferred to be equal to 0.

(90) qp_delta_info_in_ph_flag equal to 1 specifies that QP delta information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. qp_delta_info_in_ph_flag equal to 0 specifies that QP delta information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure.

(91) pps_ref_wraparound_enabled_flag equal to 1 specifies that horizontal wrap-around motion compensation is applied in inter prediction. pps_ref_wraparound_enabled_flag equal to 0 specifies that horizontal wrap-around motion compensation is not applied. When the value of CtbSizeY/MinCbSizeY+1 is greater than pic_width_in_luma_samples/MinCbSizeY-1, the value of pps_ref_wraparound_enabled_flag shall be equal to 0. When sps_ref_wraparound_enabled_flag is equal to 0, the value of pps_ref_wraparound_enabled_flag shall be equal to 0.

(92) pps_ref_wraparound_offset plus (CtbSizeY/MinCbSizeY)+2 specifies the offset used for computing the horizontal wrap-around position in units of MinCbSizeY luma samples. The value of pps_ref_wraparound_offset shall be in the range of 0 to (pic_width_in_luma_samples/MinCbSizeY)-(CtbSizeY/MinCbSizeY)-2, inclusive. The variable PpsRefWraparoundOffset is set equal to pps_ref_wraparound_offset+ (CtbSizeY/MinCbSizeY)+2.

(93) picture_header_extension_present_flag equal to 0 specifies that no PH extension syntax elements are present in PHs referring to the PPS.

picture_header_extension_present_flag equal to 1 specifies that PH extension syntax elements are present in PHs referring to the PPS.

picture_header_extension_present_flag shall be equal to 0 in bitstreams conforming to this version of this Specification.

(94) slice_header_extension_present_flag equal to 0 specifies that no slice header extension syntax elements are present in the slice headers for coded pictures referring to the PPS. slice_header_extension_present_flag equal to 1 specifies that slice header extension syntax elements are present in the slice headers for coded pictures referring to the PPS. slice_header_extension_present_flag shall be equal to 0 in bitstreams conforming to this version of this Specification.

(95) pps_extension_flag equal to 0 specifies that no pps_extension_data_flag syntax elements are present in the PPS RBSP syntax structure.

pps_extension_flag equal to 1 specifies that there are pps_extension_data_flag syntax elements present in the PPS RBSP syntax structure.

(96) pps_extension_data_flag may have any value. Its presence and value do not affect decoder conformance to profiles specified in this version of this Specification. Decoders conforming to this version of this Specification shall ignore all pps_extension_data_flag syntax elements.

3.2. APS Syntax and Semantics

(97) In the latest VVC draft text, the APS syntax and semantics are as follows:

(98) TABLE-US-00002 Descriptor adaptation_parameter_set_rbsp() { adaptation_parameter_set_id u(5) aps_params_type u(3) if(aps_params_type == ALF_APS) alf_data() else if(aps_params_type == LMCS_APS) lmcs_data() else if(aps_params_type == SCALING_APS) scaling_list_data() aps_extension_flag u(1) if(aps_extension_flag) while(more_rbsp_data()) aps_extension_data_flag u(1) rbsp_trailing_bits() }

(99) The APS RBSP contains a ALF syntax structure, i.e., alf_data() ()

(100) TABLE-US-00003 Descriptor alf_data() { alf_luma_filter_signal_flag u(1) alf_chroma_filter_signal_flag u(1) alf_cc_cb_filter_signal_flag u(1) alf_cc_cr_filter_signal_flag u(1) if(alf_luma_filter_signal_flag) { alf_luma_clip_flag u(1) alf_luma_num_filters_signalled_minus1 ue(v) if(alf_luma_num_filters_signalled_minus1 > 0) for(filtIdx = 0; filtIdx < NumAlfFilters; filtIdx++) { alf_luma_coeff_delta_idx[filtIdx] u(v) for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1; sflIdx++) { for(j = 0; j < 12; j++) { alf_luma_coeff_abs[sflIdx][j] ue(v) if(alf_luma_coeff_abs[sflIdx][j]) alf_luma_coeff_sign[sflIdx][j] u(1) } if(alf_luma_clip_flag) for(sflIdx = 0; sflIdx <= alf_luma_num_filters_signalled_minus1; sflIdx++) { for(j = 0; j < 12; j++) { alf_luma_clip_idx[sflIdx][j] u(2) } if(alf_chroma_filter_signal_flag) { alf_chroma_clip_flag u(1) alf_chroma_num_alt_filters_minus1 ue(v) for(altIdx = 0; altIdx <= alf_chroma_num_alt_filters_minus1; altIdx++) { for(j = 0; j < 6; j++) { alf_chroma_coeff_abs[altIdx][j] ue(v) if(alf_chroma_coeff_abs[altIdx][j] > 0) alf_chroma_coeff_sign[altIdx][j] u(1) } if(alf_chroma_clip_flag) for(j = 0; j < 6; j++) { alf_chroma_clip_idx[altIdx][j] u(2) } } if(alf_cc_cb_filter_signal_flag) { alf_cc_cb_filters_signalled_minus1 ue(v) for(k = 0; k < alf_cc_cb_filters_signalled_minus1 + 1; k++) { for(j = 0; j < 7; j++) { alf_cc_cb_mapped_coeff_abs[k][j] u(3) if(alf_cc_cb_mapped_coeff_abs[k][j]) alf_cc_cb_coeff_sign[k][j] u(1) } } } if(alf_cc_cr_filter_signal_flag) { alf_cc_cr_filters_signalled_minus1 ue(v) for(k = 0; k < alf_cc_cr_filters_signalled_minus1 + 1; k++) { for(j = 0; j < 7; j++) { alf_cc_cr_mapped_coeff_abs[k][j] u(3) if(alf_cc_cr_mapped_coeff_abs[k][j]) alf_cc_cr_coeff_sign[k][j] u(1) } } } } }

(101) The APS RBSP contains a luma mapping with chroma scaling (LMCS) syntax structure, i.e., lmcs_data() ()

(102) TABLE-US-00004 Descriptor lmcs_data() { lmcs_min_bin_idx ue(v) lmcs_delta_max_bin_idx ue(v) lmcs_delta_cw_prec_minus1 ue(v) for(i = lmcs_min_bin_idx; i <= LmcsMaxBinIdx; i++) { lmcs_delta_abs_cw[i] u(v) if(lmcs_delta_abs_cw[i] > 0) lmcs_delta_sign_cw_flag[i] u(1) } lmcs_delta_abs_crs u(3) if(lmcs_delta_abs_crs > 0) lmcs_delta_sign_crs_flag u(1) }

(103) The APS RBSP contains a scaling list data syntax structure, i.e., scaling_list_data() ()

(104) TABLE-US-00005 Descriptor scaling_list_data() { scaling_matrix_for_lfnst_disabled_flag u(1) scaling_list_chroma_present_flag u(1) for(id = 0; id < 28; id++) { matrixSize = (id < 2) ? 2 : ((id < 8) ? 4 : 8) if(scaling_list_chroma_present_flag || (id % 3 == 2) || (id == 27)) { scaling_list_copy_mode_flag[id] u(1) if(!scaling_list_copy_mode_flag[id]) scaling_list_pred_mode_flag[id] u(1) if((scaling_list_copy_mode_flag[id] || scaling_list_pred_mode_flag[id]) && id != 0 && id != 2 && id != 8) scaling_list_pred_id_delta[id] ue(v) if(!scaling_list_copy_mode_flag[id]) { nextCoef = 0 if(id > 13) { scaling_list_dc_coeff[id - 14] se(v) nextCoef += scaling_list_dc_coeff[id - 14] } for(i = 0; i < matrixSize * matrixSize; i++) { x = DiagScanOrder[3][3][i][0] y = DiagScanOrder[3][3][i][1] if(!(id > 25 && x >= 4 && y >= 4)) { scaling_list_delta_coeff[id][i] se(v) nextCoef += scaling_list_delta_coeff[id][i] } ScalingList[id][i] = nextCoef } } } }

(105) Each APS RBSP shall be available to the decoding process prior to it being referenced, included in at least one access unit (AU) with

TemporalId less than or equal to the TemporalId of the coded slice NAL unit that refers to it or provided through external means.

(106) All APS NAL units with a particular value of adaptation_parameter_set_id and a particular value of aps_params_type within a PU, regardless of whether they are prefix or suffix APS NAL units, shall have the same content.

(107) adaptation_parameter_set_id provides an identifier for the APS for reference by other syntax elements.

(108) When aps_params_type is equal to ALF_APS or SCALING_APS, the value of adaptation_parameter_set_id shall be in the range of 0 to 7, inclusive.

(109) When aps_params_type is equal to LMCS_APS, the value of adaptation_parameter_set_id shall be in the range of 0 to 3, inclusive.

(110) Let apsLayerId be the value of the nuh_layer_id of a particular APS NAL unit, and vclLayerId be the value of the nuh_layer_id of a particular VCL NAL unit. The particular VCL NAL unit shall not refer to the particular APS NAL unit unless apsLayerId is less than or equal to vclLayerId and the layer with nuh_layer_id equal to apsLayerId is included in at least one output layer set (OLS) that includes the layer with nuh_layer_id equal to vclLayerId. aps_params_type specifies the type of APS parameters carried in the APS as specified in Table 6.

(111) TABLE-US-00006 TABLE 6 APS parameters type codes and types of APS parameters

Name of APS parameters	Type of aps_params_type	aps_params_type
APS parameters 0	ALF_APS	ALF parameters
APS parameters 1	LMCS_APS	LMCS parameters
APS parameters 2	SCALING_APS	Scaling list parameters
APS parameters 3 . . . 7	Reserved	Reserved

(112) All APS NAL units with a particular value of aps_params_type, regardless of the nuh_layer_id values, share the same value space for adaptation_parameter_set_id. APS NAL units with different values of aps_params_type use separate values spaces for adaptation_parameter_set_id. NOTE 1—An APS NAL unit (with a particular value of adaptation_parameter_set_id and a particular value of aps_params_type) can be shared across pictures, and different slices within a picture can refer to different ALF APSs. NOTE 2—A suffix APS NAL unit associated with a particular VCL NAL unit (this VCL NAL unit precedes the suffix APS NAL unit in decoding order) is not for use by the particular VCL NAL unit, but for use by VCL NAL units following the suffix APS NAL unit in decoding order.

(113) aps_extension_flag equal to 0 specifies that no aps_extension_data_flag syntax elements are present in the APS RBSP syntax structure.

aps_extension_flag equal to 1 specifies that there are aps_extension_data_flag syntax elements present in the APS RBSP syntax structure.

(114) aps_extension_data_flag may have any value. Its presence and value do not affect decoder conformance to profiles specified in this version of this Specification. Decoders conforming to this version of this Specification shall ignore all aps_extension_data_flag syntax elements.

(115) alf_luma_filter_signal_flag equal to 1 specifies that a luma filter set is signalled. alf_luma_filter_signal_flag equal to 0 specifies that a luma filter set is not signalled.

(116) alf_chroma_filter_signal_flag equal to 1 specifies that a chroma filter is signalled. alf_chroma_filter_signal_flag equal to 0 specifies that a chroma filter is not signalled. When ChromaArrayType is equal to 0, alf_chroma_filter_signal_flag shall be equal to 0.

(117) At least one of the values of alf_luma_filter_signal_flag, alf_chroma_filter_signal_flag, alf_cc_cb_filter_signal_flag and alf_cc_cr_filter_signal_flag shall be equal to 1.

(118) The variable NumAlfFilters specifying the number of different adaptive loop filters is set equal to 25. alf_luma_clip_flag equal to 0 specifies that linear adaptive loop filtering is applied on luma component. alf_luma_clip_flag equal to 1 specifies that non-linear adaptive loop filtering may be applied on luma component. alf_luma_num_filters_signalled_minus1 plus 1 specifies the number of adaptive loop filter classes for which luma coefficients can be signalled. The value of alf_luma_num_filters_signalled_minus1 shall be in the range of 0 to NumAlfFilters-1, inclusive.

(119) alf_luma_coeff_delta_idx[filtIdx] specifies the indices of the signalled adaptive loop filter luma coefficient deltas for the filter class indicated by filtIdx ranging from 0 to NumAlfFilters-1. When alf_luma_coeff_delta_idx[filtIdx] is not present, it is inferred to be equal to 0. The length of alf_luma_coeff_delta_idx[filtIdx] is Ceil(Log2(alf_luma_num_filters_signalled_minus1+1)) bits. The value of alf_luma_coeff_delta_idx[filtIdx] shall be in the range of 0 to alf_luma_num_filters_signalled_minus1, inclusive.

(120) alf_luma_coeff_abs[sfIdx][j] specifies the absolute value of the j-th coefficient of the signalled luma filter indicated by sfIdx. When alf_luma_coeff_abs[sfIdx][j] is not present, it is inferred to be equal 0. The value of alf_luma_coeff_abs[sfIdx][j] shall be in the range of 0 to 128, inclusive.

(121) alf_luma_coeff_sign[sfIdx][j] specifies the sign of the j-th luma coefficient of the filter indicated by sfIdx as follows: If alf_luma_coeff_sign[sfIdx][j] is equal to 0, the corresponding luma filter coefficient has a positive value. Otherwise (alf_luma_coeff_sign[sfIdx][j] is equal to 1), the corresponding luma filter coefficient has a negative value.

(122) When alf_luma_coeff_sign[sfIdx][j] is not present, it is inferred to be equal to 0.

(123) The variable filtCoeff[sfIdx][j] with sfIdx=0 . . . alf_luma_num_filters_signalled_minus1, j=0 . . . 11 is initialized as follows:

filtCoeff[sfIdx][j]=alf_luma_coeff_abs[sfIdx][j]*(1-2*alf_luma_coeff_sign[sfIdx][j]) (93)

(124) The luma filter coefficients AlfCoeff.sub.L[adaptation_parameter_set_id] with elements AlfCoeff.sub.L[adaptation_parameter_set_id][filtIdx][j], with filtIdx=0 . . . NumAlfFilters-1 and j=0 . . . 11 are derived as follows:

AlfCoeff.sub.L[adaptation_parameter_set_id][filtIdx][j]=filtCoeff[alf_luma_coeff_delta_idx[filtIdx]][j] (94)

(125) The fixed filter coefficients AlfFixFiltCoeff[i][j] with i=0 . . . 64, j=0 . . . 11 and the class to filter mapping AlfClassToFiltMap[m][n] with m=0 . . . 15 and n=0 . . . 24 are derived as follows:

(126) TABLE-US-00007 (95) AlfFixFiltCoeff = { { 0, 0, 2, -3, 1, -4, 1, 7, -1, 1, -1, 5 } { 0, 0, 0, 0, 0, -1, 0, 1, 0, 0, -1, 2 } { 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, -1 } { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -1, 1 } { 2, 2, -7, -3, 0, -5, 13, 22, 12, -3, -3, 17 } { -1, 0, 6, -8, 1, -5, 1, 23, 0, 2, -5, 10 } { 0, 0, -1, -1, 0, -1, 2, 1, 0, 0, -1, 4 } { 0, 0, 3, -11, 1, 0, -1, 35, 5, 2, -9, 9 } { 0, 0, 8, -8, -2, -7, 4, 4, 2, 1, -1, 25 } { 0, 0, 1, -1, 0, -3, 1, 3, -1, 1, -1, 3 } { 0, 0, 3, -3, 0, -6, 5, -1, 2, 1, -4, 21 } { -7, 1, 5, 4, -3, 5, 11, 13, 12, -8, 11, 12 } { -5, -3, 6, -2, -3, 8, 14, 15, 2, -7, 11, 16 } { 2, -1, -6, -5, -2, -2, 20, 14, -4, 0, -3, 25 } { 3, 1, -8, -4, 0, -8, 22, 5, -3, 2, -10, 29 } { 2, 1, -7, -1, 2, -11, 23, -5, 0, 2, -10, 29 } { -6, -3, 8, 9, -4, 8, 9, 7, 14, -2, 8, 9 } { 2, 1, -4, -7, 0, -8, 17, 22, 1, -1, -4, 23 } { 3, 0, -5, -7, 0, -7, 15, 18, -5, 0, -5, 27 } { 2, 0, 0, -7, 1, -10, 13, 13, -4, 2, -7, 24 } { 3, 3, -13, 4, -2, -5, 9, 21, 25, -2, -3, 12 } { -5, -2, 7, -3, -7, 9, 8, 9, 16, -2, 15, 12 } { 0, -1, 0, -7, -5, 4, 11, 11, 8, -6, 12, 21 } { 3, -2, -3, -8, -4, -1, 16, 15, -2, -3, 3, 26 } { 2, 1, -5, -4, -1, -8, 16, 4, -2, 1, -7, 33 } { 2, 1, -4, -2, 1, -10, 17, -2, 0, 2, -11, 33 } { 1, -2, 7, -15, -16, 10, 8, 8, 20, 11, 14, 11 } { 2, 2, 3, -13, -13, 4, 8, 12, 2, -3, 16, 24 } { 1, 4, 0, -7, -8, -4, 9, 9, -2, -2, 8, 29 } { 1, 1, 2, -4, -1, -6, 6, 3, -1, -1, -3, 30 } { -7, 3, 2, 10, -2, 3, 7, 11, 19, -7, 8, 10 } { 0, -2, -5, -3, -2, 4, 20, 15, -1, -3, -1, 22 } { 3, -1, -8, -4, -1, -4, 22, 8, -4, 2, -8, 28 } { 0, 3, -14, 3, 0, 1, 19, 17, 8, -3, -7, 20 } { 0, 2, -1, -8, 3, -6, 5, 21, 1, 1, -9, 13 } { -4, -2, 8, 20, -2, 2, 3, 5, 21, 4, 6, 1 } { 2, -2, -3, -9, -4, 2, 14, 16, 3, -6, 8, 24 } { 2, 1, 5, -16, -7, 2, 3, 11, 15, -3, 11, 22 } { 1, 2, 3, -11, -2, -5, 4, 8, 9, -3, -2, 26 } { 0, -1, 10, -9, -1, -8, 2, 3, 4, 0, 0, 29 } { 1, 2, 0, -5, 1, -9, 9, 3, 0, 1, -7, 20 } { -2, 8, -6, -4, 3, -9, -8, 45, 14, 2, -13, 7 } { 1, -1, 16, -19, -8, -4, -3, 2, 19, 0, 4, 30 } { 1, 1, -3, 0, 2, -11, 15, -5, 1, 2, -9, 24 } { 0, 1, -2, 0, 1, -4, 4, 0, 0, 1, -4, 7 } { 0, 1, 2, -5, 1, -6, 4, 10, -2, 1, -4, 10 } { 3, 0, -3, -6, -2, -6, 14, 8, -1, -1, -3, 31 } { 0, 1, 0, -2, 1, -6, 5, 1, 0, 1, -5, 13 } { 3, 1, 9, -19, -21, 9, 7, 6, 13, 5, 15, 21 } { 2, 4, 3, -12, -13, 1, 7, 8, 3, 0, 12, 26 } { 3, 1, -8, -2, 0, -6, 18, 2, -2, 3, -10, 23 } { 1, 1, -4, -1, 1, -5, 8, 1, -1, 2, -5, 10 } { 0, 1, -1, 0, 0, -2, 2, 0, 0, 1, -2, 3 } { 1, 1, -2, -7, 1, -7, 14, 18, 0, 0, -7, 21 } { 0, 1, 0, -2, 0, -7, 8, 1, -2, 0, -3, 24 } { 0, 1, 1, -2, 2, -10, 10, 0, -2, 1, -7, 23 } { 0, 2, 2, -11, 2, -4, -3, 39, 7, 1, -10, 9 } { 1, 0, 13, -16, -5, -6, -1, 8, 6, 0, 6, 29 } { 1, 3, 1, -6, -4, -7, 9, 6, -3, -2, 3, 33 } { 4, 0, -17, -1, -1, 5, 26, 8, -2, 3, -15, 30 } { 0, 1, -2, 0, 2, -8, 12, -6, 1, 1, -6, 16 } { 0, 0, 0, -1, 1, -4, 4, 0, 0, 0, -3, 11 } { 0, 1, 2, -8, 2, -6, 5, 15, 0, 2, -7, 9 } { 1, -1, 12, -15, -7, -2, 3, 6, 6, -1, 7, 30 } },

(127) TABLE-US-00008 (96) AlfClassToFiltMap = { { 8, 2, 2, 2, 3, 4, 53, 9, 9, 52, 4, 4, 5, 9, 2, 8, 10, 9, 1, 3, 39, 39, 10, 9, 52 } { 11, 12, 13, 14, 15, 30, 11, 17, 18, 19, 16, 20, 20, 4, 53, 21, 22, 23, 14, 25, 26, 26, 27, 28, 10 } { 16, 12, 31, 32, 14, 16, 30, 33, 53, 34, 35, 16, 20, 4, 7, 16, 21, 36, 18, 19, 21, 26, 37, 38, 39 } { 35, 11, 13, 14, 43, 35, 16, 4, 34, 62, 35, 35, 30, 56, 7, 35, 21, 38, 24, 40, 16, 21, 48, 57, 39 } { 11, 31, 32, 43, 44,

16, 4, 17, 34, 45, 30, 20, 20, 7, 5, 21, 22, 46, 26, 48, 63, 58, 10 } { 12, 13, 50, 51, 52, 11, 17, 53, 45, 9, 30, 4, 53, 19, 0, 22, 23, 25, 43, 44, 37, 27, 28, 10, 55 } { 30, 33, 62, 51, 44, 20, 41, 56, 34, 45, 20, 41, 41, 56, 5, 30, 56, 38, 40, 47, 11, 37, 42, 57, 8 } { 35, 11, 23, 32, 14, 35, 20, 4, 17, 18, 21, 20, 20, 20, 4, 16, 21, 36, 46, 25, 41, 26, 48, 49, 58 } { 12, 31, 59, 59, 3, 33, 33, 59, 59, 52, 4, 33, 17, 59, 55, 22, 36, 59, 59, 60, 22, 36, 59, 25, 55 } { 31, 25, 15, 60, 60, 22, 17, 19, 55, 55, 20, 20, 53, 19, 55, 22, 46, 25, 43, 60, 37, 28, 10, 55, 52 } { 12, 31, 32, 50, 51, 11, 33, 53, 19, 45, 16, 4, 4, 53, 5, 22, 36, 18, 25, 43, 26, 27, 27, 28, 10 } { 5, 2, 44, 52, 3, 4, 53, 45, 9, 3, 4, 56, 5, 0, 2, 5, 10, 47, 52, 3, 63, 39, 10, 9, 52 } { 12, 34, 44, 44, 3, 56, 56, 62, 45, 9, 56, 56, 7, 5, 0, 22, 38, 40, 47, 52, 48, 57, 39, 10, 9 } { 35, 11, 23, 14, 51, 35, 20, 41, 56, 62, 16, 20, 41, 56, 7, 16, 21, 38, 24, 40, 26, 26, 42, 57, 39 } { 33, 34, 51, 51, 52, 41, 41, 34, 62, 0, 41, 41, 56, 7, 5, 56, 38, 38, 40, 44, 37, 42, 57, 39, 10 } { 16, 31, 32, 15, 60, 30, 4, 17, 19, 25, 22, 20, 4, 53, 19, 21, 22, 46, 25, 55, 26, 48, 63, 58, 55 } },

(128) It is a requirement of bitstream conformance that the values of $\text{AlfCcoeff.sub.L}[\text{adaptation_parameter_set_id}][\text{filtIdx}][j]$ with $\text{filtIdx}=0 \dots \text{NumAlfFilters}-1, j=0 \dots 11$ shall be in the range of $-2.\text{sup.7}$ to $2.\text{sup.7}-1$, inclusive.

(129) $\text{alf_luma_clip_idx}[\text{sflIdx}][j]$ specifies the clipping index of the clipping value to use before multiplying by the j -th coefficient of the signalled luma filter indicated by sflIdx . It is a requirement of bitstream conformance that the values of $\text{alf_luma_clip_idx}[\text{sflIdx}][j]$ with $\text{sflIdx}=0 \dots \text{alf_luma_num_filters_signalled_minus1}$ and $j=0 \dots 11$ shall be in the range of 0 to 3, inclusive.

(130) The luma filter clipping values $\text{AlfClip.sub.L}[\text{adaptation_parameter_set_id}]$ with elements $\text{AlfClip.sub.L}[\text{adaptation_parameter_set_id}][\text{filtIdx}][j]$, with $\text{filtIdx}=0 \dots \text{NumAlfFilters}-1$ and $j=0 \dots 11$ are derived as specified in Table 8 depending on BitDepth and clipIdx set equal to $\text{alf_luma_clip_idx}[\text{alf_luma_coeff_delta_idx}[\text{filtIdx}][j]]$.

(131) $\text{alf_chroma_clip_flag}$ equal to 0 specifies that linear adaptive loop filtering is applied on chroma components; $\text{alf_chroma_clip_flag}$ equal to 1 specifies that non-linear adaptive loop filtering is applied on chroma components. When not present, $\text{alf_chroma_clip_flag}$ is inferred to be equal to 0.

(132) $\text{alf_chroma_num_alt_filters_minus1}$ plus 1 specifies the number of alternative filters for chroma components. The value of $\text{alf_chroma_num_alt_filters_minus1}$ shall be in the range of 0 to 7, inclusive.

(133) $\text{alf_chroma_coeff_abs}[\text{altIdx}][j]$ specifies the absolute value of the j -th chroma filter coefficient for the alternative chroma filter with index altIdx . When $\text{alf_chroma_coeff_abs}[\text{altIdx}][j]$ is not present, it is inferred to be equal 0. The value of $\text{alf_chroma_coeff_abs}[\text{sflIdx}][j]$ shall be in the range of 0 to 128, inclusive.

(134) $\text{alf_chroma_coeff_sign}[\text{altIdx}][j]$ specifies the sign of the j -th chroma filter coefficient for the alternative chroma filter with index altIdx as follows: If $\text{alf_chroma_coeff_sign}[\text{altIdx}][j]$ is equal to 0, the corresponding chroma filter coefficient has a positive value. Otherwise ($\text{alf_chroma_coeff_sign}[\text{altIdx}][j]$ is equal to 1), the corresponding chroma filter coefficient has a negative value.

(135) When $\text{alf_chroma_coeff_sign}[\text{altIdx}][j]$ is not present, it is inferred to be equal to 0.

(136) The chroma filter coefficients $\text{AlfCcoeff.sub.C}[\text{adaptation_parameter_set_id}][\text{altIdx}]$ with elements $\text{AlfCcoeff.sub.C}[\text{adaptation_parameter_set_id}][\text{altIdx}][j]$, with $\text{altIdx}=0 \dots \text{alf_chroma_num_alt_filters_minus1}, j=0 \dots 5$ are derived as follows: $\text{AlfCcoeff.sub.C}[\text{adaptation_parameter_set_id}][\text{altIdx}][j] = \text{alf_chroma_coeff_abs}[\text{altIdx}][j] * (1 - 2 * \text{alf_chroma_coeff_sign}[\text{altIdx}][j])$ (97)

(137) It is a requirement of bitstream conformance that the values of $\text{AlfCcoeff.sub.C}[\text{adaptation_parameter_set_id}][\text{altIdx}][j]$ with $\text{altIdx}=0 \dots \text{alf_chroma_num_alt_filters_minus1}, j=0 \dots 5$ shall be in the range of $-2.\text{sup.7}$ to $2.\text{sup.7}-1$, inclusive.

(138) $\text{alf_cc_cb_filter_signal_flag}$ equal to 1 specifies that cross-component filters for the Cb colour component are signalled. $\text{alf_cc_cb_filter_signal_flag}$ equal to 0 specifies that cross-component filters for Cb colour component are not signalled. When ChromaArrayType is equal to 0, $\text{alf_cc_cb_filter_signal_flag}$ shall be equal to 0.

(139) $\text{alf_cc_cb_filters_signalled_minus1}$ plus 1 specifies the number of cross-component filters for the Cb colour component signalled in the current ALF APS. The value of $\text{alf_cc_cb_filters_signalled_minus1}$ shall be in the range of 0 to 3, inclusive.

(140) $\text{alf_cc_cb_mapped_coeff_abs}[k][j]$ specifies the absolute value of the j -th mapped coefficient of the signalled k -th cross-component filter for the Cb colour component. When $\text{alf_cc_cb_mapped_coeff_abs}[k][j]$ is not present, it is inferred to be equal to 0.

(141) $\text{alf_cc_cb_coeff_sign}[k][j]$ specifies the sign of the j -th coefficient of the signalled k -th cross-component filter for the Cb colour component as follows: If $\text{alf_cc_cb_coeff_sign}[k][j]$ is equal to 0, the corresponding cross-component filter coefficient has a positive value. Otherwise ($\text{alf_cc_cb_sign}[k][j]$ is equal to 1), the corresponding cross-component filter coefficient has a negative value.

(142) When $\text{alf_cc_cb_coeff_sign}[k][j]$ is not present, it is inferred to be equal to 0.

(143) The signalled k -th cross-component filter coefficients for the Cb colour component $\text{CcAlfApsCoeff.sub.Cb}[\text{adaptation_parameter_set_id}][k][j]$, with $j=0 \dots 6$ are derived as follows: If $\text{alf_cc_cb_mapped_coeff_abs}[k][j]$ is equal to 0, $\text{CcAlfApsCoeff.sub.Cb}[\text{adaptation_parameter_set_id}][k][j]$ is set equal to 0. Otherwise, $\text{CcAlfApsCoeff.sub.Cb}[\text{adaptation_parameter_set_id}][k][j]$ is set equal to $(1 - 2 * \text{alf_cc_cb_coeff_sign}[k][j]) * 2.\text{sup.alf_cc_cb_mapped_coeff_abs}[k][j] - 1$.

(144) $\text{alf_cc_cr_filter_signal_flag}$ equal to 1 specifies that cross-component filters for the Cr colour component are signalled. $\text{alf_cc_cr_filter_signal_flag}$ equal to 0 specifies that cross-component filters for the Cr colour component are not signalled. When ChromaArrayType is equal to 0, $\text{alf_cc_cr_filter_signal_flag}$ shall be equal to 0.

(145) $\text{alf_cc_cr_filters_signalled_minus1}$ plus 1 specifies the number of cross-component filters for the Cr colour component signalled in the current ALF APS. The value of $\text{alf_cc_cr_filters_signalled_minus1}$ shall be in the range of 0 to 3, inclusive.

(146) $\text{alf_cc_cr_mapped_coeff_abs}[k][j]$ specifies the absolute value of the j -th mapped coefficient of the signalled k -th cross-component filter for the Cr colour component. When $\text{alf_cc_cr_mapped_coeff_abs}[k][j]$ is not present, it is inferred to be equal to 0.

(147) $\text{alf_cc_cr_coeff_sign}[k][j]$ specifies the sign of the j -th coefficient of the signalled k -th cross-component filter for the Cr colour component as follows: If $\text{alf_cc_cr_coeff_sign}[k][j]$ is equal to 0, the corresponding cross-component filter coefficient has a positive value. Otherwise ($\text{alf_cc_cr_sign}[k][j]$ is equal to 1), the corresponding cross-component filter coefficient has a negative value.

(148) When $\text{alf_cc_cr_coeff_sign}[k][j]$ is not present, it is inferred to be equal to 0.

(149) The signalled k -th cross-component filter coefficients for the Cr colour component $\text{CcAlfApsCoeff.sub.Cr}[\text{adaptation_parameter_set_id}][k][j]$, with $j=0 \dots 6$ are derived as follows: If $\text{alf_cc_cr_mapped_coeff_abs}[k][j]$ is equal to 0, $\text{CcAlfApsCoeff.sub.Cr}[\text{adaptation_parameter_set_id}][k][j]$ is set equal to 0. Otherwise, $\text{CcAlfApsCoeff.sub.Cr}[\text{adaptation_parameter_set_id}][k][j]$ is set equal to $(1 - 2 * \text{alf_cc_cr_coeff_sign}[k][j]) * 2.\text{sup.alf_cc_cr_mapped_coeff_abs}[k][j] - 1$.

(150) $\text{alf_chroma_clip_idx}[\text{altIdx}][j]$ specifies the clipping index of the clipping value to use before multiplying by the j -th coefficient of the alternative chroma filter with index altIdx . It is a requirement of bitstream conformance that the values of $\text{alf_chroma_clip_idx}[\text{altIdx}][j]$ with $\text{altIdx}=0 \dots \text{alf_chroma_num_alt_filters_minus1}, j=0 \dots 5$ shall be in the range of 0 to 3, inclusive.

(151) The chroma filter clipping values $\text{AlfClipc}[\text{adaptation_parameter_set_id}][\text{altIdx}]$ with elements $\text{AlfClipc}[\text{adaptation_parameter_set_id}][\text{altIdx}][j]$, with $\text{altIdx}=0 \dots \text{alf_chroma_num_alt_filters_minus1}, j=0 \dots 5$ are derived as specified in Table 8 depending on BitDepth and clipIdx set equal to $\text{alf_chroma_clip_idx}[\text{altIdx}][j]$.

(152) TABLE-US-00009 TABLE 8 Specification AlfClip depending on BitDepth and clipIdx

clipIdx	BitDepth 0	1	2	3	8	2.sup.8	2.sup.5	2.sup.3
2	2.sup.1	9	2.sup.9	2.sup.6	2.sup.4	2.sup.2	10	2.sup.10
2	2.sup.7	2.sup.5	13	2.sup.13	2.sup.10	2.sup.8	2.sup.6	14
2	2.sup.14	2.sup.11	2.sup.9	2.sup.7	15	2.sup.15	2.sup.12	2.sup.10
2	2.sup.8	16	2.sup.16	2.sup.13	2.sup.11	2.sup.9		

(153) lmcs_min_bin_idx specifies the minimum bin index used in the luma mapping with chroma scaling construction process. The value of lmcs_min_bin_idx shall be in the range of 0 to 15, inclusive.

(154) $\text{lmcs_delta_value_bin_idx}$ specifies the delta value between the maximum bin index LmcsMaxBinIdx used in the luma mapping with chroma scaling construction process. The value of $\text{lmcs_delta_max_bin_idx}$ shall be in the range of 0 to 15, inclusive. The value of LmcsMaxBinIdx is set equal to $15 - \text{lmcs_delta_max_bin_idx}$. The value of LmcsMaxBinIdx shall be greater than or equal to lmcs_min_bin_idx .

(155) $\text{lmcs_delta_cw_prec_minus1}$ plus 1 specifies the number of bits used for the representation of the syntax $\text{lmcs_delta_abs_cw}[i]$. The value of $\text{lmcs_delta_cw_prec_minus1}$ shall be in the range of 0 to $\text{BitDepth} - 2$, inclusive.

(156) $\text{lmcs_delta_abs_cw}[i]$ specifies the absolute delta codeword value for the i th bin.

(157) $\text{lmcs_delta_sign_cw_flag}[i]$ specifies the sign of the variable $\text{lmcsDeltaCW}[i]$ as follows: If $\text{lmcs_delta_sign_cw_flag}[i]$ is equal to 0, $\text{lmcsDeltaCW}[i]$ is a positive value. Otherwise ($\text{lmcs_delta_sign_cw_flag}[i]$ is not equal to 0), $\text{lmcsDeltaCW}[i]$ is a negative value.

(158) When $\text{lmcs_delta_sign_cw_flag}[i]$ is not present, it is inferred to be equal to 0.

(159) The variable OrgCW is derived as follows:

$$\text{OrgCW} = (1 \ll \text{BitDepth}) / 16 \quad (98)$$

(160) The variable $\text{lmcsDeltaCW}[i]$, with $i = \text{lmcs_min_bin_idx} \dots \text{LmcsMaxBinIdx}$, is derived as follows:

$$\text{lmcsDeltaCW}[i] = (1 - 2 * \text{lmcs_delta_sign_cw_flag}[i]) * \text{lmcs_delta_abs_cw}[i] \quad (99)$$

(161) The variable $\text{lmcsCW}[i]$ is derived as follows: For $i = 0 \dots \text{lmcs_min_bin_idx} - 1$, $\text{lmcsCW}[i]$ is set equal 0. For $i = \text{lmcs_min_bin_idx} \dots \text{LmcsMaxBinIdx}$, the following applies:

$$\text{lmcsCW}[i] = \text{OrgCW} + \text{lmcsDeltaCW}[i] \quad (100)$$

(162) The value of $\text{lmcsCW}[i]$ shall be in the range of ($\text{OrgCW} \gg 3$) to ($\text{OrgCW} \ll 3 - 1$), inclusive. For $i = \text{LmcsMaxBinIdx} + 1 \dots 15$, $\text{lmcsCW}[i]$ is set equal 0.

(163) It is a requirement of bitstream conformance that the following condition is true:

$$\sum_{i=0}^{\text{sup}} 15 \text{lmcsCW}[i] \leq (1 \ll \text{BitDepth}) - 1 = 0 \quad (101)$$

(164) The variable $\text{InputPivot}[i]$, with $i = 0 \dots 16$, is derived as follows:

$$\text{InputPivot}[i] = i * \text{OrgCW} \quad (102)$$

(165) The variable $\text{LmcsPivot}[i]$ with $i = 0 \dots 16$, the variables $\text{ScaleCoeff}[i]$ and $\text{InvScaleCoeff}[i]$ with $i = 0 \dots 15$, are derived as follows:

$$\text{LmcsPivot}[0] = 0; \text{ for } (i=0; i \leq 15; i++) \{ \text{LmcsPivot}[i+1] = \text{LmcsPivot}[i] + \text{lmcsCW}[i] \text{ScaleCoeff}[i] = (\text{lmcsCW}[i] * (1 \ll 11) + (1 \ll (\text{Log2}(\text{OrgCW}) - 1))) \gg (\text{Log2}(\text{OrgCW})) \text{ if } (\text{lmcsCW}[i] = 0) \text{InvScaleCoeff}[i] = 0 \text{ else } \text{InvScaleCoeff}[i] = \text{OrgCW} * (1 \ll 11) / \text{lmcsCW}[i] \} \quad (103)$$

(166) It is a requirement of bitstream conformance that, for $i = \text{lmcs_min_bin_idx} \dots \text{LmcsMaxBinIdx}$, when the value of $\text{LmcsPivot}[i]$ is not a multiple of $1 \ll (\text{BitDepth} - 5)$, the value of ($\text{LmcsPivot}[i] \gg (\text{BitDepth} - 5)$) shall not be equal to the value of ($\text{LmcsPivot}[i+1] \gg (\text{BitDepth} - 5)$).

(167) $\text{lmcs_delta_abs_crs}$ specifies the absolute codeword value of the variable lmcsDeltaCrs . The value of $\text{lmcs_delta_abs_crs}$ shall be in the range of 0 and 7, inclusive. When not present, $\text{lmcs_delta_abs_crs}$ is inferred to be equal to 0.

(168) $\text{lmcs_delta_sign_crs_flag}$ specifies the sign of the variable lmcsDeltaCrs . When not present, $\text{lmcs_delta_sign_crs_flag}$ is inferred to be equal to 0.

(169) The variable lmcsDeltaCrs is derived as follows:

$$\text{lmcsDeltaCrs} = (1 - 2 * \text{lmcs_delta_sign_crs_flag}) * \text{lmcs_delta_abs_crs} \quad (104)$$

(170) It is a requirement of bitstream conformance that, when $\text{lmcsCW}[i]$ is not equal to 0, ($\text{lmcsCW}[i] + \text{lmcsDeltaCrs}$) shall be in the range of ($\text{OrgCW} \gg 3$) to ($(\text{OrgCW} \ll 3) - 1$), inclusive.

(171) The variable $\text{ChromaScaleCoeff}[i]$, with $i = 0 \dots 15$, is derived as follows: if ($\text{lmcsCW}[i] = 0$) $\text{ChromaScaleCoeff}[i] = (1 \ll 11)$ else $\text{ChromaScaleCoeff}[i] = \text{OrgCW} * (1 \ll 11) / (\text{lmcsCW}[i] + \text{lmcsDeltaCrs})$ scaling_matrix_for_lfnst_disabled_flag equal to 1 specifies that scaling matrices are not applied to blocks coded with low frequency non-separable transform (LFNST). scaling_matrix_for_lfnst_disabled_flag equal to 0 specifies that the scaling matrices may apply to the blocks coded with LFNST.

(172) scaling_list_chroma_present_flag equal to 1 specifies that chroma scaling lists are present in scaling_list_data ()
scaling_list_chroma_present_flag equal to 0 specifies that chroma scaling lists are not present in scaling_list_data () It is a requirement of bitstream conformance that scaling_list_chroma_present_flag shall be equal to 0 when ChromaArrayType is equal to 0, and shall be equal to 1 when ChromaArrayType is not equal to 0. scaling_list_copy_mode_flag[id] equal to 1 specifies that the values of the scaling list are the same as the values of a reference scaling list. The reference scaling list is specified by scaling_list_pred_id_delta[id].

(173) scaling_list_copy_mode_flag[id] equal to 0 specifies that scaling_list_pred_mode_flag is present. scaling_list_pred_mode_flag[id] equal to 1 specifies that the values of the scaling list can be predicted from a reference scaling list. The reference scaling list is specified by scaling_list_pred_id_delta[id]. scaling_list_pred_mode_flag[id] equal to 0 specifies that the values of the scaling list are explicitly signalled. When not present, the value of scaling_list_pred_mode_flag[id] is inferred to be equal to 0.

(174) scaling_list_pred_id_delta[id] specifies the reference scaling list used to derive the predicted scaling matrix $\text{ScalingMatrixPred}[id]$. When not present, the value of scaling_list_pred_id_delta[id] is inferred to be equal to 0. The value of scaling_list_pred_id_delta[id] shall be in the range of 0 to maxIdDelta with maxIdDelta derived depending on id as follows:

$$\text{maxIdDelta} = (\text{id} < 2) ? \text{id} : ((\text{id} < 8) ? (\text{id} - 2) : (\text{id} - 8)) \quad (106)$$

(175) The variables refId and matrixSize are derived as follows:

$$\text{refId} = \text{id} - \text{scaling_list_pred_id_delta}[id] \quad (107)$$

$$\text{matrixSize} = (\text{id} < 2) ? 2 : ((\text{id} < 8) ? 4 : 8) \quad (108)$$

(176) The (matrixSize) $\times$ (matrixSize) array $\text{ScalingMatrixPred}[x][y]$ with $x = 0 \dots \text{matrixSize} - 1$, $y = 0 \dots \text{matrixSize} - 1$ and the variable $\text{ScalingMatrixDCPred}$ are derived as follows: When both scaling_list_copy_mode_flag[id] and scaling_list_pred_mode_flag[id] are equal to 0, all elements of ScalingMatrixPred are set equal to 8, and the value of $\text{ScalingMatrixDCPred}$ is set equal to 8. Otherwise, when scaling_list_pred_id_delta[id] is equal to 0, all elements of ScalingMatrixPred are set equal to 16, and $\text{ScalingMatrixDCPred}$ is set equal to 16. Otherwise (either scaling_list_copy_mode_flag[id] or scaling_list_pred_mode_flag[id] is equal to 1 and scaling_list_pred_id_delta[id] is greater than 0), ScalingMatrixPred is set equal to $\text{ScalingMatrixRec}[\text{refId}]$, and the following applies for $\text{ScalingMatrixDCPred}$: If refId is greater than 13, $\text{ScalingMatrixDCPred}$ is set equal to $\text{ScalingMatrixDCRec}[\text{refId} - 14]$. Otherwise (refId is less than or equal to 13), $\text{ScalingMatrixDCPred}$ is set equal to $\text{ScalingMatrixPred}[0][0]$. scaling_list_de_coeff[id-14] is used to derive the value of the variable $\text{ScalingMatrixDC}[id-14]$ when id is greater than 13 as follows:

$$\text{ScalingMatrixDCRec}[id-14] = (\text{ScalingMatrixDCPred} + \text{scaling_list_de_coeff}[id-14]) \& 255 \quad (109)$$

(177) When not present, the value of scaling_list_de_coeff[id-14] is inferred to be equal to 0. The value of scaling_list_dc_coeff[id-14] shall be in the range of -128 to 127, inclusive. The value of $\text{ScalingMatrixDCRec}[id-14]$ shall be greater than 0.

(178) scaling_list_delta_coeff[id][i] specifies the difference between the current matrix coefficient $\text{ScalingList}[id][i]$ and the previous matrix coefficient $\text{ScalingList}[id][i-1]$, when scaling_list_copy_mode_flag[id] is equal to 0.

(179) The value of scaling_list_delta_coeff[id][i] shall be in the range of -128 to 127, inclusive. When scaling_list_copy_mode_flag[id] is equal to 1, all elements of $\text{ScalingList}[id]$ are set equal to 0.

(180) The (matrixSize) $\times$ (matrixSize) array $\text{ScalingMatrixRec}[id]$ is derived as follows:

$$\text{ScalingMatrixRec}[id][x][y] = (\text{ScalingMatrixPred}[x][y] + \text{ScalingList}[id][k]) \& 255 \text{ with } k = 0 \dots (\text{matrixSize} * \text{matrixSize} - 1), \quad (110)$$

$$X = \text{DiagScanOrder}[\text{Log2}(\text{matrixSize})][\text{Log2}(\text{matrixSize})][k][0], \text{ and } y = \text{DiagScanOrder}[\text{Log2}(\text{matrixSize})][\text{Log2}(\text{matrixSize})][k][1]$$

(181) The value of $\text{ScalingMatrixRec}[id][x][y]$ shall be greater than 0.

(182) 3.3. Picture Header (PH) Syntax and Semantics:

(183) In the latest VVC draft text, the PH syntax and semantics are as follows:

(184) TABLE-US-00010 Descriptor picture_header_rbsp() { picture_header_structure() rbsp_trailing_bits() }

(185) The PH RBSP contains a PH syntax structure, i.e., picture_header_structure().

(186) TABLE-US-00011 Descriptor picture_header_structure() { gdr_or_irap_pic_flag u(1) if(gdr_or_irap_pic_flag) gdr_pic_flag u(1)
ph_inter_slice_allowed_flag u(1) if(ph_inter_slice_allowed_flag) ph_intra_slice_allowed_flag u(1) non_reference_picture_flag u(1)
ph_pic_parameter_set_id ue(v) ph_pic_order_cnt_lsb u(v) if(gdr_or_irap_pic_flag) no_output_of_prior_pics_flag u(1) if(
gdr_pic_flag) recovery_poc_cnt ue(v) for(i = 0; i < NumExtraPhBits; i++) ph_extra_bit[i] u(1) if(sps_poc_msb_flag) {
ph_poc_msb_present_flag u(1) if(ph_poc_msb_present_flag) poc_msb_val u(v) } if(sps_alf_enabled_flag &&
alf_info_in_ph_flag) { ph_alf_enabled_flag u(1) if(ph_alf_enabled_flag) { ph_num_alf_aps_ids_luma u(3) for(i = 0; i <
ph_num_alf_aps_ids_luma; i++) ph_alf_aps_id_luma[i] u(3) if(ChromaArrayType != 0) ph_alf_chroma_idc u(2)
if(ph_alf_chroma_idc > 0) ph_alf_aps_id_chroma u(3) if(sps_ccalf_enabled_flag) { ph_cc_alf_cb_enabled_flag
u(1) if(ph_cc_alf_cb_enabled_flag) ph_cc_alf_cr_enabled_flag u(1) if(ph_cc_alf_cr_enabled_flag) ph_cc_alf_cr_aps_id u(3)
ph_cc_alf_cr_enabled_flag) { ph_cc_alf_cr_aps_id u(3) } } if(sps_lmcs_enabled_flag) { ph_lmcs_enabled_flag
u(1) if(ph_lmcs_enabled_flag) { ph_lmcs_aps_id u(2) if(ChromaArrayType != 0) ph_chroma_residual_scale_flag
u(1) } } if(sps_scaling_list_enabled_flag) { ph_scaling_list_present_flag u(1) if(ph_scaling_list_present_flag)
ph_scaling_list_aps_id u(3) } if(sps_virtual_boundaries_enabled_flag && !sps_virtual_boundaries_present_flag) {
ph_virtual_boundaries_present_flag u(1) if(ph_virtual_boundaries_present_flag) { ph_num_ver_virtual_boundaries u(2) for(i =
0; i < ph_num_ver_virtual_boundaries; i++) ph_virtual_boundaries_pos_x[i] u(13) ph_num_hor_virtual_boundaries u(2)
for(i = 0; i < ph_num_hor_virtual_boundaries; i++) ph_virtual_boundaries_pos_y[i] u(13) } } if(output_flag_present_flag)
pic_output_flag u(1) if(rpl_info_in_ph_flag) ref_pic_lists() if(partition_constraints_override_enabled_flag)
partition_constraints_override_flag u(1) if(ph_intra_slice_allowed_flag) { if(partition_constraints_override_flag) {
ph_log2_diff_min_qt_min_cb_intra_slice_luma ue(v) ph_max_mtt_hierarchy_depth_intra_slice_luma ue(v) if(
ph_max_mtt_hierarchy_depth_intra_slice_luma != 0) { ph_log2_diff_max_bt_min_qt_intra_slice_luma ue(v)
ph_log2_diff_max_tt_min_qt_intra_slice_luma ue(v) } if(qtbtt_dual_tree_intra_flag) {
ph_log2_diff_min_qt_min_cb_intra_slice_chroma ue(v) ph_max_mtt_hierarchy_depth_intra_slice_chroma ue(v) if(
ph_max_mtt_hierarchy_depth_intra_slice_chroma != 0) { ph_log2_diff_max_bt_min_qt_intra_slice_chroma ue(v)
ph_log2_diff_max_tt_min_qt_intra_slice_chroma ue(v) } } } if(cu_qp_delta_enabled_flag)
ph_cu_qp_delta_subdiv_intra_slice ue(v) if(pps_cu_chroma_qp_offset_list_enabled_flag) ph_cu_chroma_qp_offset_subdiv_intra_slice
ue(v) } if(ph_inter_slice_allowed_flag) { if(partition_constraints_override_flag) { ph_log2_diff_min_qt_min_cb_inter_slice ue(v)
ph_max_mtt_hierarchy_depth_inter_slice ue(v) if(ph_max_mtt_hierarchy_depth_inter_slice != 0) {
ph_log2_diff_max_bt_min_qt_inter_slice ue(v) ph_log2_diff_max_tt_min_qt_inter_slice ue(v) } } } if(cu_qp_delta_
enabled_flag) ph_cu_qp_delta_subdiv_inter_slice ue(v) if(pps_cu_chroma_qp_offset_list_enabled_flag)
ph_cu_chroma_qp_offset_subdiv_inter_slice ue(v) if(sps_temporal_mvp_enabled_flag) { ph_temporal_mvp_enabled_flag u(1)
if(ph_temporal_mvp_enabled_flag && rpl_info_in_ph_flag) { ph_collocated_from_l0_flag u(1) if((
ph_collocated_from_l0_flag && num_ref_entries [0][RplIdx[0]] > 1) || (!ph_collocated_from_l0_flag &&
num_ref_entries [1][RplIdx[1]] > 1)) ph_collocated_ref_idx ue(v) } } mvd_l1_zero_flag u(1)
if(sps_fpe1_mmvd_enabled_flag) ph_fpe1_mmvd_enabled_flag u(1) if(sps_bdof_pic_present_flag) ph_disable_bdof_flag u(1)
if(sps_dmvr_pic_present_flag) ph_disable_dmvr_flag u(1) if(sps_prof_pic_present_flag) ph_disable_prof_flag u(1)
if((pps_weighted_pred_flag || pps_weighted_bipred_flag) && wp_info_in_ph_flag) pred_weight_table() } if(
qp_delta_info_in_ph_flag) ph_qp_delta se(v) if(sps_joint_cbr_enabled_flag) ph_joint_cbr_sign_flag u(1) if(
sps_sao_enabled_flag && sao_info_in_ph_flag) { ph_sao_luma_enabled_flag u(1) if(ChromaArrayType != 0)
ph_sao_chroma_enabled_flag u(1) } if(sps_dep_quant_enabled_flag) ph_dep_quant_enabled_flag u(1) if(
sps_sign_data_hiding_enabled_flag && !ph_dep_quant_enabled_flag) pic_sign_data_hiding_enabled_flag u(1) if(
deblocking_filter_override_enabled_flag && dbf_info_in_ph_flag) { ph_deblocking_filter_override_flag u(1) if(
ph_deblocking_filter_override_flag) { ph_deblocking_filter_disabled_flag u(1) if(!ph_deblocking_filter_disabled_flag) {
ph_beta_offset_div2 se(v) ph_tc_offset_div2 se(v) ph_cb_beta_offset_div2 se(v) ph_cb_tc_offset_div2 se(v)
ph_cr_beta_offset_div2 se(v) ph_cr_tc_offset_div2 se(v) } } } if(picture_header_extension_present_flag) {
ph_extension_length ue(v) for(i = 0; i < ph_extension_length; i++) ph_extension_data_byte[i] u(8) } }

(187) The PH syntax structure contains information that is common for all slices of the coded picture associated with the PH syntax structure.

(188) gdr_or_irap_pic_flag equal to 1 specifies that the current picture is a gradual decoding refresh (GDR) or IRAP picture. gdr_or_irap_pic_flag equal to 0 specifies that the current picture may or may not be a GDR or IRAP picture.

(189) gdr_pic_flag equal to 1 specifies the picture associated with the PH is a GDR picture. gdr_pic_flag equal to 0 specifies that the picture associated with the PH is not a GDR picture. When not present, the value of gdr_pic_flag is inferred to be equal to 0. When gdr_enabled_flag is equal to 0, the value of gdr_pic_flag shall be equal to 0.

(190) ph_inter_slice_allowed_flag equal to 0 specifies that all coded slices of the picture have slice_type equal to 2. ph_inter_slice_allowed_flag equal to 1 specifies that there may or may not be one or more coded slices in the picture that have slice_type equal to 0 or 1.

(191) ph_intra_slice_allowed_flag equal to 0 specifies that all coded slices of the picture have slice_type equal to 0 or 1. ph_intra_slice_allowed_flag equal to 1 specifies that there may or may not be one or more coded slices in the picture that have slice_type equal to 2. When not present, the value of ph_intra_slice_allowed_flag is inferred to be equal to 1. NOTE 1—For bitstreams that are supposed to work subpicture based bitstream merging without the need of changing PH NAL units, the encoder is expected to set the values of both ph_inter_slice_allowed_flag and ph_intra_slice_allowed_flag equal to 1.

(192) non_reference_picture_flag equal to 1 specifies the picture associated with the PH is never used as a reference picture.

non_reference_picture_flag equal to 0 specifies the picture associated with the PH may or may not be used as a reference picture.

(193) ph_pic_parameter_set_id specifies the value of pps_pic_parameter_set_id for the PPS in use. The value of ph_pic_parameter_set_id shall be in the range of 0 to 63, inclusive.

(194) It is a requirement of bitstream conformance that the value of TemporalId of the PH shall be greater than or equal to the value of TemporalId of the PPS that has pps_pic_parameter_set_id equal to ph_pic_parameter_set_id.

(195) ph_pic_order_cnt_lsb specifies the picture order count modulo MaxPicOrderCntLsb for the current picture. The length of the

ph_pic_order_cnt_lsb syntax element is log₂ max_pic_order_cnt_lsb_minus4+4 bits. The value of the ph_pic_order_cnt_lsb shall be in the range of 0 to MaxPicOrderCntLsb-1, inclusive.

(196) no_output_of_prior_pics_flag affects the output of previously-decoded pictures in the decoded picture buffer (DPB) after the decoding of a CLVS start (CLVSS) picture that is not the first picture in the bitstream as specified in Annex C.

(197) recovery_poc_cnt specifies the recovery point of decoded pictures in output order. If the current picture is a GDR picture that is associated with the PH, and there is a picture picA that follows the current GDR picture in decoding order in the CLVS that has PicOrderCntVal equal to the

PicOrderCntVal of the current GDR picture plus the value of recovery_poc_cnt, the picture picA is referred to as the recovery point picture. Otherwise, the first picture in output order that has PicOrderCntVal greater than the PicOrderCntVal of the current picture plus the value of recovery_poc_cnt is referred to as the recovery point picture. The recovery point picture shall not precede the current GDR picture in decoding order. The value of recovery_poc_cnt shall be in the range of 0 to MaxPicOrderCntLsb-1, inclusive.

(198) When the current picture is a GDR picture, the variable RpPicOrderCntVal is derived as follows:

$$\text{RpPicOrderCntVal} = \text{PicOrderCntVal} + \text{recovery_poc_cnt} \quad (82)$$
NOTE 2-When gdr_enabled_flag is equal to 1 and PicOrderCntVal of the current picture is greater than or equal to RpPicOrderCntVal of the associated GDR picture, the current and subsequent decoded pictures in output order are exact match to the corresponding pictures produced by starting the decoding process from the previous IRAP picture, when present, preceding the associated GDR picture in decoding order.

(199) ph_extra_bit[i] may be equal to 1 or 0. Decoders conforming to this version of this Specification shall ignore the value of ph_extra_bit[i]. Its value does not affect decoder conformance to profiles specified in this version of specification.

(200) ph_poc_msb_present_flag equal to 1 specifies that the syntax element poc_msb_val is present in the PH. ph_poc_msb_present_flag equal to 0 specifies that the syntax element poc_msb_val is not present in the PH. When vps_independent_layer_flag[GeneralLayerIdx[nuh_layer_id]] is equal to 0 and there is a picture in the current AU in a reference layer of the current layer, the value of ph_poc_msb_present_flag shall be equal to 0.

(201) poc_msb_val specifies the picture order count (POC) most significant bit (MSB) value of the current picture. The length of the syntax element poc_msb_val is poc_msb_len_minus1+1 bits.

(202) ph_alf_enabled_flag equal to 1 specifies that adaptive loop filter is enabled for all slices associated with the PH and may be applied to Y, Cb, or Cr colour component in the slices. ph_alf_enabled_flag equal to 0 specifies that adaptive loop filter may be disabled for one, or more, or all slices associated with the PH. When not present, ph_alf_enabled_flag is inferred to be equal to 0.

(203) ph_num_alf_aps_ids_luma specifies the number of ALF APSs that the slices associated with the PH refers to.

(204) ph_alf_aps_id_luma[i] specifies the adaptation_parameter_set_id of the i-th ALF APS that the luma component of the slices associated with the PH refers to.

(205) The value of alf_luma_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_alf_aps_id_luma[i] shall be equal to 1.

(206) The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_alf_aps_id_luma[i] shall be less than or equal to the TemporalId of the picture associated with the PH. ph_alf_chroma_idc equal to 0 specifies that the adaptive loop filter is not applied to Cb and Cr colour components. ph_alf_chroma_idc equal to 1 indicates that the adaptive loop filter is applied to the Cb colour component. ph_alf_chroma_idc equal to 2 indicates that the adaptive loop filter is applied to the Cr colour component. ph_alf_chroma_idc equal to 3 indicates that the adaptive loop filter is applied to Cb and Cr colour components. When ph_alf_chroma_idc is not present, it is inferred to be equal to 0.

(207) ph_alf_aps_id_chroma specifies the adaptation_parameter_set_id of the ALF APS that the chroma component of the slices associated with the PH refers to.

(208) The value of alf_chroma_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_alf_aps_id_chroma shall be equal to 1.

(209) The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_alf_aps_id_chroma shall be less than or equal to the TemporalId of the picture associated with the PH.

(210) ph_cc_alf_cb_enabled_flag equal to 1 specifies that cross-component filter for Cb colour component is enabled for all slices associated with the PH and may be applied to Cb colour component in the slices. ph_cc_alf_cb_enabled_flag equal to 0 specifies that cross-component filter for Cb colour component may be disabled for one, or more, or all slices associated with the PH. When not present, ph_cc_alf_cb_enabled_flag is inferred to be equal to 0.

(211) ph_cc_alf_cb_aps_id specifies the adaptation_parameter_set_id of the ALF APS that the Cb colour component of the slices associated with the PH refers to.

(212) The value of alf_cc_cb_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_cc_alf_cb_aps_id shall be equal to 1.

(213) The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_cc_alf_cb_aps_id shall be less than or equal to the TemporalId of the picture associated with the PH. ph_cc_alf_cr_enabled_flag equal to 1 specifies that cross-component filter for Cr colour component is enabled for all slices associated with the PH and may be applied to Cr colour component in the slices. ph_cc_alf_cr_enabled_flag equal to 0 specifies that cross-component filter for Cr colour component may be disabled for one, or more, or all slices associated with the PH. When not present, ph_cc_alf_cr_enabled_flag is inferred to be equal to 0.

(214) ph_cc_alf_cr_aps_id specifies the adaptation_parameter_set_id of the ALF APS that the Cr colour component of the slices associated with the PH refers to.

(215) The value of alf_cc_cr_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_cc_alf_cr_aps_id shall be equal to 1.

(216) The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to ph_cc_alf_cr_aps_id shall be less than or equal to the TemporalId of the picture associated with the PH.

(217) ph_lmcs_enabled_flag equal to 1 specifies that luma mapping with chroma scaling is enabled for all slices associated with the PH. ph_lmcs_enabled_flag equal to 0 specifies that luma mapping with chroma scaling may be disabled for one, or more, or all slices associated with the PH. When not present, the value of ph_lmcs_enabled_flag is inferred to be equal to 0.

(218) ph_lmcs_aps_id specifies the adaptation_parameter_set_id of the LMCS APS that the slices associated with the PH refers to. The TemporalId of the APS NAL unit having aps_params_type equal to LMCS_APS and adaptation parameter_set_id equal to ph_lmcs_aps_id shall be less than or equal to the TemporalId of the picture associated with PH.

(219) ph_chroma_residual_scale_flag equal to 1 specifies that chroma residual scaling is enabled for the all slices associated with the PH. ph_chroma_residual_scale_flag equal to 0 specifies that chroma residual scaling may be disabled for one, or more, or all slices associated with the PH. When ph_chroma_residual_scale_flag is not present, it is inferred to be equal to 0.

(220) ph_scaling_list_present_flag equal to 1 specifies that the scaling list data used for the slices associated with the PH is derived based on the scaling list data contained in the referenced scaling list APS. ph_scaling_list_present_flag equal to 0 specifies that the scaling list data used for the slices associated with the PH is set to be equal to 16. When not present, the value of ph_scaling_list_present_flag is inferred to be equal to 0.

(221) ph_scaling_list_aps_id specifies the adaptation_parameter_set_id of the scaling list APS. The TemporalId of the APS NAL unit having aps_params_type equal to SCALING_APS and adaptation_parameter_set_id equal to ph_scaling_list_aps_id shall be less than or equal to the TemporalId of the picture associated with PH.

(222) ph_virtual_boundaries_present_flag equal to 1 specifies that information of virtual boundaries is signalled in the PH. ph_virtual_boundaries_present_flag equal to 0 specifies that information of virtual boundaries is not signalled in the PH. When there is one or more than one virtual boundaries signalled in the PH, the in-loop filtering operations are disabled across the virtual boundaries in the picture. The in-loop filtering operations include the deblocking filter, sample adaptive offset filter, and adaptive loop filter operations. When not present, the value of ph_virtual_boundaries_present_flag is inferred to be equal to 0.

(223) It is a requirement to conform to that, when subpic_info_present_flag is equal to 1, the value of ph_virtual_boundaries_present_flag shall be equal to 0.

(224) The variable VirtualBoundariesPresentFlag is derived as follows:

$$\text{VirtualBoundariesPresentFlag} = 0 \text{ if } (\text{sps_virtual_boundaries_enabled_flag}) \text{ VirtualBoundariesPresentFlag} = \text{sps_virtual_boundaries_present_flag} = \text{ph_virtual_boundaries_present_flag}$$
 (83)

(225) ph_num_ver_virtual_boundaries specifies the number of ph_virtual_boundaries_pos_x[i] syntax elements that are present in the PH. When ph_num_ver_virtual_boundaries is not present, it is inferred to be equal to 0.

(226) The variable NumVerVirtualBoundaries is derived as follows:

$$\text{NumVerVirtualBoundaries} = 0 \text{ if } (\text{sps_virtual_boundaries_enabled_flag}) \text{ NumVerVirtualBoundaries} = \text{sps_virtual_boundaries_present_flag} ?$$
 (84)

sps_num_ver_virtual_boundaries: ph_num_ver_virtual_boundaries ph_virtual_boundaries_pos_x[i] specifies the location of the i-th vertical virtual boundary in units of luma samples divided by 8. The value of ph_virtual_boundaries_pos_x[i] shall be in the range of 1 to $\text{Ceil}(\text{pic_width_in_luma_samples} \div 8) - 1$, inclusive.

(227) The list VirtualBoundariesPosX[i] for i ranging from 0 to NumVerVirtualBoundaries-1, inclusive, in units of luma samples, specifying the locations of the vertical virtual boundaries, is derived as follows:
for (i=0; i<NumVerVirtualBoundaries; i++) VirtualBoundariesPosX[i] = (sps_virtual_boundaries_present_flag ?
sps_virtual_boundaries_pos_x[i] : ph_virtual_boundaries_pos_x[i]) * 8 (85)

(228) The distance between any two vertical virtual boundaries shall be greater than or equal to CtbSize Y luma samples.
ph_num_hor_virtual_boundaries specifies the number of ph_virtual_boundaries_pos_y[i] syntax elements that are present in the PH. When ph_num_hor_virtual_boundaries is not present, it is inferred to be equal to 0.

(229) The parameter NumHorVirtualBoundaries is derived as follows:

$$\text{NumHorVirtualBoundaries} = 0 \text{ if } (\text{sps_virtual_boundaries_enabled_flag}) \text{ NumHorVirtualBoundaries} = \text{sps_virtual_boundaries_present_flag} ?$$
 (86)

(230) When sps_virtual_boundaries_enabled_flag is equal to 1 and ph_virtual_boundaries_present_flag is equal to 1, the sum of ph_num_ver_virtual_boundaries and ph_num_hor_virtual_boundaries shall be greater than 0.

(231) ph_virtual_boundaries_pos_y[i] specifies the location of the i-th horizontal virtual boundary in units of luma samples divided by 8. The value of ph_virtual_boundaries_pos_y[i] shall be in the range of 1 to $\text{Ceil}(\text{pic_height_in_luma_samples} \div 8) - 1$, inclusive.

(232) The list VirtualBoundariesPosY[i] for i ranging from 0 to NumHorVirtualBoundaries-1, inclusive, in units of luma samples, specifying the locations of the horizontal virtual boundaries, is derived as follows:
for (i=0; i<NumHorVirtualBoundaries; i++) VirtualBoundariesPosY[i] = (sps_virtual_boundaries_present_flag ? sps_virtual_boundaries_pos_y[i] :
ph_virtual_boundaries_pos_y[i]) * 8 (87)

(233) The distance between any two horizontal virtual boundaries shall be greater than or equal to CtbSize Y luma samples.

(234) pic_output_flag affects the decoded picture output and removal processes as specified in Annex C. When pic_output_flag is not present, it is inferred to be equal to 1.

(235) partition_constraints_override_flag equal to 1 specifies that partition constraint parameters are present in the PH.
partition_constraints_override_flag equal to 0 specifies that partition constraint parameters are not present in the PH. When not present, the value of partition_constraints_override_flag is inferred to be equal to 0.

(236) ph_log 2_diff_min_qt_min_cb_intra_slice_luma specifies the difference between the base 2 logarithm of the minimum size in luma samples of a luma leaf block resulting from quadtree splitting of a CTU and the base 2 logarithm of the minimum coding block size in luma samples for luma coding units (CUs) in the slices with slice_type equal to 2 (I) associated with the PH. The value of ph_log 2_diff_min_qt_min_cb_intra_slice_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of ph_log 2_diff_min_qt_min_cb_intra_slice_luma is inferred to be equal to sps_log 2_diff_min_qt_min_cb_intra_slice_luma.

(237) ph_max_mtt_hierarchy_depth_intra_slice_luma specifies the maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_max_mtt_hierarchy_depth_intra_slice_luma shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinCbLog2SizeY})$, inclusive. When not present, the value of ph_max_mtt_hierarchy_depth_intra_slice_luma is inferred to be equal to sps_max_mtt_hierarchy_depth_intra_slice_luma.

(238) ph_log 2_diff_max_bt_min_qt_intra_slice_luma specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_log 2_diff_max_bt_min_qt_intra_slice_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY}$, inclusive. When not present, the value of ph_log 2_diff_max_bt_min_qt_intra_slice_luma is inferred to be equal to sps_log 2_diff_max_bt_min_qt_intra_slice_luma.

(239) ph_log 2_diff_max_tt_min_qt_intra_slice_luma specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_log 2_diff_max_tt_min_qt_intra_slice_luma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY}$, inclusive. When not present, the value of ph_log 2_diff_max_tt_min_qt_intra_slice_luma is inferred to be equal to sps_log 2_diff_max_tt_min_qt_intra_slice_luma.

(240) ph_log 2_diff_min_qt_min_cb_intra_slice_chroma specifies the difference between the base 2 logarithm of the minimum size in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with treeType equal to DUAL_TREE_CHROMA and the base 2 logarithm of the minimum coding block size in luma samples for chroma CUs with treeType equal to DUAL_TREE_CHROMA in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_log 2_diff_min_qt_min_cb_intra_slice_chroma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of ph_log 2_diff_min_qt_min_cb_intra_slice_chroma is inferred to be equal to sps_log 2_diff_min_qt_min_cb_intra_slice_chroma.

(241) ph_max_mtt_hierarchy_depth_intra_slice_chroma specifies the maximum hierarchy depth for chroma coding units resulting from multi-type tree splitting of a chroma quadtree leaf with treeType equal to DUAL_TREE_CHROMA in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_max_mtt_hierarchy_depth_intra_slice_chroma shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinCbLog2SizeY})$, inclusive. When not present, the value of ph_max_mtt_hierarchy_depth_intra_slice_chroma is inferred to be equal to sps_max_mtt_hierarchy_depth_intra_slice_chroma.

(242) ph_log 2_diff_max_bt_min_qt_intra_slice_chroma specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with treeType equal to DUAL_TREE_CHROMA in slices with slice_type equal to 2 (I) associated with the PH. The value of ph_log 2_diff_max_bt_min_qt_intra_slice_chroma shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraC}$, inclusive. When not present, the value of ph_log 2_diff_max_bt_min_qt_intra_slice_chroma is inferred to be equal to sps_log 2_diff_max_bt_min_qt_intra_slice_chroma.

(243) ph_log 2_diff_max_tt_min_qt_intra_slice_chroma specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a chroma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a chroma leaf block resulting from quadtree splitting of a chroma CTU with treeType equal to DUAL_TREE_CHROMA in slices with slice_type equal to 2 (I)

associated with the PH. The value of `ph_log 2_diff_max_tt_min_qt_intra_slice_chroma` shall be in the range of 0 to `CtbLog2SizeY-MinQtLog2SizeIntraC`, inclusive. When not the value present, of `ph_log 2_diff_max_tt_min_qt_intra_slice_chroma` is inferred to be equal to `sps_log 2_diff_max_tt_min_qt_intra_slice_chroma`

(244) `ph_cu_qp_delta_subdiv_intra_slice` specifies the maximum `cbSubdiv` value of coding units in intra slice that convey `cu_qp_delta_abs` and `cu_qp_delta_sign_flag`. The value of `ph_cu_qp_delta_subdiv_intra_slice` shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY} + \text{ph_max_mtt_hierarchy_depth_intra_slice_luma})$, inclusive.

(245) When not present, the value of `ph_cu_qp_delta_subdiv_intra_slice` is inferred to be equal to 0.

(246) `ph_cu_chroma_qp_offset_subdiv_intra_slice` specifies the maximum `cbSubdiv` value of coding units in intra slice that convey `cu_chroma_qp_offset_flag`. The value of `ph_cu_chroma_qp_offset_subdiv_intra_slice` shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinQtLog2SizeIntraY} + \text{ph_max_mtt_hierarchy_depth_intra_slice_luma})$, inclusive.

(247) When not present, the value of `ph_cu_chroma_qp_offset_subdiv_intra_slice` is inferred to be equal to 0. `ph_log 2_diff_min_qt_min_cb_inter_slice` specifies the difference between the base 2 logarithm of the minimum size in luma samples of a luma leaf block resulting from quadtree splitting of a CTU and the base 2 logarithm of the minimum luma coding block size in luma samples for luma CUs in the slices with `slice_type` equal to 0 (B) or 1 (P) associated with the PH. The value of `ph_log 2_diff_min_qt_min_cb_inter_slice` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinCbLog2SizeY}$, inclusive. When not present, the value of `ph_log 2_diff_min_qt_min_cb_inter_slice` is inferred to be equal to `sps_log 2_diff_min_qt_min_cb_inter_slice`.

(248) `ph_max_mtt_hierarchy_depth_inter_slice` specifies the maximum hierarchy depth for coding units resulting from multi-type tree splitting of a quadtree leaf in slices with `slice_type` equal to 0 (B) or 1 (P) associated with the PH. The value of `ph_max_mtt_hierarchy_depth_inter_slice` shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinCbLog2SizeY})$, inclusive. When not present, the value of `ph_max_mtt_hierarchy_depth_inter_slice` is inferred to be equal to `sps_max_mtt_hierarchy_depth_inter_slice`.

(249) `ph_log 2_diff_max_bt_min_qt_inter_slice` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a binary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in the slices with `slice_type` equal to 0 (B) or 1 (P) associated with the PH. The value of `ph_log 2_diff_max_bt_min_qt_inter_slice` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY}$, inclusive. When not present, the value of `ph_log 2_diff_max_bt_min_qt_inter_slice` is inferred to be equal to `sps_log 2_diff_max_bt_min_qt_inter_slice`.

(250) `ph_log 2_diff_max_tt_min_qt_inter_slice` specifies the difference between the base 2 logarithm of the maximum size (width or height) in luma samples of a luma coding block that can be split using a ternary split and the minimum size (width or height) in luma samples of a luma leaf block resulting from quadtree splitting of a CTU in slices with `slice_type` equal to 0 (B) or 1 (P) associated with the PH. The value of `ph_log 2_diff_max_tt_min_qt_inter_slice` shall be in the range of 0 to $\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY}$, inclusive. When not present, the value of `ph_log 2_diff_max_tt_min_qt_inter_slice` is inferred to be equal to `sps_log 2_diff_max_tt_min_qt_inter_slice`.

(251) `ph_cu_qp_delta_subdiv_inter_slice` specifies the maximum `cbSubdiv` value of coding units that in inter slice convey `cu_qp_delta_abs` and `cu_qp_delta_sign_flag`. The value of `ph_cu_qp_delta_subdiv_inter_slice` shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY} + \text{ph_max_mtt_hierarchy_depth_inter_slice})$, inclusive.

(252) When not present, the value of `ph_cu_qp_delta_subdiv_inter_slice` is inferred to be equal to 0.

(253) `ph_cu_chroma_qp_offset_subdiv_inter_slice` specifies the maximum `cbSubdiv` value of coding units in inter slice that convey `cu_chroma_qp_offset_flag`. The value of `ph_cu_chroma_qp_offset_subdiv_inter_slice` shall be in the range of 0 to $2 * (\text{CtbLog2SizeY} - \text{MinQtLog2SizeInterY} + \text{ph_max_mtt_hierarchy_depth_inter_slice})$, inclusive. When not present, the value of `ph_cu_chroma_qp_offset_subdiv_inter_slice` is inferred to be equal to 0.

(254) `ph_temporal_mvp_enabled_flag` specifies whether temporal motion vector predictors can be used for inter prediction for slices associated with the PH. If `ph_temporal_mvp_enabled_flag` is equal to 0, the syntax elements of the slices associated with the PH shall be constrained such that no temporal motion vector predictor is used in decoding of the slices. Otherwise (`ph_temporal_mvp_enabled_flag` is equal to 1), temporal motion vector predictors may be used in decoding of the slices associated with the PH. When not present, the value of `ph_temporal_mvp_enabled_flag` is inferred to be equal to 0. When no reference picture in the DPB has the same spatial resolution as the current picture, the value of `ph_temporal_mvp_enabled_flag` shall be equal to 0.

(255) The maximum number of subblock-based merging motion vector prediction (MVP) candidates, `MaxNumSubblockMergeCand`, is derived as follows:

`if(sps_affine_enabled_flag)MaxNumSubblockMergeCand=5-five_minus_max_num_subblock_merge_cand else`
`MaxNumSubblockMergeCand=sps_sbtmvp_enabled_flag && ph_temporal_mvp_enabled_flag` (88)

The value of `MaxNumSubblockMergeCand` shall be in the range of 0 to 5, inclusive.

(256) `ph_collocated_from_10_flag` equal to 1 specifies that the collocated picture used for temporal motion vector prediction is derived from reference picture list 0. `ph_collocated_from_10_flag` equal to 0 specifies that the collocated picture used for temporal motion vector prediction is derived from reference picture list 1.

(257) `ph_collocated_ref_idx` specifies the reference index of the collocated picture used for temporal motion vector prediction.

(258) When `ph_collocated_from_10_flag` is equal to 1, `ph_collocated_ref_idx` refers to an entry in reference picture list 0, and the value of `ph_collocated_ref_idx` shall be in the range of 0 to `num_ref_entries[0][RplIdx[0]]-1`, inclusive. When `ph_collocated_from_10_flag` is equal to 0, `ph_collocated_ref_idx` refers to an entry in reference picture list 1, and the value of `ph_collocated_ref_idx` shall be in the range of 0 to `num_ref_entries[1][RplIdx[1]]-1`, inclusive. When not present, the value of `ph_collocated_ref_idx` is inferred to be equal to 0.

(259) `mvd_11_zero_flag` equal to 1 indicates that the `mvd_coding` (`x0`, `y0`, 1) syntax structure is not parsed and `MvdL1[x0][y0][compIdx]` and `MvdCpL1[x0][y0][cpIdx][compIdx]` are set equal to 0 for `compIdx=0 . . . 1` and `cpIdx=0 . . . 2`. `mvd_11_zero_flag` equal to 0 indicates that the `mvd_coding` (`x0`, `y0`, 1) syntax structure is parsed.

(260) `ph_fpel_mmvd_enabled_flag` equal to 1 specifies that merge mode with motion vector difference uses integer sample precision in the slices associated with the PH. `ph_fpel_mmvd_enabled_flag` equal to 0 specifies that merge mode with motion vector difference can use fractional sample precision in the slices associated with the PH. When not present, the value of `ph_fpel_mmvd_enabled_flag` is inferred to be 0.

(261) `ph_disable_bdof_flag` equal to 1 specifies that bi-directional optical flow inter prediction based inter bi-prediction is disabled in the slices associated with the PH. `ph_disable_bdof_flag` equal to 0 specifies that bi-directional optical flow inter prediction based inter bi-prediction may or may not be enabled in the slices associated with the PH.

(262) When `ph_disable_bdof_flag` is not present, the following applies: If `sps_bdof_enabled_flag` is equal to 1, the value of `ph_disable_bdof_flag` is inferred to be equal to 0. Otherwise (`sps_bdof_enabled_flag` is equal to 0), the value of `ph_disable_bdof_flag` is inferred to be equal to 1.

`ph_disable_dmvr_flag` equal to 1 specifies that decoder motion vector refinement based inter bi-prediction is disabled in the slices associated with the PH. `ph_disable_dmvr_flag` equal to 0 specifies that decoder motion vector refinement based inter bi-prediction may or may not be enabled in the slices associated with the PH.

(263) When `ph_disable_dmvr_flag` is not present, the following applies: If `sps_dmvr_enabled_flag` is equal to 1, the value of `ph_disable_dmvr_flag` is inferred to be equal to 0. Otherwise (`sps_dmvr_enabled_flag` is equal to 0), the value of `ph_disable_dmvr_flag` is inferred to be equal to 1.

`ph_disable_prof_flag` equal to 1 specifies that prediction refinement with optical flow is disabled in the slices associated with the PH.

`ph_disable_prof_flag` equal to 0 specifies that prediction refinement with optical flow may or may not be enabled in the slices associated with the

PH.

(264) When `ph_disable_prof_flag` is not present, the following applies: If `sps_affine_prof_enabled_flag` is equal to 1, the value of `ph_disable_prof_flag` is inferred to be equal to 0. Otherwise (`sps_affine_prof_enabled_flag` is equal to 0), the value of `ph_disable_prof_flag` is inferred to be equal to 1.

(265) `ph_qp_delta` specifies the initial value of `Qp.sub.Y` to be used for the coding blocks in the picture until modified by the value of `CuQpDeltaVal` in the coding unit layer.

(266) When `qp_delta_info_in_ph_flag` is equal to 1, the initial value of the `Qp.sub.Y` quantization parameter for all slices of the picture, `SliceQp.sub.Y`, is derived as follows:

$$\text{SliceQp.sub.Y} = 26 + \text{init_qp_minus26} + \text{ph_qp_delta} \quad (89)$$

The value of `SliceQp.sub.Y` shall be in the range of $-\text{QpBdOffset}$ to $+63$, inclusive. `ph_joint_cbr_sign_flag` specifies whether, in transform units with `tu_joint_cbr_residual_flag[x0][y0]` equal to 1, the collocated samples both chroma residual of components have inverted signs. When `tu_joint_cbr_residual_flag[x0][y0]` equal to 1 for a transform unit, `ph_joint_cbr_sign_flag` equal to 0 specifies that the sign of each residual sample of the Cr (or Cb) component is identical to the sign of the collocated Cb (or Cr) residual sample and `ph_joint_cbr_sign_flag` equal to 1 specifies that the sign of each residual sample of the Cr (or Cb) component is given by the inverted sign of the collocated Cb (or Cr) residual sample.

(267) `ph_sao_luma_enabled_flag` equal to 1 specifies that SAO is enabled for the luma component in all slices associated with the PH;

`ph_sao_luma_enabled_flag` equal to 0 specifies that SAO for the luma component may be disabled for one, or more, or all slices associated with the PH. When `ph_sao_luma_enabled_flag` is not present, it is inferred to be equal to 0.

(268) `ph_sao_chroma_enabled_flag` equal to 1 specifies that SAO is enabled for the chroma component in all slices associated with the PH;

`ph_sao_chroma_enabled_flag` equal to 0 specifies that SAO for chroma component may be disabled for one, or more, or all slices associated with the PH. When `ph_sao_chroma_enabled_flag` is not present, it is inferred to be equal to 0.

(269) `ph_dep_quant_enabled_flag` equal to 0 specifies that dependent quantization is disabled for the current picture. `ph_dep_quant_enabled_flag` equal to 1 specifies that dependent quantization is enabled for the current picture. When `ph_dep_quant_enabled_flag` is not present, it is inferred to be equal to 0.

(270) `pic_sign_data_hiding_enabled_flag` equal to 0 specifies that sign bit hiding is disabled for the current picture.

`pic_sign_data_hiding_enabled_flag` equal to 1 specifies that sign bit hiding is enabled for the current picture. When

`pic_sign_data_hiding_enabled_flag` is not present, it is inferred to be equal to 0.

(271) `ph_deblocking_filter_override_flag` equal to 1 specifies that deblocking parameters are present in the PH. `ph_deblocking_filter_override_flag` equal to 0 specifies that deblocking parameters are not present in the PH. When not present, the value of `ph_deblocking_filter_override_flag` is inferred to be equal to 0.

(272) `ph_deblocking_filter_disabled_flag` equal to 1 specifies that the operation of the deblocking filter is not applied for the slices associated with the PH. `ph_deblocking_filter_disabled_flag` equal to 0 specifies that the operation of the deblocking filter is applied for the slices associated with the PH. When `ph_deblocking_filter_disabled_flag` is not present, it is inferred to be equal to `pps_deblocking_filter_disabled_flag`.

(273) `ph_beta_offset_div2` and `ph_tc_offset_div2` specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the luma component for the slices associated with the PH. The values of `ph_beta_offset_div2` and `ph_to_offset_div2` shall both be in the range of -12 to 12 , inclusive. When not present, the values of `ph_beta_offset_div2` and `ph_tc_offset_div2` are inferred to be equal to `pps_beta_offset_div2` and `pps_tc_offset_div2`, respectively.

(274) `ph_cb_beta_offset_div2` and `ph_cb_tc_offset_div2` specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cb component for the slices associated with the PH. The values of `ph_cb_beta_offset_div2` and `ph_cb_tc_offset_div2` shall both be in the range of -12 to 12 , inclusive. When not present, the values of `ph_cb_beta_offset_div2` and `ph_cb_tc_offset_div2` are inferred to be equal to `pps_cb_beta_offset_div2` and `pps_cb_tc_offset_div2`, respectively.

(275) `ph_cr_beta_offset_div2` and `ph_cr_tc_offset_div2` specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cr component for the slices associated with the PH. The values of `ph_cr_beta_offset_div2` and `ph_cr_tc_offset_div2` shall both be in the range of -12 to 12 , inclusive. When not present, the values of `ph_cr_beta_offset_div2` and `ph_cr_to_offset_div2` are inferred to be equal to `pps_cr_beta_offset_div2` and `pps_cr_tc_offset_div2`, respectively.

(276) `ph_extension_length` specifies the length of the PH extension data in bytes, not including the bits used for signalling `ph_extension_length` itself. The value of `ph_extension_length` shall be in the range of 0 to 256, inclusive. When not present, the value of `ph_extension_length` is inferred to be equal to 0.

(277) `ph_extension_data_byte` may have any value. Decoders conforming to this version of this Specification shall ignore the value of `ph_extension_data_byte`. Its value does not affect decoder conformance to profiles specified in this version of specification.

(278) 3.4. Slice Header (SH) Syntax and Semantics

(279) In the latest VVC draft text, the SH syntax and semantics are as follows:

(280) TABLE-US-00012 Descriptor `slice_header()` {
 `picture_header_in_slice_header_flag` u(1) if(`picture_header_in_slice_header_flag`)
 `picture_header_structure()` if(`subpic_info_present_flag`) `slice_subpic_id` u(v) if((`rect_slice_flag` && `NumSlicesInSubpic`[`CurrSubpicIdx`] > 1) || (!`rect_slice_flag` && `NumTilesInPic` > 1)) `slice_address` u(v) for(i = 0; i < `NumExtraShBits`; i++) `sh_extra_bit`[i] u(1) if(!`rect_slice_flag` && `NumTilesInPic` > 1) `num_tiles_in_slice_minus1` ue(v) if(`ph_inter_slice_allowed_flag`) `slice_type` ue(v)
 if(`sps_alf_enabled_flag` && !`alf_info_in_ph_flag`) { `slice_alf_enabled_flag` u(1) if(`slice_alf_enabled_flag`) {
 `slice_num_alf_aps_ids_luma` u(3) for(i = 0; i < `slice_num_alf_aps_ids_luma`; i++) `slice_alf_aps_id_luma`[i] u(3) if(`ChromaArrayType` != 0)
 `slice_alf_chroma_idc` u(2) if(`slice_alf_chroma_idc`) `slice_alf_aps_id_chroma` u(3) if(`sps_ccalf_enabled_flag`) {
 `slice_cc_alf_cb_enabled_flag` u(1) if(`slice_cc_alf_cb_enabled_flag`)
 `slice_cc_alf_cb_aps_id` u(3) `slice_cc_alf_cr_enabled_flag` u(1) if(`slice_cc_alf_cr_enabled_flag`)
 `slice_cc_alf_cr_aps_id` u(3) } } if(`separate_colour_plane_flag` == 1) `colour_plane_id` u(2) if(!`rpl_info_in_ph_flag` && (`nal_unit_type` != `IDR_W_RADL` && `nal_unit_type` != `IDR_N_LP`) || `sps_idr_rpl_present_flag`) `ref_pic_lists()` if(`rpl_info_in_ph_flag` || (`nal_unit_type` != `IDR_W_RADL` && `nal_unit_type` != `IDR_N_LP`) || `sps_idr_rpl_present_flag`)) && (`slice_type` != I && `num_ref_entries`[0][`RplsIdx`[0]] > 1) || (`slice_type` == B && `num_ref_entries`[1][`RplsIdx`[1]] > 1)) {
 `num_ref_idx_active_override_flag` u(1) if(`num_ref_idx_active_override_flag`) for(i = 0; i < (`slice_type` == B ? 2 : 1); i++)
 if(`num_ref_entries`[i][`RplsIdx`[i]] > 1) `num_ref_idx_active_minus1`[i] ue(v) } if(`slice_type` != I) { if(`cabac_init_present_flag`) `cabac_init_flag` u(1) if(`ph_temporal_mvp_enabled_flag` && !`rpl_info_in_ph_flag`) { if(`slice_type` == B)
 `slice_collocated_from_10_flag` u(1) if((`slice_collocated_from_10_flag` && `NumRefIdxActive`[0] > 1) || (!
 `slice_collocated_from_10_flag` && `NumRefIdxActive`[1] > 1)) `slice_collocated_ref_idx` ue(v) } if(!`wp_info_in_ph_flag` && (`pps_weighted_pred_flag` && `slice_type` == P) || (`pps_weighted_bipred_flag` && `slice_type` == B))) `pred_weight_table()` }
 if(!`qp_delta_info_in_ph_flag`) `slice_qp_delta` se(v) if(`pps_slice_chroma_qp_offsets_present_flag`) { `slice_cb_qp_offset` se(v)
 `slice_cr_qp_offset` se(v) if(`sps_joint_cbr_enabled_flag`) `slice_joint_cbr_qp_offset` se(v) } if(`pps_cu_chroma_qp_offset_list_enabled_flag`) `cu_chroma_qp_offset_enabled_flag` u(1) if(`sps_sao_enabled_flag` && !`sao_info_in_ph_flag`)
 { `slice_sao_luma_flag` u(1) if(`ChromaArrayType` != 0) `slice_sao_chroma_flag` u(1) } if(`deblocking_filter_override_enabled_flag` && !`dbf_info_in_ph_flag`) `slice_deblocking_filter_override_flag` u(1) if(

`slice_deblocking_filter_override_flag ) { slice_deblocking_filter_disabled_flag ) {`
`slice_beta_offset_div2 se(v) slice_tc_offset_div2 se(v) slice_cb_beta_offset_div2 se(v) slicecb_tc_offset_div2 se(v)`
`slice_cr_beta_offset_div2 se(v) slice_cr_tc_offset_div2 se(v) } } slice_ts_residual_coding_disabled_flag u(1) if(`
`ph_lmcs_enabled_flag ) slice_lmcs_enabled_flag u(1) if( ph_scaling_list_enabled_flag ) slice_scaling_list_present_flag u(1) if(`
`NumEntryPoints > 0 ) { offset_len_minus1 ue(v) for( i = 0; i < NumEntryPoints; i++ ) entry_point_offset_minus1[i] u(v) } if(`
`slice_header_extension_present_flag ) { slice_header_extension_length ue(v) for( i = 0; i < slice_header_extension_length; i++ )`
`slice_header_extension_data_byte[i] u(8) } byte_alignment( ) }`
(281) The variable `CuQpDeltaVal`, specifying the difference between a luma quantization parameter for the coding unit containing `cu_qp_delta_abs` and its prediction, is set equal to 0. The variables `CuQpOffsetcb`, `CuQpOffsetcr`, and `CuQpOffsetcbCr`, specifying values to be used when determining the respective values of the `Qp'.sub.Cb`, `Qp'.sub.Cr`, and `Qp'.sub.CbCr` quantization parameters for the coding unit containing `cu_chroma_qp_offset_flag`, are all set equal to 0. `picture_header_in_slice_header_flag` equal to 1 specifies that the PH syntax structure is present in the slice header. `picture_header_in_slice_header_flag` equal to 0 specifies that the PH syntax structure is not present in the slice header.
(282) It is a requirement of bitstream conformance that the value of `picture_header_in_slice_header_flag` shall be the same in all coded slices in a CLVS.
(283) When `picture_header_in_slice_header_flag` is equal to 1 for a coded slice, it is a requirement of bitstream conformance that no VCL NAL unit with `nal_unit_type` equal to `PH_NUT` shall be present in the CLVS.
(284) When `picture_header_in_slice_header_flag` is equal to 0, all coded slices in the current picture shall have `picture_header_in_slice_header_flag` is equal to 0, and the current PU shall have a PH NAL unit.
(285) `slice_subpic_id` specifies the subpicture ID of the subpicture that contains the slice. If `slice_subpic_id` is present, the value of the variable `CurrSubpicIdx` is derived to be such that `SubpicIdVal[CurrSubpicIdx]` is equal to `slice_subpic_id`. Otherwise (`slice_subpic_id` is not present), `CurrSubpicIdx` is derived to be equal to 0. The length of `slice_subpic_id` is `sps_subpic_id_len_minus1+1` bits.
(286) `slice_address` specifies the slice address of the slice. When not present, the value of `slice_address` is inferred to be equal to 0. When `rect_slice_flag` is equal to 1 and `NumSlicesInSubpic[CurrSubpicIdx]` is equal to 1, the value of `slice_address` is inferred to be equal to 0.
(287) If `rect_slice_flag` is equal to 0, the following applies: The slice address is the raster scan tile index. The length of `slice_address` is `Ceil(Log2(NumTilesInPic))` bits. The value of `slice_address` shall be in the range of 0 to `NumTilesInPic-1`, inclusive.
(288) Otherwise (`rect_slice_flag` is equal to 1), the following applies: The slice address is the subpicture-level slice index of the slice. The length of `slice_address` is `Ceil(Log2(NumSlicesInSubpic[CurrSubpicIdx]))` bits. The value of `slice_address` shall be in the range of 0 to `NumSlicesInSubpic[CurrSubpicIdx]-1`, inclusive.
(289) It is a requirement of bitstream conformance that the following constraints apply: If `rect_slice_flag` is equal to 0 or `subpic_info_present_flag` is equal to 0, the value of `slice_address` shall not be equal to the value of `slice_address` of any other coded slice NAL unit of the same coded picture. Otherwise, the pair of `slice_subpic_id` and `slice_address` values shall not be equal to the pair of `slice_subpic_id` and `slice_address` values of any other coded slice NAL unit of the same coded picture. The shapes of the slices of a picture shall be such that each CTU, when decoded, shall have its entire left boundary and entire top boundary consisting of a picture boundary or consisting of boundaries of previously decoded CTU(s).
(290) `sh_extra_bit[i]` may be equal to 1 or 0. Decoders conforming to this version of this Specification shall ignore the value of `sh_extra_bit[i]`. Its value does not affect decoder conformance to profiles specified in this version of specification.
(291) `num_tiles_in_slice_minus1` plus 1, when present, specifies the number of tiles in the slice. The value of `num_tiles_in_slice_minus1` shall be in the range of 0 to `NumTilesInPic-1`, inclusive.
(292) The variable `NumCtusInCurrSlice`, which specifies the number of CTUs in the current slice, and the list `CtbAddrInCurrSlice[i]`, for `i` ranging from 0 to `NumCtusInCurrSlice-1`, inclusive, specifying the picture raster scan address of the `i`-th CTB within the slice, are derived as follows:
(293) TABLE-US-00013 `if(rect_slice_flag) { picLevelSliceIdx = slice_address for(j = 0; j < CurrSubpicIdx; j++) picLevelSliceIdx += NumSlicesInSubpic[j] NumCtusInCurrSlice = NumCtusInSlice[picLevelSliceIdx] for(i = 0; i < NumCtusInCurrSlice; i++) CtbAddrInCurrSlice[i] = CtbAddrInSlice[picLevelSliceIdx][i] (117) } else { NumCtusInCurrSlice = 0 for(tileIdx = slice_address; tileIdx <= slice_address + num_tiles_in_slice_minus1; tileIdx++) { tileX = tileIdx % NumTileColumns tileY = tileIdx / NumTileColumns for(ctbY = tileRowBd[tileY]; ctbY < tileRowBd[tileY + 1]; ctbY++) { for(ctbX = tileColBd[tileX]; ctbX < tileColBd[tileX + 1]; ctbX++) { CtbAddrInCurrSlice[NumCtusInCurrSlice] = ctbY * PicWidthInCtb + ctbX NumCtusInCurrSlice++ } } }`
(294) `slice_type` specifies the coding type of the slice according to Table 9.
(295) TABLE-US-00014 TABLE 9 Name association to `slice_type`

<code>slice_type</code>	Name of slice_type
0	B (B slice)
1	P (P slice)
2	I (I slice)

(296) When not present, the value of `slice_type` is inferred to be equal to 2.
(297) When `ph_intra_slice_allowed_flag` is equal to 0, the value of `slice_type` shall be equal to 0 or 1. When `nal_unit_type` is in the range of `IDR_W_RADL` to `CRA_NUT`, inclusive, and `vps_independent_layer_flag[GeneralLayerIdx[nuh_layer_id]]` is equal to 1, `slice_type` shall be equal to 2.
(298) The variables `MinQtLog2SizeY`, `MinQtLog2SizeC`, `MinQtSizeY`, `MinQtSizeC`, `MaxBtSizeY`, `MaxBtSizeC`, `MinBtSizeY`, `MaxTtSizeY`, `MaxTtSizeC`, `MinTtSizeY`, `MaxMttDepthY` and `MaxMttDepthC` are derived as follows: If `slice_type` equal to 2 (I), the following applies:
`MinQtLog2SizeY=MinCbLog2SizeY+ph_log_2_diff_min_qt_min_cb_intra_slice_luma (119)`
`MinQtLog2SizeC=MinCbLog2SizeY+ph_log_2_diff_min_qt_min_cb_intra_slice_chroma (120)`
`MaxBtSizeY=1<<(MinQtLog2SizeY+ph_log_2_diff_max_bt_min_qt_intra_slice_luma) (121)`
`MaxBtSizeC=1<<(MinQtLog2SizeC+ph_log_2_diff_max_bt_min_qt_intra_slice_chroma) (122)`
`MaxTtSizeY=1<<(MinQtLog2SizeY+ph_log_2_diff_max_tt_min_qt_intra_slice_luma) (123)`
`MaxTtSizeC=1<<(MinQtLog2SizeC+ph_log_2_diff_max_tt_min_qt_intra_slice_chroma) (124)`
`MaxMttDepthY=ph_max_mtt_hierarchy_depth_intra_slice_luma (125)`
`MaxMttDepthC=ph_max_mtt_hierarchy_depth_intra_slice_chroma (126)`
`CuQpDeltaSubdiv=ph_cu_qp_delta_subdiv_intra_slice (127)`
`CuChromaQpOffsetSubdiv=ph_cu_chroma_qp_offset_subdiv_intra_slice (128)`
Otherwise (`slice_type` equal to 0 (B) or 1 (P)), the following applies:
`MinQtLog2SizeY=MinCbLog2SizeY+ph_log_2_diff_min_qt_min_cb_inter_slice (129)`
`MinQtLog2SizeC=MinCbLog2SizeY+ph_log_2_diff_min_qt_min_cb_inter_slice (130)`
`MaxBtSizeY=1<<(MinQtLog2SizeY+ph_log_2_diff_max_bt_min_qt_inter_slice) (131)`
`MaxBtSizeC=1<<(MinQtLog2SizeC+ph_log_2_diff_max_bt_min_qt_inter_slice) (132)`
`MaxTtSizeY=1<<(MinQtLog2SizeY+ph_log_2_diff_max_tt_min_qt_inter_slice) (133)`
`MaxTtSizeC=1<<(MinQtLog2SizeC+ph_log_2_diff_max_tt_min_qt_inter_slice) (134)`
`MaxMttDepthY=ph_max_mtt_hierarchy_depth_inter_slice (135)`
`MaxMttDepthC=ph_max_mtt_hierarchy_depth_inter_slice (136)`
`CuQpDeltaSubdiv=ph_cu_qp_delta_subdiv_inter_slice (137)`
`CuChromaQpOffsetSubdiv=ph_cu_chroma_qp_offset_subdiv_inter_slice (138)`
The following applies:

MinQtSizeY=1<<MinQtLog2Size Y (139)

MinQtSizeC=1<<MinQtLog2SizeC (140)

MinBtSizeY=1<<MinCbLog2Size Y (141)

MinTtSizeY=1<<MinCbLog2SizeY (142)

slice_alf_enabled_flag equal to 1 specifies that adaptive loop filter is enabled and may be applied to Y, Cb, or Cr colour component in a slice.

slice_alf_enabled_flag equal to 0 specifies that adaptive loop filter is disabled for all colour components in a slice. When not present, the value of slice_alf_enabled_flag is inferred to be equal to ph_alf_enabled_flag.

(299) slice_num_alf_aps_ids_luma specifies the number of ALF APSs that the slice refers to. When slice_alf_enabled_flag is equal to 1 and slice_num_alf_aps_ids_luma is not present, the value of slice_num_alf_aps_ids_luma is inferred to be equal to the value of ph_num_alf_aps_ids_luma.

(300) slice_alf_aps_id_luma[i] specifies the adaptation_parameter_set_id of the i-th ALF APS that the luma component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_alf_aps_id_luma[i] shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_alf_enabled_flag is equal to 1 and slice_alf_aps_id_luma[i] is not present, the value of slice_alf_aps_id_luma[i] is inferred to be equal to the value of ph_alf_aps_id_luma[i].

(301) The value of alf_luma_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_alf_aps_id_luma[i] shall be equal to 1.

(302) slice_alf_chroma_idc equal to 0 specifies that the adaptive loop filter is not applied to Cb and Cr colour components. slice_alf_chroma_idc equal to 1 indicates that the adaptive loop filter is applied to the Cb colour component. slice_alf_chroma_idc equal to 2 indicates that the adaptive loop filter is applied to the Cr colour component. slice_alf_chroma_idc equal to 3 indicates that the adaptive loop filter is applied to Cb and Cr colour components. When slice_alf_chroma_idc is not present, it is inferred to be equal to ph_alf_chroma_idc.

(303) slice_alf_aps_id_chroma specifies the adaptation_parameter_set_id of the ALF APS that the chroma component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_alf_aps_id_chroma shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_alf_enabled_flag is equal to 1 and slice_alf_aps_id_chroma is not present, the value of slice_alf_aps_id_chroma is inferred to be equal to the value of ph_alf_aps_id_chroma.

(304) The value of alf_chroma_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_alf_aps_id_chroma shall be equal to 1.

(305) slice_cc_alf_cb_enabled_flag equal to 0 specifies that the cross-component filter is not applied to the Cb colour component.

slice_cc_alf_cb_enabled_flag equal to 1 indicates that the cross-component filter is enabled and may be applied to the Cb colour component. When slice_cc_alf_cb_enabled_flag is not present, it is inferred to be equal to ph_cc_alf_cb_enabled_flag.

(306) slice_cc_alf_cb_aps_id specifies the adaptation_parameter_set_id that the Cb colour component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cb_aps_id shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_cc_alf_cb_enabled_flag is equal to 1 and slice_cc_alf_cb_aps_id is not present, the value of slice_cc_alf_cb_aps_id is inferred to be equal to the value of ph_cc_alf_cb_aps_id.

(307) The value of alf_cc_cb_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cb_aps_id shall be equal to 1.

(308) slice_cc_alf_cr_enabled_flag equal to 0 specifies that the cross-component filter is not applied to the Cr colour component.

slice_cc_alf_cr_enabled_flag equal to 1 indicates that the cross-component adaptive loop filter is enabled and may be applied to the Cr colour component. When slice_cc_alf_cr_enabled_flag is not present, it is inferred to be equal to ph_cc_alf_cr_enabled_flag.

(309) slice_cc_alf_cr_aps_id specifies the adaptation_parameter_set_id that the Cr colour component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cr_aps_id shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_cc_alf_cr_enabled_flag is equal to 1 and slice_cc_alf_cr_aps_id is not present, the value of slice_cc_alf_cr_aps_id is inferred to be equal to the value of ph_cc_alf_cr_aps_id. The value of alf_cc_cr_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cr_aps_id shall be equal to 1.

(310) colour_plane_id identifies the colour plane associated with the current slice when separate_colour_plane_flag is equal to 1. The value of colour_plane_id shall be in the range of 0 to 2, inclusive. colour_plane_id values 0, 1 and 2 correspond to the Y, Cb and Cr planes, respectively. The value 3 of colour_plane_id is reserved for future use by ITU-T|ISO/IEC. NOTE 1—There is no dependency between the decoding processes of different colour planes of one picture. num_ref_idx_active_override_flag equal to 1 specifies that the syntax element num_ref_idx_active_minus1[0] is present for P and B slices and the syntax element num_ref_idx_active_minus1[1] is present for B slices. num_ref_idx_active_override_flag equal to 0 specifies that the syntax elements num_ref_idx_active_minus1[0] and num_ref_idx_active_minus1[1] are not present. When not present, the value of num_ref_idx_active_override_flag is inferred to be equal to 1.

(311) num_ref_idx_active_minus1[i] is used for the derivation of the variable NumRefIdxActive[i] as specified by Equation 143. The value of num_ref_idx_active_minus1[i] shall be in the range of 0 to 14, inclusive.

(312) For i equal to 0 or 1, when the current slice is a B slice, num_ref_idx_active_override_flag is equal to 1, and num_ref_idx_active_minus1[i] is not present, num_ref_idx_active_minus1[i] is inferred to be equal to 0.

(313) When the current slice is a P slice, num_ref_idx_active_override_flag is equal to 1, and num_ref_idx_active_minus1[0] is not present, num_ref_idx_active_minus1[0] is inferred to be equal to 0.

(314) The variable NumRefIdxActive[i] is derived as follows:

(315) TABLE-US-00015 for(i = 0; i < 2; i++) { if(slice_type == B || (slice_type == P && i == 0)) { if(num_ref_idx_active_override_flag) NumRefIdxActive[i] = num_ref_idx_active_minus1[i] + 1 (143) else { if(num_ref_entries[i][RplIdx[i]] >= num_ref_idx_default_active_minus1[i] + 1) NumRefIdxActive[i] = num_ref_idx_default_active_minus1[i] + 1 else NumRefIdxActive[i] = num_ref_entries[i][RplIdx[i]] } } else /* slice_type == I || (slice_type == P && i == 1) */ NumRefIdxActive[i] = 0 }

(316) The value of NumRefIdxActive[i]-1 specifies the maximum reference index for reference picture list i that may be used to decode the slice. When the value of NumRefIdxActive[i] is equal to 0, no reference index for reference picture list i may be used to decode the slice.

(317) When the current slice is a P slice, the value of NumRefIdxActive[0] shall be greater than 0.

(318) When the current slice is a B slice, both NumRefIdxActive[0] and NumRefIdxActive[1] shall be greater than 0. cabac_init_flag specifies the method for determining the initialization table used in the initialization process for context variables. When cabac_init_flag is not present, it is inferred to be equal to 0.

(319) slice_collocated_from_10_flag equal to 1 specifies that the collocated picture used for temporal motion vector prediction is derived from reference picture list 0. slice_collocated_from_10_flag equal to 0 specifies that the collocated picture used for temporal motion vector prediction is derived from reference picture list 1.

(320) When slice_type is equal to B or P, ph_temporal_mvp_enabled_flag is equal to 1, and slice_collocated_from_10_flag is not present, the following applies: If rpl_info_in_ph_flag is equal to 1, slice_collocated_from_10_flag is inferred to be equal to ph_collocated_from_10_flag. Otherwise (rpl_info_in_ph_flag is equal to 0 and slice_type is equal to P), the value of slice_collocated_from_10_flag is inferred to be equal to 1.

(321) slice_collocated_ref_idx specifies the reference index of the collocated picture used for temporal motion vector prediction.

(322) When slice_type is equal to P or when slice_type is equal to B and slice_collocated_from_10_flag is equal to 1, slice_collocated_ref_idx refers to an entry in reference picture list 0, and the value of slice_collocated_ref_idx shall be in the range of 0 to NumRefIdxActive[0]-1, inclusive.

(323) When slice_type is equal to B and slice_collocated_from_10_flag is equal to 0, slice_collocated_ref_idx refers to an entry in reference picture list 1, and the value of slice_collocated_ref_idx shall be in the range of 0 to NumRefIdxActive[1]-1, inclusive.

(324) When slice_collocated_ref_idx is not present, the following applies: If rpl_info_in_ph_flag is equal to 1, the value of slice_collocated_ref_idx is inferred to be equal to ph_collocated_ref_idx. Otherwise (rpl_info_in_ph_flag is equal to 0), the value of slice_collocated_ref_idx is inferred to be equal to 0.

(325) It is a requirement of bitstream conformance that the picture referred to by slice_collocated_ref_idx shall be the same for all slices of a coded picture.

(326) It is a requirement of bitstream conformance that the values of pic_width_in_luma_samples and pic_height_in_luma_samples of the reference picture referred to by slice_collocated_ref_idx shall be equal to the values of pic_width_in_luma_samples and pic_height_in_luma_samples, respectively, of the current picture, and RprConstraintsActive[slice_collocated_from_10_flag? 0:1][slice_collocated_ref_idx] shall be equal to 0.

(327) slice_qp_delta specifies the initial value of Qp.sub.Y to be used for the coding blocks in the slice until modified by the value of CuQpDeltaVal in the coding unit layer.

(328) When qp_delta_info_in_ph_flag is equal to 0, the initial value of the Qp.sub.Y quantization parameter for the slice, SliceQp.sub.Y, is derived as follows:

$$\text{SliceQp.sub.Y} = 26 + \text{init_qp_minus}26 + \text{slice_qp_delta} \quad (144)$$

(329) The value of SliceQp.sub.Y shall be in the range of -QpBdOffset to +63, inclusive.

(330) When either of the following conditions is true: The value of wp_info_in_ph_flag is equal to 1, pps_weighted_pred_flag is equal to 1, and slice_type is equal to P. The value of wp_info_in_ph_flag is equal to 1, pps_weighted_bipred_flag is equal to 1, and slice_type is equal to B.

(331) the following applies: The value of NumRefIdxActive[0] shall be less than or equal to the value of NumWeightsL0. For each reference picture index RefPicList[0][i] for i in the range of 0 to NumRefIdxActive[0]-1, inclusive, the luma weight, Cb weight, and Cr weight that apply to the reference picture index are LumaWeightL0[i], ChromaWeightL0[0][i], and ChromaWeightL0[1][i], respectively.

(332) When wp_info_in_ph_flag is equal to 1, pps_weighted_bipred_flag is equal to 1, and slice_type is equal to B, the following applies: The value of NumRefIdxActive[1] shall be less than or equal to the value of NumWeightsL1. For each reference picture index RefPicList[1][i] for i in the range of 0 to NumRefIdxActive[1]-1, inclusive, the luma weight, Cb weight, and Cr weight that apply to the reference picture index are LumaWeightL1[i], ChromaWeightL1[0][i], and ChromaWeightL1[1][i], respectively.

(333) slice_cb_qp_offset specifies a difference to be added to the value of pps_cb_qp_offset when determining the value of the Qp'.sub.Cb quantization parameter. The value of slice_cb_qp_offset shall be in the range of -12 to +12, inclusive. When slice_cb_qp_offset is not present, it is inferred to be equal to 0. The value of pps_cb_qp_offset+slice_cb_qp_offset shall be in the range of -12 to +12, inclusive.

(334) slice_cr_qp_offset specifies a difference to be added to the value of pps_cr_qp_offset when determining the value of the Qp'.sub.Cr quantization parameter. The value of slice_cr_qp_offset shall be in the range of -12 to +12, inclusive. When slice_cr_qp_offset is not present, it is inferred to be equal to 0. The value of pps_cr_qp_offset+slice_cr_qp_offset shall be in the range of -12 to +12, inclusive.

(335) slice_joint_cbr_qp_offset specifies a difference to be added to the value of pps_joint_cbr_qp_offset_value when determining the value of the Qp'.sub.CbCr. The value of slice_joint_cbr_qp_offset shall be in the range of -12 to +12, inclusive. When slice_joint_cbr_qp_offset is not present, it is inferred to be equal to 0. The value of pps_joint_cbr_qp_offset_value+slice_joint_cbr_qp_offset shall be in the range of -12 to +12, inclusive.

(336) cu_chroma_qp_offset_enabled_flag equal to 1 specifies that the cu_chroma_qp_offset_flag may be present in the transform unit and palette coding syntax. cu_chroma_qp_offset_enabled_flag equal to 0 specifies that the cu_chroma_qp_offset_flag is not present in the transform unit or palette coding syntax. When not present, the value of cu_chroma_qp_offset_enabled_flag is inferred to be equal to 0.

(337) slice_sao_luma_flag equal to 1 specifies that SAO is enabled for the luma component in the current slice; slice_sao_luma_flag equal to 0 specifies that SAO is disabled for the luma component in the current slice. When slice_sao_luma_flag is not present, it is inferred to be equal to ph_sao_luma_enabled_flag.

(338) slice_sao_chroma_flag equal to 1 specifies that SAO is enabled for the chroma component in the current slice; slice_sao_chroma_flag equal to 0 specifies that SAO is disabled for the chroma component in the current slice. When slice_sao_chroma_flag is not present, it is inferred to be equal to ph_sao_chroma_enabled_flag.

(339) slice_deblocking_filter_override_flag equal to 1 specifies that deblocking parameters are present in the slice header. slice_deblocking_filter_override_flag equal to 0 specifies that deblocking parameters are not present in the slice header. When not present, the value of slice_deblocking_filter_override_flag is inferred to be equal to ph_deblocking_filter_override_flag.

(340) slice_deblocking_filter_disabled_flag equal to 1 specifies that the operation of the deblocking filter is not applied for the current slice. slice_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for the current slice. When slice_deblocking_filter_disabled_flag is not present, it is inferred to be equal to ph_deblocking_filter_disabled_flag.

(341) slice_beta_offset_div2 and slice_tc_offset_div2 specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the luma component for the current slice. The values of slice_beta_offset_div2 and slice_to_offset_div2 shall both be in the range of -12 to 12, inclusive. When not present, the values of slice_beta_offset_div2 and slice_to_offset_div2 are inferred to be equal to ph_beta_offset_div2 and ph_tc_offset_div2, respectively.

(342) slice_cb_beta_offset_div2 and slice_cb_tc_offset_div2 specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cb component for the current slice. The values of slice_cb_beta_offset_div2 and slice_cb_tc_offset_div2 shall both be in the range of -12 to 12, inclusive. When not present, the values of slice_cb_beta_offset_div2 and slice_cb_to_offset_div2 are inferred to be equal to ph_cb_beta_offset_div2 and ph_cb_tc_offset_div2, respectively.

(343) slice_cr_beta_offset_div2 and slice_cr_tc_offset_div2 specify the deblocking parameter offsets for β and tC (divided by 2) that are applied to the Cr component for the current slice. The values of slice_cr_beta_offset_div2 and slice_cr_to_offset_div2 shall both be in the range of -12 to 12, inclusive. When not present, the values of slice_cr_beta_offset_div2 and slice_cr_to_offset_div2 are inferred to be equal to ph_cr_beta_offset_div2 and ph_cr_tc_offset_div2, respectively.

(344) slice_ts_residual_coding_disabled_flag equal to 1 specifies that the residual_coding () syntax structure is used to parse the residual samples of a transform skip block for the current slice. slice_ts_residual_coding_disabled_flag equal to 0 specifies that the residual_ts_coding () syntax structure is used to parse the residual samples of a transform skip block for the current slice. When slice_ts_residual_coding_disabled_flag is not present, it is inferred to be equal to 0. slice_lmcs_enabled_flag equal to 1 specifies that luma mapping with chroma scaling is enabled for the current slice. slice_lmcs_enabled_flag equal to 0 specifies that luma mapping with chroma scaling is not enabled for the current slice. When slice_lmcs_enabled_flag is not present, it is inferred to be equal to 0.

(345) slice_scaling_list_present_flag equal to 1 specifies that the scaling list data used for the current slice is derived based on the scaling list data contained in the referenced scaling list APS with aps_params_type equal to SCALING_APS and adaptation_parameter_set_id equal to ph_scaling_list_aps_id. slice_scaling_list_present_flag equal to 0 specifies that the scaling list data used for the current picture is the default scaling list data derived specified in clause 7.4.3.21.

(346) When not present, the value of slice_scaling_list_present_flag is inferred to be equal to 0.

(347) The variable NumEntryPoints, which specifies the number of entry points in the current slice, is derived as follows: NumEntryPoints=0 for

(i=1; i<NumCtbsInCurrSlice; i++) { ctbAddrX = CtbAddrInCurrSlice[i] % PicWidthInCtbsY ctbAddrY = CtbAddrInCurrSlice[i]/PicWidthInCtbsY (145) prevCtbAddrX = CtbAddrInCurrSlice[i-1] % PicWidthInCtbsY prevCtbAddrY = CtbAddrInCurrSlice[i-1]/PicWidthInCtbsY if (CtbToTileRowBd[ctbAddrY] != CtbToTileRowBd[prevCtbAddrY] || | CtbToTileColBd[ctbAddrX] != CtbToTileColBd[prevCtbAddrX] || (ctbAddrY != prevCtbAddrY && sps_wpp_entry_point_offsets_present_flag)) NumEntryPoints++ }

(348) offset_len_minus1 plus 1 specifies the length, in bits, of the entry_point_offset_minus1[i] syntax elements. The value of offset_len_minus1 shall be in the range of 0 to 31, inclusive.

(349) entry_point_offset_minus1[i] plus 1 specifies the i-th entry point offset in bytes, and is represented by offset_len_minus1 plus 1 bits. The slice data that follow the slice header consists of NumEntryPoints+1 subsets, with subset index values ranging from 0 to NumEntryPoints, inclusive. The first byte of the slice data is considered byte 0. When present, emulation prevention bytes that appear in the slice data portion of the coded slice NAL unit are counted as part of the slice data for purposes of subset identification. Subset 0 consists of bytes 0 to entry_point_offset_minus1[0], inclusive, of the coded slice data, subset k, with k in the range of 1 to NumEntryPoints-1, inclusive, consists of bytes firstByte[k] to lastByte[k], inclusive, of the coded slice data with firstByte[k] and lastByte[k] defined as:

firstByte[k] = >K=1(entry_point_offset_minus1[n-1]+1) (146)

lastByte[k] = firstByte[k] + entry_point_offset_minus1[k] (147)

(350) The last subset (with subset index equal to NumEntryPoints) consists of the remaining bytes of the coded slice data. When sps_entropy_coding_sync_enabled_flag is equal to 0 and the slice contains one or more complete tiles, each subset shall consist of all coded bits of all coding tree units (CTUs) in the slice that are within the same tile, and the number of subsets (i.e., the value of NumEntryPoints+1) shall be equal to the number of tiles in the slice. When sps_entropy_coding_sync_enabled_flag is equal to 0 and the slice contains a subset of CTU rows from a single tile, the NumEntryPoints shall be 0, and the number of subsets shall be 1. The subset shall consist of all coded bits of all CTUs in the slice.

(351) When sps_entropy_coding_sync_enabled_flag is equal to 1, each subset k with k in the range of 0 to NumEntryPoints, inclusive, shall consist of all coded bits of all CTUs in a CTU row within a tile, and the number of subsets (i.e., the value of NumEntryPoints+1) shall be equal to the total number of tile-specific CTU rows in the slice.

(352) slice_header_extension_length specifies the length of the slice header extension data in bytes, not including the bits used for signalling slice_header_extension_length itself. The value of slice_header_extension_length shall be in the range of 0 to 256, inclusive. When not present, the value of slice_header_extension_length is inferred to be equal to 0.

(353) slice_header_extension_data_byte[i] may have any value. Decoders conforming to this version of this Specification shall ignore the values of all the slice_header_extension_data_byte[i] syntax elements. Its value does not affect decoder conformance to profiles specified in this version of specification.

(354) 3.5. Chroma QP mapping table

(355) In clause 7.3.2.3 of JVT-Q2001-vC, the SPS includes a structure named chroma QP table, shown as follows:

(356) TABLE-US-00016 Descriptor seq_parameter_set_rbsp() { ... if (ChromaArrayType != 0) { sps_joint_cbr_enabled_flag u(1) same_qp_table_for_chroma u(1) numQpTables = same_qp_table_for_chroma ? 1 : (sps_joint_cbr_enabled_flag ? 3 : 2) for(i = 0; i < numQpTables; i++) { qp_table_start_minus26[i] se(v) num_points_in_qp_table_minus1[i] ue(v) for(j = 0; j <= num_points_in_qp_table_minus1[i]; j++) { delta_qp_in_val_minus1[i][j] ue(v) delta_qp_diff_val[i][j] ue(v) } } } ... }

(357) They are with the following semantics and QP table derivation:

(358) sps_joint_cbr_enabled_flag equal to 0 specifies that the joint coding of chroma residuals is disabled. sps_joint_cbr_enabled_flag equal to 1 specifies that the joint coding of chroma residuals is enabled. When not present, the value of sps_joint_cbr_enabled_flag is inferred to be equal to 0.

(359) same_qp_table_for_chroma equal to 1 specifies that only one chroma QP mapping table is signalled and this table applies to Cb and Cr residuals and additionally to joint Cb-Cr residuals when sps_joint_cbr_enabled_flag is equal to 1. same_qp_table_for_chroma equal to 0 specifies that chroma QP mapping tables, two for Cb and Cr, and one additional for joint Cb-Cr when sps_joint_cbr_enabled_flag is equal to 1, are signalled in the SPS. When same_qp_table_for_chroma is not present in the bitstream, the value of same_qp_table_for_chroma is inferred to be equal to 1.

(360) qp_table_start_minus26[i] plus 26 specifies the starting luma and chroma QP used to describe the i-th chroma QP mapping table. The value of qp_table_start_minus26[i] shall be in the range of -26-QpBdOffset to 36 inclusive.

(361) When qp_table_start_minus26[i] is not present in the bitstream, the value of qp_table_start_minus26[i] is inferred to be equal to 0.

(362) num_points_in_qp_table_minus1[i] plus 1 specifies the number of points used to describe the i-th chroma QP mapping table. The value of num_points_in_qp_table_minus1[i] shall be in the range of 0 to 63+QpBdOffset, inclusive. When num_points_in_qp_table_minus1[0] is not present in the bitstream, the value of num_points_in_qp_table_minus1[0] is inferred to be equal to 0.

(363) delta_qp_in_val_minus1[i][j] specifies a delta value used to derive the input coordinate of the j-th pivot point of the i-th chroma QP mapping table. When delta_qp_in_val_minus1[0][j] is not present in the bitstream, the value of delta_qp_in_val_minus1[0][j] is inferred to be equal to 0.

(364) delta_qp_diff_val[i][j] specifies a delta value used to derive the output coordinate of the j-th pivot point of the i-th chroma QP mapping table.

(365) The i-th chroma QP mapping table ChromaQpTable[i] for i=0 . . . numQpTables-1 is derived as follows: qpIn Val[i][0]=qp_table_start_minus26[i]+26 qpOutVal[i][0]=qpIn Val[i][0] for(j=0; j<=num_points_in_qp_table_minus1[i]; j++) { qpIn Val[i][j+1]=qpInVal[i][j]+delta_qp_in_val_minus1[i][j]+1 qpOutVal[i][j+1]=qpOutVal[i][j]+ (delta_qp_in_val_minus1[i][j]+circumflex over ())delta_qp_diff_val[i][j])} ChromaQpTable[i][qpInVal[i][0]]=qpOutVal[i][0] for(k=qpIn Val[i][0]-1; k>=-QpBdOffset; k--) ChromaQpTable[i][k]=Clip3(-QpBdOffset, 63, ChromaQpTable[i][k+1]-1) for(j=0; j<=num_points_in_qp_table_minus1[i]; j++) { sh=(delta_qp_in_val_minus1[i][j]+1)>>1 for(k=qpInVal[i][j]+1, m=1; k<=qpInVal[i][j+1]; k++, m++) ChromaQpTable[i][k]=ChromaQpTable[i][qpIn Val[i][j]]+ ((qpOutVal[i][j+1]-qpOutVal[i][j])*m+sh)/(delta_qp_in_val_minus1[i][j+1]+1) } for(k=qpIn Val[i][num_points_in_qp_table_minus1[i]+1]+1; k<=63; k++) ChromaQpTable[i][k]=Clip3(QpBdOffset, 63, ChromaQpTable[i][k-1]+1)

(366) When same_qp_table_for_chroma is equal to 1, ChromaQpTable[1][k] and ChromaQpTable[2][k] are set equal to ChromaQpTable[0][k] for k in the range of -QpBdOffset to 63, inclusive.

(367) It is a requirement of bitstream conformance that the values of qpIn Val[i][j] and qpOutVal[i][j] shall be in the range of -QpBdOffset to 63, inclusive for i in the range of 0 to numQpTables-1, inclusive, and j in the range of 0 to num_points_in_qp_table_minus1[i]+1, inclusive.

(368) In the above description, QpBdOffset is derived as:

(369) bit_depth_minus8 specifies the bit depth of the samples of the luma and chroma arrays, BitDepth, and the value of the luma and chroma quantization parameter range offset, QpBdOffset, as follows: BitDepth=8+bit_depth_minus8 QpBdOffset=6*bit_depth_minus8 bit_depth_minus8 shall be in the range of 0 to 8, inclusive.

4. Technical Problems Solved by Disclosed Technical Solutions

(370) The existing designs in the latest VVC draft specification for APS, deblocking, subpicture, and QP delta have the following problems: 1) Currently, the value of the APS syntax element scaling_list_chroma_present_flag is constrained based on ChromaArrayType derived from SPS syntax elements chroma_format_idc and separate_colour_plane_flag, phrased as follows: scaling_list_chroma_present_flag shall be equal to 0 when ChromaArrayType is equal to 0, and shall be equal to 1 when ChromaArrayType is not equal to 0. Such constraints in the semantics of the APS syntax element introduce semantics dependencies of APS on SPS, which should not occur, because since there is no PPS ID or SPS ID in the APS syntax, an APS may be applied to pictures (or slices of pictures) that refer to different SPSs, which may be associated with different values of ChromaArrayType. a. Additionally, similar APS-SPS semantics dependencies also exist in the semantics of some ALF/cross component ALF (CC-

ALF) APS syntax elements, phased as follows: `alf_chroma_filter_signal_flag`, `alf_cc_cb_cr_filter_signal_flag` shall to be equal to 0 when `ChromaArrayType` is equal to 0. b. Currently, when an LMCS APS is signalled, the chroma residual scaling related syntax elements are always signalled in the LMCS APS syntax structure, regardless of whether `ChromaArrayType` is equal to 0 (i.e., there is no chroma component in the CLVS). This results in unnecessary signalling of chroma-related syntax elements. 2) It is asserted that the deblocking control mechanism in the latest VVC text is pretty complicated, not straightforward, and not easy to understand, and consequently prone to errors. Here are some example issues we observed: a. According to the current text, even if the deblocking filter is disabled in the PPS, it can be enabled in the PH or SH. For example, if `pps_deblocking_filter_disabled_flag` is firstly signalled to be equal to 1, and `deblocking_filter_override_enabled_flag` is also signalled to be equal to 1, it indicates that the deblocking filter is disabled at PPS and it also allows the deblocking filter enable/disable control being overridden in the PH or SH. Then `dbf_info_in_ph_flag` is signalled subsequently and the PH syntax element `ph_deblocking_filter_disabled_flag` might be signalled to be equal to 0 which ultimately enables the deblocking filter for slices associated with the PH. In such case, the deblocking is eventually enabled at PH regardless that it has been disabled at the higher level (e.g., PPS). Such design logic is unique in VVC text, and it is quite different from the design logic of other coding tools (e.g., ALF, SAO, LMCS, temporal motion vector prediction (TMVP), weighted prediction (WP), and etc.) as normally when a coding tool is disabled at a higher layer (e.g., SPS, PPS), then it is disabled completely at lower layers (e.g., PH, SH). b. Furthermore, the current definition of `pps_deblocking_filter_disabled_flag` is like “`pps_deblocking_filter_disabled_flag` equal to 1 specifies that the operation of deblocking filter is not applied for slices referring to the PPS in which `slice_deblocking_filter_disabled_flag` is not present . . . “. However, according to the current syntax table, even though `pps_deblocking_filter_disabled_flag` is equal to 1 and `slice_deblocking_filter_disabled_flag` is not present, the operation of deblocking filter would still be applied in case of `ph_deblocking_filter_disabled_flag` is present and signalled to be equal to 0. Therefore, the current definition of `pps_deblocking_filter_disabled_flag` is not correct. c. Moreover, according to the current text, if both the PPS syntax elements `deblocking_filter_override_enabled_flag` and `pps_deblocking_filter_disabled_flag` are equal to 1, it specifies that deblocking is disabled in PPS and the control of deblocking filter is intended to be overridden in the PH or SH. However, the subsequent PH syntax elements `ph_deblocking_filter_override_flag` and `ph_deblocking_filter_disabled_flag` might be still signalled to be equal to 1, which turns out that the resulted overriding process doesn't change anything (e.g., deblocking remains disabled in the PH/SH) but just use unnecessary bits for meaningless signalling. d. In addition, according to the current text, when the SH syntax element `slice_deblocking_filter_override_flag` is not present, it is inferred to be equal to `ph_deblocking_filter_override_flag`. However, besides implicit or explicit signalling in the PPS, the deblocking parameters can only be signalled either in PH or SH according to `dbf_info_in_ph_flag`, but never both. Therefore, when `dbf_info_in_ph_flag` is true, the intention is to allow to signal the overriding deblocking filter parameters in the PH. In this case, if the PH override flag is true and the SH override flag is not signalled but inferred to be equal to the PH override flag, additional deblocking filter parameters will still be signalled in the SH which is conflicting with the intention. e. In addition, there is no SPS-level deblocking on/off control, which may be added and the related syntax elements in PPS/PH/SH may be updated accordingly. 3) Currently, when the PPS syntax element `single_slice_per_subpic_flag` is not present, it is inferred to be equal to 0. `single_slice_per_subpic_flag` is not present in two cases: i) `no_pic_partition_flag` is equal to 1, and ii) `no_pic_partition_flag` is equal to 0 and `rect_slice_flag` is equal to 0. For case i), `no_pic_partition_flag` equal to 1 specifies that no picture partitioning is applied to each picture referring to the PPS, therefore, there is only one slice in each picture, and consequently, there is only one subpicture in each picture and there is only one slice in each subpicture. Therefore, in this case `single_slice_per_subpic_flag` should be inferred to be equal 1. For case ii), since `rect_slice_flag` is equal to 0, an inferred value of `single_slice_per_subpic_flag` is not needed. 4) Currently, luma qp delta in either picture or slice level is always signalled mandatorily, either in the PH or SH, never both. Whereas the slice-level chroma QP offset is optionally signalled in the SH. Such design is somewhat not consistent. a. Additionally, the current semantics of the PPS syntax element `cu_qp_delta_enabled_flag` is worded as follows: `cu_qp_delta_enabled_flag` equal to 1 specifies that the `ph_cu_qp_delta_subdiv_intra_slice` and `ph_cu_qp_delta_subdiv_inter_slice` syntax elements are present in PHs referring to the PPS and `cu_qp_delta_abs` may be present in the transform unit syntax . . . However, `cu_qp_delta_abs` may be also present in the palette coding syntax, which should also be specified by `cu_qp_delta_enabled_flag`. In other words, the current semantics of `cu_qp_delta_enabled_flag` is not clear enough and a bit confusing. 5) The current design of chroma Qp mapping table is not straightforward to represent the case of chroma Qp equal to luma Qp. 6) Currently, the `subpic_treated_as_pic_flag[i]` is inferred to be equal to the value of `sps_independent_subpics_flag`. However, the current spec only allows the horizontal wrap-around to be enabled when `subpic_treated_as_pic_flag[i]` is equal to 0, wherein the wrap-around motion compensation is designed for 360 video content. Therefore, when a pictures contains only one subpicture (especially for the case that a complete 360 video sequence contains only one subpicture), the inferred value for `subpic_treated_as_pic_flag[i]` may be inferred to be equal to 0 or a certain value that allows the wrap-around motion compensation. 7) Currently, the semantics of `pps_deblocking_filter_disabled_flag` is not correct and incomplete. e.g., when `pps_deblocking_filter_disabled_flag` is equal to 1, deblocking may be enabled or disabled for slices referring to this PPS, but those conditions are not mentioned in the semantics. Similarly for the part of semantics for `pps_deblocking_filter_disabled_flag` is equal to 0. a. Furthermore, when the PPS syntax element `pps_deblocking_filter_disabled_flag` are equal to 1, and meanwhile the PH/SH syntax element `ph/slice_deblocking_filter_override_flag` is signalled to be equal to 1, the `ph/slice_deblocking_filter_disabled_flag` is still allowed to be explicitly signalled to be equal to 1. What this combination of values of the flags does is that deblocking is said at the PPS level to be disabled and allowed to be overridden at the picture or slice, then it is indicated at the PH/SH level that it is going to be overridden, and then a bit is signalled in the same header (PH/SH) to finally determine that it is actually not overridden and deblocking remains disabled at the picture/slice level. This is asserted to have a double undesirable effects: not only a bit is spent unnecessarily, it is spent just for causing some confusion. We therefore propose to further improve the semantics of deblocking control syntax elements and remove the feature of allowing indicating overriding and then immediately send the next bit in the same PH or SH to indicate a change of mind.

5. A Listing of Solutions and Embodiments

(371) To solve the above problems and some other problems not mentioned, methods as summarized below are disclosed. The items listed below should be considered as examples to explain the general concepts and should not be interpreted in a narrow way. Furthermore, these items can be applied individually or combined in any manner.

(372) In the following discussion, an SH may be associated with a PH, i.e., the SH is associated with a slice, which is in the picture associated with the PH. An SH may be associated with a PPS, i.e., the SH is associated with a slice, which is in the picture associated with the PPS. A PH may be associated with a PPS, i.e., the PH is associated with a picture, which is associated with the PPS.

(373) In the following discussion, a SPS may be associated with a PPS, i.e., the PPS may refer to the SPS.

(374) In the following discussion, the changed texts are based on the latest VVC text in JVET-Q2001-vE. Most relevant parts that have been added or modified are highlighted in boldface italics, and some of the deleted parts are marked with double brackets (e.g., [a] denotes the deletion of the character “a”). 1. Regarding the constraints on APS syntax elements for solving the first problem, one or more of the following approaches are disclosed: a. In one example, constrain the value of `scaling_list_chroma_present_flag` according to `ChromaArrayType` derived by the PH syntax element. i. For example, whether the value of `scaling_list_chroma_present_flag` is constrained or not may be dependent on whether `ph_scaling_list_aps_id` is present or not, e.g., as in the first set of embodiments. 1) In one example, it is required that, when `ph_scaling_list_aps_id` is present, the value of `scaling_list_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be equal to `ChromaArrayType==0? 0:1`. ii. Alternatively, `scaling_list_chroma_present_flag` is constrained based on `ChromaArrayType` derived by the PH syntax elements, but regardless of the presence of `ph_scaling_list_aps_id`, e.g., as in the first set of embodiments. 1) In one example, it is required that, the value of `scaling_list_chroma_present_flag`

of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` shall be equal to `ChromaArrayType==0?0:1`. b. In one example, constrain the value of `lmcs_delta_abs_crs` according to `ChromaArrayType` derived by the PH syntax element. i. For example, whether the value of `lmcs_delta_abs_crs` is constrained or not may be dependent on whether `ph_lmcs_aps_id` is present or not, e.g., as in the first set of embodiments. 1) For example, it is required that, when `ph_lmcs_aps_id` is present, the value of `lmcs_delta_abs_crs` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be greater than 0 otherwise. 2) Alternatively, it is required that, when `ph_lmcs_aps_id` is present, the value of `lmcs_delta_abs_crs` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0. ii. Alternatively, `lmcs_delta_abs_crs` is constrained based on `ChromaArrayType` derived by the PH syntax elements, but regardless of the presence of `ph_lmcs_aps_id`, e.g., as in the first set of embodiments. 1) For example, it is required that, the value of `lmcs_delta_abs_crs` of the APS NAL unit equal to `ph_lmcs_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be greater than 0 otherwise. 2) For example, it is required that, the value of `lmcs_delta_abs_crs` of the APS NAL unit equal to `ph_lmcs_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0. c. In one example, constrain the value of ALF APS syntax elements (e.g., `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, `alf_cc_cr_filter_signal_flag`, and etc.) according to `ChromaArrayType` derived by the PH syntax elements and/or the SH syntax elements. i. For example, whether the value of `alf_chroma_filter_signal_flag` and/or `alf_cc_cb_filter_signal_flag` and/or `alf_cc_cr_filter_signal_flag` are constrained or not may be dependent on whether `ph_alf_aps_id_luma[i]` or `slice_alf_aps_id_luma[i]` is present or not and/or whether `ChromaArrayType` is equal to 0 or not, e.g., as in the first set of embodiments. 1) For example, it is required that, when `ph_alf_aps_id_luma[i]` is present and `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall all be equal to 0. 2) Additionally, it is required that, when `slice_alf_aps_id_luma[i]` is present and `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall all be equal to 0. ii. Alternatively, `alf_chroma_filter_signal_flag` and/or `alf_cc_cb_filter_signal_flag` and/or `alf_cc_cr_filter_signal_flag` are constrained based on `ChromaArrayType` derived by the PH syntax elements or SH syntax elements, but regardless of the presence of `ph_alf_aps_id_luma[i]` and/or `slice_alf_aps_id_luma[i]`, e.g., as in the first set of embodiments. 1) For example, it is required that, when `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` shall all be equal to 0. 2) Additionally, it is required that, when `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` shall all be equal to 0. iii. Alternatively, `alf_chroma_filter_signal_flag` and/or `alf_cc_cb_filter_signal_flag` and/or `alf_cc_cr_filter_signal_flag` are constrained based on `ChromaArrayType` derived by the chroma APS ID related PH or SH syntax elements, e.g., as in the first set of embodiments. 1) For example, `alf_chroma_filter_signal_flag` is constrained according to `ChromaArrayType` derived by the PH syntax element `ph_alf_aps_id_chroma` and/or the SH syntax element `slice_alf_aps_id_chroma`. 2) For example, `alf_cc_cb_filter_signal_flag` is constrained according to `ChromaArrayType` derived by the PH syntax element `ph_cc_alf_cb_aps_id` and/or the SH syntax element `slice_cc_alf_cb_aps_id`. 3) For example, `alf_cc_cr_filter_signal_flag` is constrained according to `ChromaArrayType` derived by the PH syntax element `ph_cr_alf_cb_aps_id` and/or the SH syntax element `slice_cr_alf_cb_aps_id`. d. In one example, the semantics of APS syntax elements in the ALF and/or SCALING LIST and/or LMCS data syntax structure may be not dependent on whether it is 4:0:0 video coding and/or separate color plane coding. i. For example, the semantics of APS syntax elements in the ALF data syntax structure (e.g., `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, `alf_cc_cr_filter_signal_flag`, and etc.) may be not dependent on variables/syntaxes derived by SPS/PH/SH syntax elements (e.g., `ChromaArrayType`), e.g., as in the first set of embodiments. ii. Additionally, alternatively, the semantics of APS syntax elements in the SCALING LIST data syntax structure (e.g., `scaling_list_chroma_present_flag`, and etc.) may be not dependent on variables/syntaxes derived by SPS/PH/SH syntax elements (e.g., `ChromaArrayType`), e.g., as in the first set of embodiments. e. Additionally, whether the temporalId of the ALF/SCALING/LMCS APS NAL unit is constrained or not may be dependent on whether the corresponding APS ID is present or not, e.g., as in the first set of embodiments. i. For example, whether the temporalId of the ALF APS NAL unit is constrained or not may be dependent on whether `ph_alf_aps_id_luma[i]` and/or `ph_alf_aps_id_chroma` and/or `ph_cc_alf_cb_aps_id` and/or `ph_cc_alf_cr_aps_id` is present or not. ii. For example, whether the temporalId of the LMCS APS NAL unit is constrained or not may be dependent on whether `ph_lmcs_aps_id` is present or not. iii. For example, whether the temporalId of the SCALING APS NAL unit is constrained or not may be dependent on whether `ph_scaling_list_aps_id` is present or not. f. Additionally, whether the values of `alf_luma_filter_signal_flag`, `alf_chroma_filter_signal_flag` and/or `alf_cc_cb_filter_signal_flag` and/or `alf_cc_cr_filter_signal_flag` shall be equal to 1 may be dependent on whether the corresponding APS ID is present or not, e.g., as in the first set of embodiments. i. For example, whether `alf_luma_filter_signal_flag` shall be equal to 1 or not may be dependent on whether `ph_alf_aps_id_luma[i]` and/or `slice_alf_aps_id_luma[i]` is present or not. ii. For example, whether `alf_chroma_filter_signal_flag` shall be equal to 1 or not may be dependent on whether `ph_alf_aps_id_chroma` and/or `slice_alf_aps_id_chroma` is present or not. iii. For example, whether `alf_cc_cb_filter_signal_flag` shall be equal to 1 or not may be dependent on whether `ph_cc_alf_cb_aps_id` and/or `slice_cc_alf_cb_aps_id` is present or not. iv. For example, whether `alf_cc_cr_filter_signal_flag` shall be equal to 1 or not may be dependent on whether `ph_cc_alf_cr_aps_id` and/or `slice_cc_alf_cr_aps_id` is present or not. g. Additionally, alternatively, whether the chroma ALF APS ID syntax elements in the SH (e.g., `slice_alf_aps_id_chroma`, `slice_cc_alf_cb_aps_id`, `slice_cr_alf_cb_aps_id`, and etc.) is inferred or not may be dependent on the value of `ChromaArrayType`, e.g., as in the first set of embodiments. i. For example, when `ChromaArrayType` is not equal to 0, the value of the chroma ALF APS ID syntax elements in the SH (e.g., `slice_alf_aps_id_chroma`, `slice_cc_alf_cb_aps_id`, `slice_cr_alf_cb_aps_id`, and etc.) may be inferred. h. In one example, the constraint of APS syntax elements based on `ChromaArrayType` may be derived by the PH or SH syntax elements. i. In one example, the APS in which syntax elements are constrained is determined by an index (such as `ph_num_alf_aps_ids_luma`, `ph_alf_aps_id_chroma`, `ph_lmcs_aps_id` and `ph_scaling_list_aps_id`) signaled in PH or SH. ii. In one example, `ChromaArrayType` may be derived by information (such as `chroma_format_idc` and `separate_colour_plane_flag`) signaled in a SPS, which is determined by an index (such as `pps_seq_parameter_set_id`) signaled in a PPS, which is further determined by an index (such as `ph_pic_parameter_set_id`) signaled in the PH or SH. iii. In one example, the constraints should be checked after parsing the APS and the PH or SH. iv. In one example, a syntax element (e.g., named `aps_chroma_present_flag`) may be signalled in the APS syntax structure (e.g., `adaptation_parameter_set_rbsp` () specifying whether the chroma-related APS syntax elements would be signalled or not. v. Additionally, the syntax element `aps_chroma_present_flag` may be constrained by `ChromaArrayType`. a) For example, `aps_chroma_present_flag` shall be equal to 0 if `ChromaArrayType` is equal to 0. b) For example, `aps_chroma_present_flag` shall be equal to 1 if `ChromaArrayType` is larger than 0. vi. Additionally, the constraint based on `ChromaArrayType` may be derived by the PH or SH syntax elements. a) In one example, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph/slice_alf_aps_id_luma[i]` shall be equal to 0 when `chromaArrayType` is equal to 0. a. Additionally, alternatively, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph/slice_alf_aps_id_luma[i]` (and/or `ph_cc_alf_cb_aps_id` and/or `ph_cc_alf_cr_aps_id`) shall be equal to 1 when `chromaArrayType` is greater than 0. b) In one example, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be equal to 0 when `chromaArrayType` is equal to 0. a. Additionally, alternatively, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be equal to 1 when `chromaArrayType` is greater than 0. c) In one example, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and

adaptation_parameter_set_id equal to ph_lmcs_list_apc_id shall be equal to 0 when chromaArrayType is equal to 0. vii. Additionally, the constraint based on ChromaArrayType may be derived by the PH syntax elements or SH syntax elements, but regardless of the presence of the APS ID in the PH/SH. a) In one example, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to SCALING_APS and/or ALF_APS and/or LMCS_APS shall be equal to 0 when chromaArrayType is equal to 0. b) Additionally, alternatively, it is required that, the value of `aps_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to SCALING_APS and/or ALF_APS and/or LMCS_APS shall be equal to 1 when chromaArrayType is greater than 0. 2. Regarding the signalling of deblocking control for solving the second problem, one or more of the following approaches are disclosed, e.g., as in the second set of embodiments: a. In one example, an N-bit (such as N=2) deblocking mode indicator (e.g., named `deblocking_filter_mode_idc`) is signalled. i. In one example, the syntax element `deblocking_filter_mode_idc` is u(2) coded. a) Alternatively, the parsing process of `deblocking_filter_mode_idc` is unsigned integer with N (such as N=2) bits. ii. In one example, the syntax element `deblocking_filter_mode_idc` is signalled in the PPS. iii. In one example, the syntax element `deblocking_filter_mode_idc` is used to specify the following four modes: a) deblocking fully disabled and not used for all slices; b) deblocking used for all slices using 0-valued β and tC offsets; c) deblocking used for all slices using β and tC offsets explicitly signalled in the PPS; and d) deblocking further controlled at either picture or slice level. b. A syntax flag `ph/slice_deblocking_filter_used_flag` is signalled either in the PH or SH, specifying whether deblocking is used for the current picture/slice. c. A syntax flag `ph/slice_deblocking_parameters_override_flag` is signalled either in the PH or SH, specifying whether the β and tC offsets are overridden by the values signalled in the PH/SH. i. Additionally, infer the value of `slice_deblocking_parameters_override_flag` to be equal to 0 when not present. d. In one example, the syntax elements specifying the deblocking control (e.g., enable flag, disable flag, control flag, deblocking mode indicator, deblocking filter beta/tc parameters, and etc.) may be signalled in the SPS. i. In one example, one or more syntax elements may be signalled in the SPS specifying whether the deblocking is enabled or not in the video unit (e.g., CLVS). ii. Additionally, when the deblocking is disabled in the SPS, it is required that the syntax element in the PPS/PH/SH regarding the deblocking on/off control at PPS/PH/SH level shall be equal to a certain value that specifying the deblocking is fully disabled and not used for all slices. iii. In one example, the deblocking filter control present flag may be signalled in the SPS. iv. For example, an N-bit (such as N=2) deblocking mode indicator (e.g., named `deblocking_filter_mode_idc`) may be signalled in the SPS. v. For example, beta/tc deblocking parameters may be signalled in the SPS. vi. For example, whether the deblocking is enabled with 0-valued beta/tc deblocking parameters may be dependent on the SPS syntax element. vii. For example, the deblocking may be applied at the SPS/PPS/PH/SH level and use the beta/tc deblocking parameters signalled in the SPS. viii. For example, the deblocking may be applied at the SPS/PPS/PH/SH level and use the 0-valued deblocking parameters signalled in the SPS. 3. Regarding the inference of the PPS syntax element `single_slice_per_subpic_flag` for solving the third problem, one or more of the following approaches are disclosed: a. In one example, infer `single_slice_per_subpic_flag` to be equal to 1 when `no_pic_partition_flag` is equal to 1, e.g., the semantics of `single_slice_per_subpic_flag` is changed as follows: `single_slice_per_subpic_flag` equal to 1 specifies that each subpicture consists of one and only one rectangular slice. `single_slice_per_subpic_flag` equal to 0 specifies that each subpicture may consist of one or more rectangular slices. When `no_pic_partition_flag` is equal to 1, the value of `single_slice_per_subpic_flag` is inferred to be equal to 1. 4. Regarding the picture or slice QP delta signalling for solving the fourth problem, one or more of the following approaches are disclosed: a. In one example, picture or slice level chroma QP offset are always signalled, either in the PH or SH. i. For example, if there is chroma component in the video content (e.g., ChromaArrayType is not equal to 0), picture or slice level chroma QP offset may be always signalled, without conditioned on the present flag signalled in the PPS (e.g., `pps_slice_chroma_qp_offsets_present_flag`). ii. Alternatively, if there is chroma component in the video content (e.g., ChromaArrayType is not equal to 0), `slice_cb_qp_offset` and `slice_cr_qp_offset` syntax elements may be always present in the associated slice headers, regardless the PPS present flag (e.g., `pps_slice_chroma_qp_offsets_present_flag`). iii. Additionally, the present flag (e.g., `pps_slice_chroma_qp_offsets_present_flag`) specifying the presence of `slice_cb_qp_offset` and `slice_cr_qp_offset` syntax elements, may be not signalled. b. In one example, `pps_cu_qp_delta_enabled_flag` may be used to specify the presence of `cu_qp_delta_abs` and `cu_qp_delta_sign_flag` in both the transform unit syntax and the palette coding syntax, and the semantics of `pps_cu_qp_delta_enabled_flag` are changed as follows: `pps_cu_qp_delta_enabled_flag` equal to 1 specifies that the `ph_cu_qp_delta_subdiv_intra_slice` and `ph_cu_qp_delta_subdiv_inter_slice` syntax elements are present in PHs referring to the PPS, and the `cu_qp_delta_abs` and `cu_qp_delta_sign_flag` syntax elements may be present in the transform unit syntax and the palette coding syntax. `pps_cu_qp_delta_enabled_flag` equal to 0 specifies that the `ph_cu_qp_delta_subdiv_intra_slice` and `ph_cu_qp_delta_subdiv_inter_slice` syntax elements are not present in PHs referring to the PPS, and the `cu_qp_delta_abs` and `cu_qp_delta_sign_flag` syntax elements are not present in the transform unit syntax or the palette coding syntax. c. In one example, the luma QP delta may be signalled in both the PH and the SH. i. For example, luma QP delta present flag may be signalled in the PPS and/or PH, and/or the SH. ii. For example, whether the luma QP delta is signalled in PH/SH is dependent on the present flags in the PPS and/or PH/SH. iii. For example, the values of the PH luma QP delta and the SH luma QP delta may be additive and used for calculating the luma quantization parameter such as `SliceQp.sub.Y`. d. In one example, the chroma QP offsets may be signalled in both the PH and the SH. i. For example, chroma QP offsets present flag may be signalled in the PPS and/or PH, and/or the SH. ii. For example, whether the chroma QP offsets are signalled in PH/SH is dependent on the present flags in the PPS and/or PH/SH. iii. For example, the values of the PH chroma QP offsets and the SH chroma QP offsets may be additive and used for deriving the chroma quantization parameter for the Cb and Cr components. 5. Regarding the chroma Qp mapping tables, one or more of the following approaches are disclosed: a. In one example, in the derivation process of the chroma QP table, the exclusive or (XOR) operator should be performed between $(\text{delta_qp_in_val_minus1}[i][j]+1)$ and $\text{delta_qp_diff_val}[i][j]$, e.g. as in the third set of embodiment. b. It is proposed to have a flag in `sps_multiple_sets_of_chroma_qp_table_present_flag` in SPS. i. When `sps_multiple_sets_of_chroma_qp_table_present_flag` is equal to 0, only one set of chroma Qp mapping table is allowed to be signalled. ii. When `sps_multiple_sets_of_chroma_qp_table_present_flag` is equal to 1, more than one set of chroma Qp mapping table are allowed to be signalled. c. More than one set of chroma Qp mapping tables may be disallowed to be signalled for a sequence without B/P slices. 6. Regarding the `sps_independent_subpics_flag` and `subpic_treated_as_pic_flag[i]` for solving the sixth problem, one or more of the following approaches are disclosed: a. In one example, the presence of `sps_independent_subpics_flag` is dependent on whether the number of subpictures is greater than 1. i. For example, only when the number of subpictures is greater than 1 (e.g., if $(\text{sps_num_subpics_minus1} > 0)$), `sps_independent_subpics_flag` is signalled. ii. For example, when the number of subpictures is equal to 1 (e.g., if $(\text{sps_num_subpics_minus1} = 0)$), then the signalling of `sps_independent_subpics_flag` is skipped. b. Additionally, when `sps_independent_subpics_flag` is not present, it is inferred to be equal to a certain value (such as 0 or 1). c. In one example, when `subpic_treated_as_pic_flag[i]` is not present, it is inferred to be equal to a certain value (such as 0 or 1). d. In one example, when `subpic_treated_as_pic_flag[i]` is not present, it is inferred to be equal to a certain value wherein the wrap-around motion compensation is enabled (or may be used). i. Additionally, when `subpic_treated_as_pic_flag[i]` is not present, it is inferred to be equal to a certain value wherein the horizontal wrap-around motion compensation is enabled (or may be used). e. In one example, the inferred value of `subpic_treated_as_pic_flag[i]` may be dependent on whether a picture only consists of one subpicture; and/or whether the subpicture has the same width as the picture. i. In one example, if the subpicture has the same width as the picture, `subpic_treated_as_pic_flag[i]` may be inferred to X (e.g., X=0). f. In one example, when `sps_independent_subpics_flag` is not present, what value that `sps_independent_subpics_flag` is inferred to may be dependent on other syntax element(s) or variable(s). i. For example, the inferred value may depend on whether the subpicture information is present or not (e.g., `subpic_info_present_flag` is equal to 0 or 1). ii. For example, when `subpic_info_present_flag` is equal to 0 and `sps_independent_subpics_flag` is not present, it is inferred to be equal to a certain value (such as 0 or 1). iii. For example, when `subpic_info_present_flag` is equal to 1 and `sps_independent_subpics_flag` is not present, it is inferred to be equal to a certain value (such as 0 or 1). g. In one example, when `subpic_treated_as_pic_flag[i]` is not present, what value that `subpic_treated_as_pic_flag[i]` is inferred to may be dependent on the presence of subpicture information (e.g., `subpic_info_present_flag`) and/or the number of subpictures in the CLVS (e.g., $\text{sps_num_subpics_minus1}$) and/or `sps_independent_subpics_flag`. i. In one example, when `subpic_info_present_flag` is equal to 0, and

subpic_treated_as_pic_flag[i] is not present, the value of subpic_treated_as_pic_flag[i] is inferred to be equal to a certain value (such as 0). ii. In one example, when subpic_info_present_flag is equal to 1, and subpic_treated_as_pic_flag[i] is not present, the value of subpic_treated_as_pic_flag[i] is inferred to be equal to a certain value (such as 1). iii. In one example, when subpic_info_present_flag is equal to 1, and sps_num_subpics_minus1 is equal to 0, and subpic_treated_as_pic_flag[i] is not present, the value of subpic_treated_as_pic_flag[i] is inferred to be equal to a certain value (such as 0 or 1). iv. In one example, when subpic_info_present_flag is equal to 1, sps_num_subpics_minus1 is greater than 0, sps_independent_subpics_flag is equal to 1, and subpic_treated_as_pic_flag[i] is not present, the value of subpic_treated_as_pic_flag[i] is inferred to be equal to a certain value (such as 0 or 1). 7. How to do padding or clipping on a boundary during the inter-prediction process may depend on a combined checking of the type of boundary, the indication of wrap around padding or clipping (e.g. pps_ref_wraparound_enabled_flag, sps_ref_wraparound_enabled_flag, and etc.) and the indication of treating subpicture boundary as picture boundary (e.g. subpic_treated_as_pic_flag[i]). a. For example, if a boundary is a picture boundary, the indication of wrap around padding is true, wrap around padding (or wrap around clipping) may be applied, without considering the indication of treating subpicture boundary as picture boundary. i. In one example, the boundary must be a vertical boundary. b. For example, if both two vertical boundaries are picture boundaries, the indication of wrap around padding is true, wrap around padding (or wrap around clipping) may be applied, without considering the indication of treating subpicture boundary as picture boundary. c. In one example, the above wrap around padding (or wrap around clipping) may indicate horizontal wrap-around padding/clipping. 8. In one example, different indications for wrap around padding or clipping may be signaled for different subpictures. 9. In one example, different offsets for wrap around padding or clipping may be signaled for different subpictures. 10. In the PH/SH, a variable X is used to indicate whether B slice is allowed/used in a picture/slice, and the variable may be derived using one of the following ways: a) (rpl_info_in_ph_flag && num_ref_entries[0][RplsIdx[0]]>0 && num_ref_entries[1][RplsIdx[1]]>0); b) (rpl_info_in_ph_flag && num_ref_entries[1][RplsIdx[1]]>0); c) (rpl_info_in_ph_flag && num_ref_entries[1][RplsIdx[1]]>1); d) (rpl_info_in_ph_flag && num_ref_entries[1][RplsIdx[1]]>0); e) based on NumRefIdxActive (e.g., NumRefIdxActive for list 1 greater than K (e.g., K=0)) in the VVC text; f) based on number of allowed reference pictures for list 1. 1) Alternatively, furthermore, signalling and/or semantics and/or inference of one or multiple syntax elements signalled in PH may be modified according to the variable. i. In one example, the one or multiple syntax elements are those for enabling a coding tool which requires more than one prediction signal, such as bi-prediction or mixed intra and inter coding, or prediction with linear/non-linear weighting from multiple prediction blocks. ii. In one example, the one or multiple syntax elements may include, but not limited to: a) ph_collocated_from_10_flag b) mvd_11_zero_flag c) ph_disable_bdof_flag d) ph_disable_dmvf_flag e) num_11_weights iii. In one example, only when the variable indicates that the picture may contain one or more B slices, the one or multiple syntax elements may be signalled. Otherwise, the signalling is skipped, and the values of the syntax element are inferred. a) Alternatively, furthermore, whether to signal the one or more syntax elements may depend on the first syntax elements in bullet 1.1) and 2.1), such as (X being true or 1). b) ph_disable_bdof_flag may be signalled only when (sps_bdof_pic_present_flag && X) is true. c) ph_disable_dmvf_flag may be signalled only when (sps_dmvf_pic_present_flag && X) is true. iv. In one example, when X is equal to 0 (or false), mvd_11_zero_flag is not signalled, and its value is inferred to be 1. v. In one example, the inference of the one or multiple syntax elements are dependent on the value of the first syntax element. a) In one example, for the ph_disable_bdof_flag, the following applies: If sps_bdof_enabled_flag is equal to 1 and X is equal to 1 (or true), the value of ph_disable_bdof_flag is inferred to be equal to 0. Otherwise (sps_bdof_enabled_flag is equal to 0 or X is equal to 0 (or false)), the value of ph_disable_bdof_flag is inferred to be equal to 1. b) In one example, for the ph_disable_dmvf_flag, the following applies: If sps_dmvf_enabled_flag is equal to 1 and X is equal to 1 (or true), the value of ph_disable_dmvf_flag is inferred to be equal to 0. Otherwise (sps_dmvf_enabled_flag is equal to 0 or X is equal to 0 (or false)), the value of ph_disable_dmvf_flag is inferred to be equal to 1. c) In one example, when ph_temporal_mv_enabled_flag and rpl_info_in_ph_flag are both equal to 1 and X is equal to 0 (or false), the value of ph_collocated_from_10_flag is inferred to be equal to 1. d) In one example, when X is equal to 0 (or false), num_11_weights is not signalled and its value is inferred to be 0, and, consequently, weighted prediction parameters for reference picture list 1 are not signalled in the PH or SHs of the picture. 11. It is proposed that the signaling of indication of whether to partition a picture into tiles/slices/subpictures (such as no_pic_partition_flag in PPS) maybe conditioned on the number of CTBs in a picture. 1) In one example, no_pic_partition_flag is not signaled if the number of CTUs in a picture is equal to 1 (or smaller than 2). 2) Alternatively, it is constrained that no_pic_partition_flag shall be equal to 0 if the number of CTUs in a picture is equal to 1 (or smaller than 2). 12. Regarding improving the syntax and semantics of deblocking for solving the seventh problem, one or more of the following approaches are disclosed, e.g., as in the fourth embodiment: a. Whether the operation of deblocking filter is disabled (or enabled) for slices referring to the PPS is dependent on both the deblocking syntax signalled in the PPS (e.g., pps_deblocking_filter_disabled_flag) and the deblocking syntax elements signalled at the picture or slice level. i. In one example, pps_deblocking_filter_disabled_flag equal to 1 specifies that the operation of deblocking filter is disabled for slices referring to the PPS unless indicated otherwise at the picture or slice level. ii. In one example, pps_deblocking_filter_disabled_flag equal to 0 specifies that the operation of deblocking filter is enabled for slices referring to the PPS unless indicated otherwise at the picture or slice level. b. Infer slice_deblocking_filter_override_flag to be equal to 0 when not present. c. Skip the signalling of deblocking on/off control flag in the PH/SH when deblocking filter is disabled in the PPS and going to be overridden in the PH/SH. i. In one example, whether to signal deblocking on/off control flag in the PH (e.g., ph_deblocking_filter_disabled_flag) may be dependent on whether the deblocking is disabled in the PPS (e.g., whether the value of pps_deblocking_filter_disabled_flag is equal to 1 or not) and/or the override flag in the PH (e.g., ph_deblocking_filter_override_flag is equal to 1 or not). a) For example, when pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are equal to 1, the signalling of ph_deblocking_filter_disabled_flag may be skipped. b) Alternatively, when deblocking_filter_override_enabled_flag and pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are equal to 1, the signalling of ph_deblocking_filter_disabled_flag may be skipped. ii. Additionally, when ph_deblocking_filter_disabled_flag is not present, it may be inferred as follows: a) If deblocking_filter_override_enabled_flag, pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are all equal to 1, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to 0. b) Otherwise, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to pps_deblocking_filter_disabled_flag. iii. Additionally, alternatively, when ph_deblocking_filter_disabled_flag is not present, it may be inferred as follows: a) If pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are all equal to 1, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to 0. b) Otherwise, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to pps_deblocking_filter_disabled_flag. iv. In one example, whether to signal deblocking on/off control flag in the SH (e.g., slice_deblocking_filter_disabled_flag) may be dependent on whether the deblocking is disabled in the PPS (e.g., whether the value of pps_deblocking_filter_disabled_flag is equal to 1 or not) and/or the override flag in the SH (e.g., slice_deblocking_filter_override_flag is equal to 1 or not). a) In one example, when deblocking_filter_override_enabled_flag and pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are equal to 1, the signalling of slice_deblocking_filter_disabled_flag may be skipped. b) Alternatively, when pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are equal to 1, the signalling of slice_deblocking_filter_disabled_flag may be skipped. v. Additionally, when slice_deblocking_filter_disabled_flag is not present, it may be inferred as follows: a) If deblocking_filter_override_enabled_flag, pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are all equal to 1, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to 0. b) Otherwise, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to pps_deblocking_filter_disabled_flag. vi. Additionally, alternatively, when slice_deblocking_filter_disabled_flag is not present, it may be inferred as follows: a) If pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are all equal to 1, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to 0. b) Otherwise, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to pps_deblocking_filter_disabled_flag. 13. It is proposed that cu_skip_flag may be skipped when sps_ibc_enabled_flag is equal to 1 and block size is no smaller than 64x64. a. An example is shown in the fifth embodiment.

6. Example Embodiments

(375) Below are some example embodiments for some of the aspects summarized above in Section 5, which can be applied to the VVC specification. The changed texts are based on the latest VVC text in JVET-Q2001-vE. Most relevant parts that have been added or modified are highlighted in boldface italics, and some of the deleted parts are marked with double brackets (e.g., denotes the deletion of the character “a”).

6.1. First Set of Embodiments

(376) This is a set of embodiments for items 1 summarized above in Section 5.

6.1.1. An Embodiment for 1.a.i

(377) `ph_scaling_list_aps_id` specifies the `adaptation_parameter_set_id` of the scaling list APS.

(378) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(379) When `ph_scaling_list_aps_id` is present, the value of `scaling_list_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be equal to `ChromaArrayType==0? 0:1`. (alternatively, it may be phrased as follows: The value of `scaling_list_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be equal to 1 otherwise).

(380) `scaling_list_chroma_present_flag` equal to 1 specifies that chroma scaling lists are present in `scaling_list_data ( )`

`scaling_list_chroma_present_flag` equal to 0 specifies that chroma scaling lists are not present in `scaling_list_data ( )` [[It is a requirement of bitstream conformance that `scaling_list_chroma_present_flag` shall be equal to 0 when `ChromaArrayType` is equal to 0, and shall be equal to 1 when `ChromaArrayType` is not equal to 0.]]

6.1.2. An Embodiment for 1.a.ii

(381) `ph_scaling_list_aps_id` specifies the `adaptation_parameter_set_id` of the scaling list APS.

(382) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(383) The value of `scaling_list_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` shall be equal to `ChromaArrayType==0? 0:1`. (alternatively, it may be phrased as follows: The value of `scaling_list_chroma_present_flag` of the APS NAL unit having `aps_params_type` equal to `SCALING_APS` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be equal to 1 otherwise).

(384) `scaling_list_chroma_present_flag` equal to 1 specifies that chroma scaling lists are present in `scaling_list_data ( )`

`scaling_list_chroma_present_flag` equal to 0 specifies that chroma scaling lists are not present in `scaling_list_data ( )` [[It is a requirement of bitstream conformance that `scaling_list_chroma_present_flag` shall be equal to 0 when `ChromaArrayType` is equal to 0, and shall be equal to 1 when `ChromaArrayType` is not equal to 0.]]

6.1.3. An Embodiment for 1.b.i

(385) `ph_lmcs_aps_id` specifies the `adaptation_parameter_set_id` of the LMCS APS that the slices associated with the PH refers to.

(386) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(387) When `ph_lmcs_aps_id` is present, the value of `lmcs_delta_abs_crs` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be greater than 0 otherwise.

6.1.4. An Embodiment for 1.b.ii

(388) `ph_lmcs_aps_id` specifies the `adaptation_parameter_set_id` of the LMCS APS that the slices associated with the PH refers to.

(389) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(390) The value of `lmcs_delta_abs_crs` of the APS NAL unit having `aps_params_type` equal to `LMCS_APS` shall be equal to 0 if `ChromaArrayType` is equal to 0 and shall be greater than 0 otherwise.

6.1.5. An Embodiment for 1.c.i

(391) The semantics of PH syntax elements are changes as follows:

(392) `ph_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the *i*-th ALF APS that the luma component of the slices associated with the PH refers to.

(393) The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be equal to 1.

(394) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the picture associated with the PH.

(395) When `ph_alf_aps_id_luma[i]` is present and `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall all be equal to 0.

(396) The semantics of SH syntax elements are changes as follows:

(397) `slice_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the *i*-th ALF APS that the luma component of the slice refers to. -When `slice_alf_enabled_flag` is equal to 1 and `slice_alf_aps_id_luma[i]` is not present, the value of `slice_alf_aps_id_luma[i]` is inferred to be equal to the value of `ph_alf_aps_id_luma[i]`.

(398) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be equal to 1.

(399) When `slice_alf_aps_id_luma[i]` is present and `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to `ALF_APS` and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall all be equal to 0.

(400) And the semantics of the APS syntax elements in the ALF data syntax structure are changed as follows:

(401) `alf_chroma_filter_signal_flag` equal to 1 specifies that a chroma filter is signalled. `alf_chroma_filter_signal_flag` equal to 0 specifies that a chroma filter is not signalled. [[When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` shall be equal to 0.]]

(402) `alf_cc_cb_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cb colour component are signalled.

`alf_cc_cb_filter_signal_flag` equal to 0 specifies that cross-component filters for Cb colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cb_filter_signal_flag` shall be equal to 0.]] `alf_cc_cr_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cr colour component are signalled. `alf_cc_cr_filter_signal_flag` equal to 0 specifies that cross-component filters for the Cr colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` shall be equal to 0.]]

6.1.6. An Embodiment for 1.c.ii

(403) The semantics of PH syntax elements are changes as follows:

(404) `ph_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the *i*-th ALF APS that the luma component of the slices associated with the PH refers to.

(405) The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be equal to 1.

(406) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the picture associated with the PH. When `ChromaArray Type` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall all be equal to 0.

(407) `ph_alf_chroma_idc` equal to 0 specifies that the adaptive loop filter is not applied to Cb and Cr colour components. `ph_alf_chroma_idc` equal to 1 indicates that the adaptive loop filter is applied to the Cb colour component.

(408) `ph_alf_chroma_idc` equal to 2 indicates that the adaptive loop filter is applied to the Cr colour component.

(409) `ph_alf_chroma_idc` equal to 3 indicates that the adaptive loop filter is applied to Cb and Cr colour components.

(410) When `ph_alf_chroma_idc` is not present, it is inferred to be equal to 0.

(411) The semantics of SH syntax elements are changes as follows:

(412) `slice_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the i-th ALF APS that the luma component of the slice refers to. When `slice_alf_enabled_flag` is equal to 1 and `slice_alf_aps_id_luma[i]` is not present, the value of `slice_alf_aps_id_luma[i]` is inferred to be equal to the value of `ph_alf_aps_id_luma[i]`.

(413) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the coded slice NAL unit.

(414) The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be equal to 1.

(415) When `ChromaArrayType` is equal to 0, the values of `alf_chroma_filter_signal_flag`, `alf_cc_cb_filter_signal_flag`, and `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall all be equal to 0.

(416) And the semantics of the APS syntax elements in the ALF data syntax structure are changed as follows:

(417) `alf_chroma_filter_signal_flag` equal to 1 specifies that a chroma filter is signalled. `alf_chroma_filter_signal_flag` equal to 0 specifies that a chroma filter is not signalled. [[When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` shall be equal to 0.]]

(418) `alf_cc_cb_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cb colour component are signalled. `alf_cc_cb_filter_signal_flag` equal to 0 specifies that cross-component filters for Cb colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cb_filter_signal_flag` shall be equal to 0.]]

`alf_cc_cr_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cr colour component are signalled. `alf_cc_cr_filter_signal_flag` equal to 0 specifies that cross-component filters for the Cr colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` shall be equal to 0.]]

6.1.7. An Embodiment for 1.c.iii

(419) The semantics of PH syntax elements are changes as follows:

(420) `ph_alf_aps_id_chroma` specifies the `adaptation_parameter_set_id` of the ALF APS that the chroma component of the slices associated with the PH refers to.

(421) The value of `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_chroma` shall be equal to 1.

(422) When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(423) `ph_cc_alf_cb_aps_id` specifies the `adaptation_parameter_set_id` of the ALF APS that the Cb colour component of the slices associated with the PH refers to. The value of `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cb_aps_id` shall be equal to 1.

(424) When `ChromaArrayType` is equal to 0, `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(425) `ph_cc_alf_cr_aps_id` specifies the `adaptation_parameter_set_id` of the ALF APS that the Cr colour component of the slices associated with the PH refers to.

(426) The value of `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cr_aps_id` shall be equal to 1. When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(427) The semantics of SH syntax elements are changes as follows:

(428) `slice_alf_aps_id_chroma` specifies the `adaptation_parameter_set_id` of the ALF APS that the chroma component of the slice refers to. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_chroma` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. When `slice_alf_enabled_flag` is equal to 1 and `slice_alf_aps_id_chroma` is not present, the value of `slice_alf_aps_id_chroma` is inferred to be equal to the value of `ph_alf_aps_id_chroma`. The value of `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_chroma` shall be equal to 1. When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(429) `slice_cc_alf_cb_aps_id` specifies the `adaptation_parameter_set_id` that the Cb colour component of the slice refers to.

(430) The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cb_aps_id` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. When `slice_cc_alf_cb_enabled_flag` is equal to 1 and `slice_cc_alf_cb_aps_id` is not present, the value of `slice_cc_alf_cb_aps_id` is inferred to be equal to the value of `ph_cc_alf_cb_aps_id`. The value of `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cb_aps_id` shall be equal to 1. When `ChromaArrayType` is equal to 0, `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(431) `slice_cc_alf_cr_aps_id` specifies the `adaptation_parameter_set_id` that the Cr colour component of the slice refers to. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cr_aps_id` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. When `slice_cc_alf_cr_enabled_flag` is equal to 1 and `slice_cc_alf_cr_aps_id` is not present, the value of `slice_cc_alf_cr_aps_id` is inferred to be equal to the value of `ph_cc_alf_cr_aps_id`.

(432) The value of `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cr_aps_id` shall be equal to 1.

(433) When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS shall be equal to 0.

(434) And the semantics of APS syntax elements are changed as follows:

(435) `alf_chroma_filter_signal_flag` equal to 1 specifies that a chroma filter is signalled. `alf_chroma_filter_signal_flag` equal to 0 specifies that a chroma filter is not signalled. [[When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` shall be equal to 0.]]

(436) `alf_cc_cb_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cb colour component are signalled. `alf_cc_cb_filter_signal_flag` equal to 0 specifies that cross-component filters for Cb colour component are not signalled. [[When `ChromaArrayType`

is equal to 0, `alf_cc_cb_filter_signal_flag` shall be equal to 0.] `alf_cc_cr_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cr colour component are signalled. `alf_cc_cr_filter_signal_flag` equal to 0 specifies that cross-component filters for the Cr colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` shall be equal to 0.]]

6.1.8. An Embodiment for 1.d.i

(437) The semantics of APS syntax elements in the ALF data syntax structure are changed as follows:

(438) `alf_chroma_filter_signal_flag` equal to 1 specifies that a chroma filter is signalled. `alf_chroma_filter_signal_flag` equal to 0 specifies that a chroma filter is not signalled. [[When `ChromaArrayType` is equal to 0, `alf_chroma_filter_signal_flag` shall be equal to 0.]]

(439) `alf_cc_cb_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cb colour component are signalled.

`alf_cc_cb_filter_signal_flag` equal to 0 specifies that cross-component filters for Cb colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cb_filter_signal_flag` shall be equal to 0.]] `alf_cc_cr_filter_signal_flag` equal to 1 specifies that cross-component filters for the Cr colour component are signalled. `alf_cc_cr_filter_signal_flag` equal to 0 specifies that cross-component filters for the Cr colour component are not signalled. [[When `ChromaArrayType` is equal to 0, `alf_cc_cr_filter_signal_flag` shall be equal to 0.]]

6.1.9. An Embodiment for 1.d.ii

(440) The semantics of APS syntax elements in the SCALING LIST data syntax structure are changed as follows:

(441) `scaling_list_chroma_present_flag` equal to 1 specifies that chroma scaling lists are present in `scaling_list_data ( )`

`scaling_list_chroma_present_flag` equal to 0 specifies that chroma scaling lists are not present in `scaling_list_data ( )`

(442) [[It is a requirement of bitstream conformance that `scaling_list_chroma_present_flag` shall be equal to 0 when `ChromaArrayType` is equal to 0, and shall be equal to 1 when `ChromaArrayType` is not equal to 0 . . .]]

6.1.10. An Embodiment for 1.e and 1.f

(443) `ph_scaling_list_aps_id` specifies the `adaptation_parameter_set_id` of the scaling list APS.

(444) When `ph_scaling_list_aps_id` is present, the following applies: The `TemporalId` of the APS NAL unit having `aps_params_type` equal to SCALING_APS and `adaptation_parameter_set_id` equal to `ph_scaling_list_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(445) `ph_lmcs_aps_id` specifies the `adaptation_parameter_set_id` of the LMCS APS that the slices associated with the PH refers to.

(446) When `ph_lmcs_aps_id` is present, the following applies: The `TemporalId` of the APS NAL unit having `aps_params_type` equal to LMCS_APS and `adaptation_parameter_set_id` equal to `ph_lmcs_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with PH.

(447) `ph_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the i-th ALF APS that the luma component of the slices associated with the PH refers to.

(448) When `ph_alf_aps_id_luma[i]` is present, the following applies: The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be equal to 1. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the picture associated with the PH.

(449) `ph_alf_chroma_idc` equal to 0 specifies that the adaptive loop filter is not applied to Cb and Cr colour components. `ph_alf_chroma_idc` equal to 1 indicates that the adaptive loop filter is applied to the Cb colour component. `ph_alf_chroma_idc` equal to 2 indicates that the adaptive loop filter is applied to the Cr colour component. `ph_alf_chroma_idc` equal to 3 indicates that the adaptive loop filter is applied to Cb and Cr colour components. When `ph_alf_chroma_idc` is not present, it is inferred to be equal to 0.

(450) `ph_alf_aps_id_chroma` specifies the `adaptation_parameter_set_id` of the ALF APS that the chroma component of the slices associated with the PH refers to.

(451) When `ph_alf_aps_id_chroma` is present, the following applies: The value of `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_chroma` shall be equal to 1. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_alf_aps_id_chroma` shall be less than or equal to the `TemporalId` of the picture associated with the PH.

(452) `ph_cc_alf_cb_aps_id` specifies the `adaptation_parameter_set_id` of the ALF APS that the Cb colour component of the slices associated with the PH refers to.

(453) When `ph_cc_alf_cb_aps_id` is present, the following applies: The value of `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cb_aps_id` shall be equal to 1. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cb_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with the PH.

(454) `ph_cc_alf_cr_aps_id` specifies the `adaptation_parameter_set_id` of the ALF APS that the Cr colour component of the slices associated with the PH refers to.

(455) When `ph_cc_alf_cr_aps_id` is present, the following applies: The value of `alf_cc_cr_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cr_aps_id` shall be equal to 1. The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `ph_cc_alf_cr_aps_id` shall be less than or equal to the `TemporalId` of the picture associated with the PH.

(456) `slice_alf_aps_id_luma[i]` specifies the `adaptation_parameter_set_id` of the i-th ALF APS that the luma component of the slice refers to.-When `slice_alf_enabled_flag` is equal to 1 and `slice_alf_aps_id_luma[i]` is not present, the value of `slice_alf_aps_id_luma[i]` is inferred to be equal to the value of `ph_alf_aps_id_luma[i]`.

(457) When `slice_alf_aps_id_luma[i]` is present, the following applies: The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. The value of `alf_luma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_luma[i]` shall be equal to 1.

(458) `slice_alf_aps_id_chroma` specifies the `adaptation_parameter_set_id` of the ALF APS that the chroma component of the slice refers to. When `slice_alf_enabled_flag` is equal to 1 and `slice_alf_aps_id_chroma` is not present, the value of `slice_alf_aps_id_chroma` is inferred to be equal to the value of `ph_alf_aps_id_chroma`.

(459) When `slice_alf_aps_id_chroma` is present, the following applies: The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_chroma` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. The value of `alf_chroma_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_alf_aps_id_chroma` shall be equal to 1.

(460) `slice_cc_alf_cb_aps_id` specifies the `adaptation_parameter_set_id` that the Cb colour component of the slice refers to.

(461) When `slice_cc_alf_cb_enabled_flag` is equal to 1 and `slice_cc_alf_cb_aps_id` is not present, the value of `slice_cc_alf_cb_aps_id` is inferred to be equal to the value of `ph_cc_alf_cb_aps_id`.

(462) When `slice_cc_alf_cb_aps_id` is present, the following applies: The `TemporalId` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cb_aps_id` shall be less than or equal to the `TemporalId` of the coded slice NAL unit. The value of `alf_cc_cb_filter_signal_flag` of the APS NAL unit having `aps_params_type` equal to ALF_APS and `adaptation_parameter_set_id` equal to `slice_cc_alf_cb_aps_id` shall be equal to 1.

(463) slice_cc_alf_cr_enabled_flag is equal to 1 and slice_cc_alf_cr_aps_id is not present, the value of slice_cc_alf_cr_aps_id is inferred to be equal to the value of ph_cc_alf_cr_aps_id.

(464) When slice_cc_alf_cr_aps_id is present, the following applies: The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cr_aps_id shall be less than or equal to the TemporalId of the coded slice NAL unit. The value of alf_cc_cr_filter_signal_flag of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cr_aps_id shall be equal to 1.

6.1.11. An Embodiment for 1.g

(465) The semantics of SH syntax elements are changes as follows:

(466) slice_alf_aps_id_chroma specifies the adaptation_parameter_set_id of the ALF APS that the chroma component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_alf_aps_id_chroma shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_alf_enabled_flag is equal to 1 and slice_alf_aps_id_chroma is not present and

(467) ChromaArrayType is not equal to 0, the value of slice_alf_aps_id_chroma is inferred to be equal to the value of ph_alf_aps_id_chroma.

(468) slice_cc_alf_cb_aps_id specifies the adaptation_parameter_set_id that the Cb colour component of the slice refers to.

(469) The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cb_aps_id shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_cc_alf_cb_enabled_flag is equal to 1 and slice_cc_alf_cb_aps_id is not present and ChromaArrayType is not equal to 0, the value of slice_cc_alf_cb_aps_id is inferred to be equal to the value of ph_cc_alf_cb_aps_id.

(470) slice_cc_alf_cr_aps_id specifies the adaptation_parameter_set_id that the Cr colour component of the slice refers to. The TemporalId of the APS NAL unit having aps_params_type equal to ALF_APS and adaptation_parameter_set_id equal to slice_cc_alf_cr_aps_id shall be less than or equal to the TemporalId of the coded slice NAL unit. When slice_cc_alf_cr_enabled_flag is equal to 1 and slice_cc_alf_cr_aps_id is not present and ChromaArrayType is not equal to 0, the value of slice_cc_alf_cr_aps_id is inferred to be equal to the value of ph_cc_alf_cr_aps_id.

6.2. Second Set of Embodiments

(471) This is a set of embodiments for items 2 (from 2.a to 2.c) summarized above in Section 5.

(472) The syntax structure pic_parameter_set_rbsp() is changed as follows:

```
(473) TABLE-US-00017 Descriptor seq_parameter_set_rbsp() { ... ... if( ChromaArrayType != 0 ) {      sps_joint_cbr_enabled_flag u(1)
same_qp_table_for_chroma u(1)      numQpTables = same_qp_table_for_chroma ?      1 : ( sps_joint_cbr_enabled_flag ? 3 : 2 )      for( i = 0; i
< numQpTables; i++ ) {          qp_table_start_minus26[i ] se(v)          num_points_in_qp_table_minus1[ i ] ue(v)          for( j = 0; j <=
num_points_in_qp_table_minus1[ i ]; j++ ) {              delta_qp_in_val_minus1[ i ][ j ] ue(v)              delta_qp_diff_val[ i ][ j ] ue(v)
}          }      } ... .. }
```

(474) deblocking_filter_mode_idc equal to 0 specifies that the deblocking filter is not applied for any slice referring to the PPS.

deblocking_filter_mode_idc equal to 1 specifies that the deblocking filter is applied for all slices referring to the PPS, using 0-valued deblocking parameter offsets for β and τ_C . deblocking_filter_mode_idc equal to 2 specifies that the deblocking filter is applied for all slices referring to the PPS, using deblocking parameter offsets for β and τ_C explicitly signalled in the PPS. deblocking_filter_mode_idc equal to 3 specifies that whether the deblocking filter is applied for a slice referring to the PPS is controlled by parameters present either in the PH or the slice header of the slice.

(475) [[deblocking_filter_control_present_flag equal to 1 specifies the presence of deblocking filter control syntax elements in the PPS.

deblocking_filter_control_present_flag equal to 0 specifies the absence of deblocking filter control syntax elements in the PPS.

(476) deblocking_filter_override_enabled_flag equal to 1 specifies the presence of ph_deblocking_filter_override_flag in the PHs referring to the PPS or slice_deblocking_filter_override_flag in the slice headers referring to the PPS. deblocking_filter_override_enabled_flag equal to 0 specifies the absence of ph_deblocking_filter_override_flag in PHs referring to the PPS or slice_deblocking_filter_override_flag in slice headers referring to the PPS. When not present, the value of deblocking_filter_override_enabled_flag is inferred to be equal to 0.

(477) pps_deblocking_filter_disabled_flag equal to 1 specifies that the operation of deblocking filter is not applied for slices referring to the PPS in which slice_deblocking_filter_disabled_flag is not present. pps_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for slices referring to the PPS in which slice_deblocking_filter_disabled_flag is not present. When not present, the value of pps_deblocking_filter_disabled_flag is inferred to be equal to 0.]]

(478) dbf_info_in_ph_flag equal to 1 specifies that deblocking filter information is present in the PH syntax structure and not present in slice headers referring to the PPS that do not contain a PH syntax structure. dbf_info_in_ph_flag equal to 0 specifies that deblocking filter information is not present in the PH syntax structure and may be present in slice headers referring to the PPS that do not contain a PH syntax structure. [[When not present, the value of dbf_info_in_ph_flag is inferred to be equal to 0.]]

(479) And the syntax structure picture_header_structure() is changed as follows:

```
(480) TABLE-US-00018 Descriptor picture_header_structure() {      gdr_or_irap_pic_flag u(1) ...      if(
deblocking_filter_ [[override_enabled_flag]]mode_idc = 3 && dbf_info_in_ph_flag ) {          ph_deblocking_filter_ [[override]]used_flag u(1)
if( ph_deblocking_filter_ [[override]]used_flag ) {              ph_deblocking_ [[filter_disabled]]parameters_override_flag u(1)              if(
[[!]]ph_deblocking_ [[filter_disabled]]parameters_override_flag ) {                  ph_beta_offset_div2 se(v)                  ph_tc_offset_div2 se(v)
ph_cb_beta_offset_div2 se(v)                  ph_cb_tc_offset_div2 se(v)                  ph_cr_beta_offset_div2 se(v)
ph_cr_tc_offset_div2 se(v)              }          }      } ... }
```

(481) ph_deblocking_filter_used_flag equal to 1 specifies that the deblocking filter is applied for the slices in the current picture.

ph_deblocking_filter_used_flag equal to 0 specifies that the deblocking filter is not applied for the slices in the current picture. When not present, the value of ph_deblocking_filter_used_flag is inferred to be equal to (deblocking_filter_mode_idc > 0).

(482) ph_deblocking_parameters_override_flag equal to 1 specifies that deblocking parameters are present in the PH.

ph_deblocking_parameters_override_flag equal to 0 specifies that deblocking parameters are not present in the PH. When not present, the value of ph_deblocking_filter_override_flag is inferred to be equal to 0.

(483) [[ph_deblocking_filter_disabled_flag equal to 1 specifies that the operation of the deblocking filter is not applied for the slices associated with the PH. ph_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for the slices associated with the PH. When ph_deblocking_filter_disabled_flag is not present, it is inferred to be equal to pps_deblocking_filter_disabled_flag.]]

(484) And the syntax structure slice_header() is changed as follows:

```
(485) TABLE-US-00019 Descriptor slice_header() {      picture_header_in_slice_header_flag u(1) ...      if(
deblocking_filter_ [[override_enabled_flag]]mode_idc = 3 && !dbf_info_in_ph_flag )          slice_deblocking_filter_ [[override]]used_flag u(1)
if( slice_deblocking_filter_ [[override]]used_flag ) {              slice_deblocking_ [[filter_disabled]]parameters_override_flag u(1)              if(
[[!]]slice_deblocking_ [[filter_disabled]]parameters_override_flag ) {                  slice_beta_offset_div2 se(v)                  slice_tc_offset_div2 se(v)
slice_cb_beta_offset_div2 se(v)                  slice_cb_tc_offset_div2 se(v)                  slice_cr_beta_offset_div2 se(v)
slice_cr_tc_offset_div2 se(v)              }          }      } ... }
```

(486) slice_deblocking_filter_used_flag equal to 1 specifies that the deblocking filter is applied for the current slice.

slice_deblocking_filter_used_flag equal to 0 specifies that the deblocking filter is not applied for the current slice. When not present, the value of

slice_deblocking_filter_used_flag is inferred to be equal to ph_deblocking_filter_used_flag.

(487) slice_deblocking_filter_override_flag equal to 1 specifies that deblocking parameters are present in the slice header.

slice_deblocking_filter_override_flag equal to 0 specifies that deblocking parameters are not present in the slice header. When not present, the value of slice_deblocking_filter_override_flag is inferred to be equal to [[ph_deblocking_filter_override_flag]] 0.

(488) [[slice_deblocking_filter_disabled_flag equal to 1 specifies that the operation of the deblocking filter is not applied for the current slice. slice_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for the current slice. When slice_deblocking_filter_disabled_flag is not present, it is inferred to be equal to ph_deblocking_filter_disabled_flag.]]

(489) And the decoding process of deblocking filter process is changed as follows:

(490) 8.8.3 Deblocking Filter Process

(491) 8.8.3.1 General

(492) The deblocking filter process is applied to all coding subblock edges and transform block edges of a picture, except the following types of edges: Edges that are at the boundary of the picture, Edges that coincide with the boundaries of a subpicture with subpicture index subpicIdx and loop_filter_across_subpic_enabled_flag[subpicIdx] is equal to 0, Edges that coincide with the virtual boundaries of the picture when VirtualBoundariesPresentFlag is equal to 1, Edges that coincide with tile boundaries when loop_filter_across_tiles_enabled_flag is equal to 0, Edges that coincide with slice boundaries when loop_filter_across_slices_enabled_flag is equal to 0, Edges that coincide with upper or left boundaries of slices with slice_deblocking_filter_used_flag equal to 0, Edges within slices with slice_deblocking_filter_used[[disabled]]_flag equal to 0, Edges that do not correspond to 4x4 sample grid boundaries of the luma component, Edges that do not correspond to 8x8 sample grid boundaries of the chroma component, Edges within the luma component for which both sides of the edge have intra_bdpcm_luma_flag equal to 1, Edges within the chroma components for which both sides of the edge have intra_bdpcm_chroma_flag equal to 1, Edges of chroma subblocks that are not edges of the associated transform unit.

(493) The edge type, vertical or horizontal, is represented by the variable edgeType as specified in Table 42.

(494) TABLE-US-00020 TABLE 42 edgeType Name of edgeType 0 (vertical edge) EDGE_VER 1 (horizontal edge) EDGE_HOR

(495) When slice_deblocking_filter_used[[disabled]]_flag of the current slice is equal to 1, the following applies: The variable treeType is set equal to DUAL_TREE_LUMA. The vertical edges are filtered by invoking the deblocking filter process for one direction as specified in clause 8.8.3.2 with the variable treeType, the reconstructed picture prior to deblocking, i.e., the array recPicture.sub.L and the variable edgeType set equal to EDGE_VER as inputs, and the modified reconstructed picture after deblocking, i.e., the array recPicture.sub.L as outputs. The horizontal edge are filtered by invoking the deblocking filter process for one direction as specified in clause 8.8.3.2 with the variable treeType, the modified reconstructed picture after deblocking, i.e., the array recPicture.sub.L and the variable edgeType set equal to EDGE_HOR as inputs, and the modified reconstructed picture after deblocking, i.e., the array recPicture.sub.L as outputs. When ChromaArrayType is not equal to 0, the following applies: The variable treeType is set equal to DUAL_TREE_CHROMA. The vertical edges are filtered by invoking the deblocking filter process for one direction as specified in clause 8.8.3.2 with the variable treeType, the reconstructed picture prior to deblocking, i.e., the arrays recPicture.sub.Cb and recPicture.sub.Cr, and the variable edgeType set equal to EDGE_VER as inputs, and the modified reconstructed picture after deblocking, i.e., the arrays recPicture.sub.Cb and recPicture.sub.Cr as outputs. The horizontal edge are filtered by invoking the deblocking filter process for one direction as specified in clause 8.8.3.2 with the variable treeType, the modified reconstructed picture after deblocking, i.e., the arrays recPicture.sub.Cb and recPicture.sub.Cr, and the variable edgeType set equal to EDGE_HOR as inputs, and the modified reconstructed picture after deblocking, i.e., the arrays recPicture.sub.Cb and recPicture.sub.Cr as outputs.

6.3. Third Set of Embodiments

(496) The changes, marked in boldfaced italics, are based on JVET-Q2001-vE.

(497) The i-th chroma QP mapping table ChromaQpTable[i] for i=0 . . . numQpTables-1 is derived as follows:

(498) TABLE-US-00021
$$\begin{aligned} \text{qpInVal}[i][0] &= \text{qp_table_start_minus26}[i] + 26 \\ \text{qpOutVal}[i][0] &= \text{qpInVal}[i][0] \text{ for } (j = 0; j \leq \text{num_points_in_qp_table_minus}[i]; j++) \{ \\ &\quad \text{qpInVal}[i][j+1] = \text{qpInVal}[i][j] + \text{delta_qp_in_val_minus}[i][j] + 1 \\ &\quad \text{qpOutVal}[i][j+1] = \text{qpOutVal}[i][j] + ((\text{delta_qp_in_val_minus}[i][j] + 1) \text{ circumflex over } ()) \text{delta_qp_diff_val}[i][j]) \\ &\quad \text{ChromaQpTable}[i][j] \\ \text{qpInVal}[i][0] &= \text{qpOutVal}[i][0] \text{ for } (k = \text{qpInVal}[i][0] - 1; k \geq -\text{QpBdOffset}; k--) \text{ChromaQpTable}[i][k] = \text{Clip3}(-\text{QpBdOffset}, 63, \\ &\text{ChromaQpTable}[i][k+1] - 1) \text{ for } (j = 0; j \leq \text{num_points_in_qp_table_minus1}[i]; j++) \{ \\ &\quad \text{sh} = (\text{delta_qp_in_val_minus}[i][j] + 1) >> 1 \\ &\quad \text{for } (k = \text{qpInVal}[i][j] + 1, m = 1; k \leq \text{qpInVal}[i][j+1]; k++) \text{ChromaQpTable}[i][k] = \text{ChromaQpTable}[i][\text{qpInVal}[i][j] \\ &\quad + ((\text{qpOutVal}[i][j] + 1) - \text{qpOutVal}[i][j]) * m + \text{sh}) / (\text{delta_qp_in_val_minus1}[i][j] + 1) \} \text{ for } (k = \text{qpInVal}[i][\text{num_points_in_qp_table_minus1}[i] + 1]; k \leq 63; k++) \text{ChromaQpTable}[i][k] = \text{Clip3}(-\text{QpBdOffset}, 63, \text{ChromaQpTable}[i][k-1] \\ &+ 1) \end{aligned}$$

6.4. Fourth Embodiment

(499) PPS semantics (based on the text in JVET-R0159-v2, excluding the SPS flag):

(500) deblocking_filter_control_present_flag equal to 1 specifies the presence of deblocking filter control syntax elements in the PPS. deblocking_filter_control_present_flag equal to 0 specifies the absence of deblocking filter control syntax elements in the PPS and that the deblocking filter is applied for all slices referring to the PPS, using 0-valued deblocking parameter offsets for β and tc .

(501) deblocking_filter_override_enabled_flag equal to 1 specifies the presence of ph_deblocking_filter_override_flag in the PHs referring to the PPS or slice_deblocking_filter_override_flag in the slice headers referring to the PPS. deblocking_filter_override_enabled_flag equal to 0 specifies the absence of ph_deblocking_filter_override_flag in PHs referring to the PPS or slice_deblocking_filter_override_flag in slice headers referring to the PPS. When not present, the value of deblocking_filter_override_enabled_flag is inferred to be equal to 0.

(502) [[pps_deblocking_filter_disabled_flag equal to 1 specifies that the operation of deblocking filter is not applied for slices referring to the PPS of which slice_deblocking_filter_disabled_flag and ph_deblockig_filter_disabled_flag are not present. pps_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for slices referring to the PPS of which slice_deblocking_filter_disabled_flag and ph_deblockig_filter_disabled_flag are not present. When not of present, the value pps_deblocking_filter_disabled_flag is inferred to be equal to 0.

(503) Or

(504) pps_deblocking_filter_disabled_flag equal to 1 specifies that the operation of deblocking filter is not applied for slices referring to the PPS when deblocking_filter_override_enabled_flag is equal to 0. pps_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for slices referring to the PPS when deblocking_filter_override_enabled_flag is equal to 0. When not present, the value of pps_deblocking_filter_disabled_flag is inferred to be equal to 0.]]

(505) pps_deblocking_filter_disabled_flag equal to 1 specifies that the operation of deblocking filter is disabled for slices referring to the PPS unless indicated otherwise at the picture or slice level.

(506) pps_deblocking_filter_disabled_flag equal to 0 specifies that the operation of deblocking filter is enabled for slices referring to the PPS unless indicated otherwise at the picture or slice level.

(507) The syntax structure picture_header_structure () is changed as follows:

(508) TABLE-US-00022 Descriptor picture_header_structure () { gdr_or_irap_pic_flag u(1) ... if(deblocking_filter_override_enabled_flag && dbf_info_in_ph_flag) { ph_deblocking_filter_override_flag u(1) if(ph_deblocking_filter_override_flag) { if(!pps_deblocking_filter_disabled_flag) ph_deblocking_filter_disabled_flag u(1) if(!ph_deblocking_filter_disabled_flag) { ph_beta_offset_div2 se(v) ph_tc_offset_div2 se(v) ph_cb_beta_offset_div2 se(v) ph_cb_tc_offset_div2 se(v)

ph_cr_beta_offset_div2 se(v) ph_cr_tc_offset_div2 se(v) } } } ...

(509) ph_deblocking_filter_override_flag equal to 1 specifies that deblocking parameters are present in the PH. ph_deblocking_filter_override_flag equal to 0 specifies that deblocking parameters are not present in the PH. When not present, the value of ph_deblocking_filter_override_flag is inferred to be equal to 0.

(510) ph_deblocking_filter_disabled_flag equal to 1 specifies that the operation of the deblocking filter is not applied for the slices associated with the PH [[in which slice_deblocking_filter_disabled_flag is not present [note: When ph_deblocking_filter_disabled_flag is present, slice_deblocking_filter_disabled_flag won't be present in the SH of any slice of the picture, therefore the removal.]]]. ph_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for the slices associated with the PH [[in which slice_deblocking_filter_disabled_flag is not present [note: When ph_deblocking_filter_disabled_flag is present, slice_deblocking_filter_disabled_flag won't be present in the SH of any slice of the picture, therefore the removal.]]] When ph_deblocking_filter_disabled_flag is not present, it is inferred as follows: If deblocking_filter_override_enabled_flag, pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are all equal to 1, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to 0.[alternatively, this sentence can be also phrased as: If pps_deblocking_filter_disabled_flag and ph_deblocking_filter_override_flag are all equal to 1, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to 0.] Otherwise, the value of ph_deblocking_filter_disabled_flag is inferred to be equal to pps_deblocking_filter_disabled_flag.

(511) And the syntax structure slice_header () is changed as follows:

(512) TABLE-US-00023 Descriptor slice_header() { picture_header_in_slice_header_flag u(1) ... if(deblocking_filter_override_enabled_flag && !dbf_info_in_ph_flag) slice_deblocking_filter_override_flag u(1) if(slice_deblocking_filter_override_flag) { if(!pps_deblocking_filter_disabled_flag) slice_deblocking_filter_disabled_flag u(1) if(!slice_deblocking_filter_disabled_flag) { slice_beta_offset_div2 se(v) slice_tc_offset_div2 se(v) slice_cb_beta_offset_div2 se(v) slice_cb_tc_offset_div2 se(v) slice_cr_beta_offset_div2 se(v) slice_cr_tc_offset_div2 se(v) } } ...

(513) slice_deblocking_filter_override_flag equal to 1 specifies that deblocking parameters are present in the slice header. slice_deblocking_filter_override_flag equal to 0 specifies that deblocking parameters are not present in the slice header. When not present, the value of slice_deblocking_filter_override_flag is inferred to be equal to [ph_deblocking_filter_override_flag] 0.

(514) slice_deblocking_filter_disabled_flag equal to 1 specifies that the operation of the deblocking filter is not applied for the current slice. slice_deblocking_filter_disabled_flag equal to 0 specifies that the operation of the deblocking filter is applied for the current slice.

(515) When slice_deblocking_filter_disabled_flag is not present, it is inferred as follows: 1 If deblocking_filter_override_enabled_flag, pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are all equal to 1, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to 0.[alternatively, this sentence can be also phrased as: If pps_deblocking_filter_disabled_flag and slice_deblocking_filter_override_flag are all equal to 1, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to 0.] Otherwise, the value of slice_deblocking_filter_disabled_flag is inferred to be equal to ph_deblocking_filter_disabled_flag.

6.5. Fifth Embodiment

(516) The changes marked in highlighted in boldface italics, are based on JVET-P2001-vE.

(517) TABLE-US-00024 Descriptor coding_unit(x0, y0, cbWidth, cbHeight, cqtDepth, treeType, modeType) { chType = treeType = = DUAL_TREE_CHROMA ? 1 : 0 if(slice_type != 1 || (sps_ibc_enabled_flag && cbWidth <= 64 && cbHeight <= 64)) { if(treeType != DUAL_TREE_CHROMA && ((! (cbWidth == 4 && cbHeight == 4) && modeType != MODE_TYPE_INTRA) || (sps_ibc_enabled_flag && cbWidth <= 64 && cbHeight <= 64))) cu_skip_flag[x0][y0] ae(v) if(cu_skip_flag[x0][y0] == 0 && slice_type != I && ! (cbWidth == 4 && cbHeight == 4) && modeType == MODE_TYPE_ALL) pred_mode_flag ae(v) if(((slice_type == I && cu_skip_flag[x0][y0] == 0) || (slice_type != 1 && (CuPredMode[chType][x0][y0] != MODE_INTRA || (((cbWidth == 4 && cbHeight == 4) || modeType == MODE_TYPE_INTRA) && cu_skip_flag[x0][y0] == 0)))) && cbWidth <= 64 && cbHeight <= 64 && modeType != MODE_TYPE_INTER && sps_ibc_enabled_flag && treeType != DUAL_TREE_CHROMA) pred_mode_ibc_flag ae(v) } ... }

(518) FIG. 1 is a block diagram showing an example video processing system **1900** in which various techniques disclosed herein may be implemented. Various implementations may include some or all of the components of the system **1900**. The system **1900** may include input **1902** for receiving video content. The video content may be received in a raw or uncompressed format, e.g., 8 or 10 bit multi-component pixel values, or may be in a compressed or encoded format. The input **1902** may represent a network interface, a peripheral bus interface, or a storage interface. Examples of network interface include wired interfaces such as Ethernet, passive optical network (PON), etc. and wireless interfaces such as wireless fidelity (Wi-Fi) or cellular interfaces.

(519) The system **1900** may include a coding component **1904** that may implement the various coding or encoding methods described in the present disclosure. The coding component **1904** may reduce the average bitrate of video from the input **1902** to the output of the coding component **1904** to produce a coded representation of the video. The coding techniques are therefore sometimes called video compression or video transcoding techniques. The output of the coding component **1904** may be either stored, or transmitted via a communication connected, as represented by the component **1906**. The stored or communicated bitstream (or coded) representation of the video received at the input **1902** may be used by the component **1908** for generating pixel values or displayable video that is sent to a display interface **1910**. The process of generating user-viewable video from the bitstream representation is sometimes called video decompression. Furthermore, while certain video processing operations are referred to as “coding” operations or tools, it will be appreciated that the coding tools or operations are used at an encoder and corresponding decoding tools or operations that reverse the results of the coding will be performed by a decoder.

(520) Examples of a peripheral bus interface or a display interface may include universal serial bus (USB) or high definition multimedia interface (HDMI) or Displayport, and so on. Examples of storage interfaces include serial advanced technology attachment (SATA), peripheral component interconnect (PCI), integrated drive electronics (IDE) interface, and the like. The techniques described in the present disclosure may be embodied in various electronic devices such as mobile phones, laptops, smartphones or other devices that are capable of performing digital data processing and/or video display.

(521) FIG. 2 is a block diagram of a video processing apparatus **3600**. The apparatus **3600** may be used to implement one or more of the methods described herein. The apparatus **3600** may be embodied in a smartphone, tablet, computer, Internet of Things (IoT) receiver, and so on. The apparatus **3600** may include one or more processors **3602**, one or more memories **3604** and video processing hardware **3606**. The processor(s) **3602** may be configured to implement one or more methods described in the present disclosure. The memory (memories) **3604** may be used for storing data and code used for implementing the methods and techniques described herein. The video processing hardware **3606** may be used to implement, in hardware circuitry, some techniques described in the present disclosure.

(522) FIG. 4 is a block diagram that illustrates an example video coding system **100** that may utilize the techniques of this disclosure.

(523) As shown in FIG. 4, video coding system **100** may include a source device **110** and a destination device **120**. Source device **110** generates encoded video data which may be referred to as a video encoding device. Destination device **120** may decode the encoded video data generated by source device **110** which may be referred to as a video decoding device.

(524) Source device **110** may include a video source **112**, a video encoder **114**, and an input/output (I/O) interface **116**.

(525) Video source **112** may include a source such as a video capture device, an interface to receive video data from a video content provider, and/or a computer graphics system for generating video data, or a combination of such sources. The video data may comprise one or more pictures. Video

encoder **114** encodes the video data from video source **112** to generate a bitstream. The bitstream may include a sequence of bits that form a coded representation of the video data. The bitstream may include coded pictures and associated data. The coded picture is a coded representation of a picture. The associated data may include sequence parameter sets, picture parameter sets, and other syntax structures. I/O interface **116** may include a modulator/demodulator (modem) and/or a transmitter. The encoded video data may be transmitted directly to destination device **120** via I/O interface **116** through network **130a**. The encoded video data may also be stored onto a storage medium/server **130b** for access by destination device **120**.

(526) Destination device **120** may include an I/O interface **126**, a video decoder **124**, and a display device **122**.

(527) I/O interface **126** may include a receiver and/or a modem. I/O interface **126** may acquire encoded video data from the source device **110** or the storage medium/server **130b**. Video decoder **124** may decode the encoded video data. Display device **122** may display the decoded video data to a user. Display device **122** may be integrated with the destination device **120**, or may be external to destination device **120** which be configured to interface with an external display device.

(528) Video encoder **114** and video decoder **124** may operate according to a video compression standard, such as the High Efficiency Video Coding (HEVC) standard, Versatile Video Coding (VVC) standard and other current and/or further standards.

(529) FIG. 5 is a block diagram illustrating an example of video encoder **200**, which may be video encoder **114** in the system **100** illustrated in FIG. 4.

(530) Video encoder **200** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. 5, video encoder **200** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of video encoder **200**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(531) The functional components of video encoder **200** may include a partition unit **201**, a prediction unit **202** which may include a mode select unit **203**, a motion estimation unit **204**, a motion compensation unit **205** and an intra prediction unit **206**, a residual generation unit **207**, a transform unit **208**, a quantization unit **209**, an inverse quantization unit **210**, an inverse transform unit **211**, a reconstruction unit **212**, a buffer **213**, and an entropy encoding unit **214**.

(532) In other examples, video encoder **200** may include more, fewer, or different functional components. In an example, prediction unit **202** may include an intra block copy (IBC) unit. The IBC unit may perform prediction in an IBC mode in which at least one reference picture is a picture where the current video block is located.

(533) Furthermore, some components, such as motion estimation unit **204** and motion compensation unit **205** may be highly integrated, but are represented in the example of FIG. 5 separately for purposes of explanation.

(534) Partition unit **201** may partition a picture into one or more video blocks. Video encoder **200** and video decoder **300** may support various video block sizes.

(535) Mode select unit **203** may select one of the coding modes, intra or inter, e.g., based on error results, and provide the resulting intra- or inter-coded block to a residual generation unit **207** to generate residual block data and to a reconstruction unit **212** to reconstruct the encoded block for use as a reference picture. In some example, mode select unit **203** may select a combination of intra and inter prediction (CIIP) mode in which the prediction is based on an inter prediction signal and an intra prediction signal. Mode select unit **203** may also select a resolution for a motion vector (e.g., a sub-pixel or integer pixel precision) for the block in the case of inter-prediction.

(536) To perform inter prediction on a current video block, motion estimation unit **204** may generate motion information for the current video block by comparing one or more reference frames from buffer **213** to the current video block. Motion compensation unit **205** may determine a predicted video block for the current video block based on the motion information and decoded samples of pictures from buffer **213** other than the picture associated with the current video block.

(537) Motion estimation unit **204** and motion compensation unit **205** may perform different operations for a current video block, for example, depending on whether the current video block is in an I slice, a P slice, or a B slice.

(538) In some examples, motion estimation unit **204** may perform uni-directional prediction for the current video block, and motion estimation unit **204** may search reference pictures of list 0 or list 1 for a reference video block for the current video block. Motion estimation unit **204** may then generate a reference index that indicates the reference picture in list 0 or list 1 that contains the reference video block and a motion vector that indicates a spatial displacement between the current video block and the reference video block. Motion estimation unit **204** may output the reference index, a prediction direction indicator, and the motion vector as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current block based on the reference video block indicated by the motion information of the current video block.

(539) In other examples, motion estimation unit **204** may perform bi-directional prediction for the current video block, motion estimation unit **204** may search the reference pictures in list 0 for a reference video block for the current video block and may also search the reference pictures in list 1 for another reference video block for the current video block. Motion estimation unit **204** may then generate reference indexes that indicate the reference pictures in list 0 and list 1 containing the reference video blocks and motion vectors that indicate spatial displacements between the reference video blocks and the current video block. Motion estimation unit **204** may output the reference indexes and the motion vectors of the current video block as the motion information of the current video block. Motion compensation unit **205** may generate the predicted video block of the current video block based on the reference video blocks indicated by the motion information of the current video block.

(540) In some examples, motion estimation unit **204** may output a full set of motion information for decoding processing of a decoder.

(541) In some examples, motion estimation unit **204** may not output a full set of motion information for the current video. Rather, motion estimation unit **204** may signal the motion information of the current video block with reference to the motion information of another video block. For example, motion estimation unit **204** may determine that the motion information of the current video block is sufficiently similar to the motion information of a neighboring video block.

(542) In one example, motion estimation unit **204** may indicate, in a syntax structure associated with the current video block, a value that indicates to the video decoder **300** that the current video block has the same motion information as another video block.

(543) In another example, motion estimation unit **204** may identify, in a syntax structure associated with the current video block, another video block and a motion vector difference (MVD). The motion vector difference indicates a difference between the motion vector of the current video block and the motion vector of the indicated video block. The video decoder **300** may use the motion vector of the indicated video block and the motion vector difference to determine the motion vector of the current video block.

(544) As discussed above, video encoder **200** may predictively signal the motion vector. Two examples of predictive signaling techniques that may be implemented by video encoder **200** include advanced motion vector prediction (AMVP) and merge mode signaling.

(545) Intra prediction unit **206** may perform intra prediction on the current video block. When intra prediction unit **206** performs intra prediction on the current video block, intra prediction unit **206** may generate prediction data for the current video block based on decoded samples of other video blocks in the same picture. The prediction data for the current video block may include a predicted video block and various syntax elements.

(546) Residual generation unit **207** may generate residual data for the current video block by subtracting (e.g., indicated by the minus sign) the predicted video block(s) of the current video block from the current video block. The residual data of the current video block may include residual video blocks that correspond to different sample components of the samples in the current video block.

(547) In other examples, there may be no residual data for the current video block for the current video block, for example in a skip mode, and residual generation unit **207** may not perform the subtracting operation.

(548) Transform processing unit **208** may generate one or more transform coefficient video blocks for the current video block by applying one or

more transforms to a residual video block associated with the current video block.

(549) After transform processing unit **208** generates a transform coefficient video block associated with the current video block, quantization unit **209** may quantize the transform coefficient video block associated with the current video block based on one or more quantization parameter (QP) values associated with the current video block.

(550) Inverse quantization unit **210** and inverse transform unit **211** may apply inverse quantization and inverse transforms to the transform coefficient video block, respectively, to reconstruct a residual video block from the transform coefficient video block. Reconstruction unit **212** may add the reconstructed residual video block to corresponding samples from one or more predicted video blocks generated by the prediction unit **202** to produce a reconstructed video block associated with the current block for storage in the buffer **213**.

(551) After reconstruction unit **212** reconstructs the video block, loop filtering operation may be performed reduce video blocking artifacts in the video block.

(552) Entropy encoding unit **214** may receive data from other functional components of the video encoder **200**. When entropy encoding unit **214** receives the data, entropy encoding unit **214** may perform one or more entropy encoding operations to generate entropy encoded data and output a bitstream that includes the entropy encoded data.

(553) FIG. **6** is a block diagram illustrating an example of video decoder **300** which may be video decoder **124** in the system **100** illustrated in FIG. **4**.

(554) The video decoder **300** may be configured to perform any or all of the techniques of this disclosure. In the example of FIG. **6**, the video decoder **300** includes a plurality of functional components. The techniques described in this disclosure may be shared among the various components of the video decoder **300**. In some examples, a processor may be configured to perform any or all of the techniques described in this disclosure.

(555) In the example of FIG. **6**, video decoder **300** includes an entropy decoding unit **301**, a motion compensation unit **302**, an intra prediction unit **303**, an inverse quantization unit **304**, an inverse transformation unit **305**, and a reconstruction unit **306** and a buffer **307**. Video decoder **300** may, in some examples, perform a decoding pass generally reciprocal to the encoding pass described with respect to video encoder **200** (FIG. **5**).

(556) Entropy decoding unit **301** may retrieve an encoded bitstream. The encoded bitstream may include entropy coded video data (e.g., encoded blocks of video data). Entropy decoding unit **301** may decode the entropy coded video data, and from the entropy decoded video data, motion compensation unit **302** may determine motion information including motion vectors, motion vector precision, reference picture list indexes, and other motion information. Motion compensation unit **302** may, for example, determine such information by performing the AMVP and merge mode.

(557) Motion compensation unit **302** may produce motion compensated blocks, possibly performing interpolation based on interpolation filters. Identifiers for interpolation filters to be used with sub-pixel precision may be included in the syntax elements.

(558) Motion compensation unit **302** may use interpolation filters as used by video encoder **200** during encoding of the video block to calculate interpolated values for sub-integer pixels of a reference block. Motion compensation unit **302** may determine the interpolation filters used by video encoder **200** according to received syntax information and use the interpolation filters to produce predictive blocks.

(559) Motion compensation unit **302** may use some of the syntax information to determine sizes of blocks used to encode frame(s) and/or slice(s) of the encoded video sequence, partition information that describes how each macroblock of a picture of the encoded video sequence is partitioned, modes indicating how each partition is encoded, one or more reference frames (and reference frame lists) for each inter-encoded block, and other information to decode the encoded video sequence.

(560) Intra prediction unit **303** may use intra prediction modes for example received in the bitstream to form a prediction block from spatially adjacent blocks. Inverse quantization unit **304** inverse quantizes, i.e., de-quantizes, the quantized video block coefficients provided in the bitstream and decoded by entropy decoding unit **301**. Inverse transform unit **305** applies an inverse transform.

(561) Reconstruction unit **306** may sum the residual blocks with the corresponding prediction blocks generated by motion compensation unit **302** or intra-prediction unit **303** to form decoded blocks. If desired, a deblocking filter may also be applied to filter the decoded blocks in order to remove blockiness artifacts. The decoded video blocks are then stored in buffer **307**, which provides reference blocks for subsequent motion compensation/intra prediction and also produces decoded video for presentation on a display device.

(562) A listing of examples preferred by some embodiments is provided next.

(563) The first set of clauses show example embodiments of techniques discussed in the previous section. The following clauses show example embodiments of techniques discussed in the previous section (e.g., item 1).

(564) 1. A video processing method (e.g., method **3000** shown in FIG. **3**), comprising: performing (**3002**) a conversion between a video having one or more chroma components, the video comprising one or more video pictures comprising one or more slices and a coded representation of the video, wherein the coded representation conforms to a format rule, wherein the format rule specifies that a chroma array type field controls a constraint on a conversion characteristic of chroma used during the conversion.

(565) 2. The method of clause 1, wherein the conversion characteristic includes a constraint on a field indicative of presence of one or more scaling lists for the one or more chroma components.

(566) 3. The method of clause 1, wherein the conversion characteristic includes a constraint on a value of a field indicative of a codeword used for signaling luma mapping with chroma scaling.

(567) 4. The method of clause 1, wherein the conversion characteristic includes a constraint on values of syntax elements describing an adaptation parameter set for an adaptive loop filter used during the conversion.

(568) 5. The method of clause 1, wherein the format rule specifies to use a same semantics of one or more entries of an adaptation parameter set for the chroma array type field signaling a 4:0:0 format or a separate color coding format.

(569) 6. The method of clause 5, wherein the one or more entries include an adaptive loop filter parameter or a scaling list parameter or a luma mapping with chroma scaling parameter.

(570) 7. The method of clauses 5-6, wherein the format rule further specifies that a constraint on the one or more entries of the adaptation parameter set is dependent on whether an identifier of the adaptation parameter set is included in the bitstream.

(571) The following clauses show example embodiments of techniques discussed in the previous section (e.g., item 2).

(572) 8. A video processing method, comprising: performing a conversion between a video comprising one or more video pictures comprising one or more video regions and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies the include a deblocking mode indicator for a video region indicative of applicability of a deblocking filter to the video region during the conversion.

(573) 9. The method of clause 8, wherein the deblocking mode indicator is an N bit field where N is an integer greater than 1.

(574) 10. The method of any of clauses 8-9, wherein the deblocking mode indicator for the video region is included in a picture parameter set.

(575) 11. The method of clause 8, wherein the deblocking mode indicator corresponds to a flag included in a header of the video region indicating applicability of the deblocking filter to the video region.

(576) 12. The method of any of clauses 8-11, wherein the format rule specifies that a flag that signals whether deblocking filter parameters signaled in the deblocking mode indicator are to override default parameters.

(577) 13. The method of any of clauses 8 -12, wherein the video region corresponds to a video picture or a video slice.

(578) The following clauses show example embodiments of techniques discussed in the previous section (e.g., item 3).

(579) 14. A video processing method, comprising: performing a conversion between a video comprising one or more video pictures comprising one or more video slices and/or one or more video subpictures and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies that a flag indicating whether a single slice per subpicture mode is deemed to be enabled for a video picture in case that a

picture partitioning is disabled for the video picture.

(580) The following clauses show example embodiments of techniques discussed in the previous section (e.g., item 4).

(581) 15. A video processing method, comprising: performing a conversion between a video comprising one or more video pictures comprising one or more video slices and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies that a picture or a slice level chroma quantization parameter offset is signaled in a picture header or a slice header.

(582) 16. The method of clause 15, wherein the format rule specifies to include slice level chroma quantization parameter offsets in the slice header.

(583) The following clauses show example embodiments of techniques discussed in the previous section (e.g., item 5).

(584) 17. A video processing method, comprising: performing a conversion between a video comprising one or more video pictures comprising one or more video slices and a coded representation of the video, wherein the coded representation conforms to a format rule that specifies that a chroma quantization parameter (QP) table applicable for conversion of a video block of the video is derived as an XOR operation between $(\text{delta_qp_in_val_minus1}[i][j]+1)$ and $\text{delta_qp_diff_val}[i][j]$, wherein $\text{delta_qp_in_val_minus1}[i][j]$ specifies a delta value used to derive the input coordinate of the j-th pivot point of the i-th chroma mapping table and $\text{delta_qp_diff_val}[i][j]$ specifies a delta value used to derive the output coordinate of the j-th pivot point of the i-th chroma QP mapping table, where i and j are integers.

(585) 18. The method of any of clauses 1 to 17, wherein the conversion comprises encoding the video into the coded representation.

(586) 19. The method of any of clauses 1 to 17, wherein the conversion comprises decoding the coded representation to generate pixel values of the video.

(587) 20. A video decoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 19.

(588) 21. A video encoding apparatus comprising a processor configured to implement a method recited in one or more of clauses 1 to 19.

(589) 22. A computer program product having computer code stored thereon, the code, when executed by a processor, causes the processor to implement a method recited in any of clauses 1 to 19.

(590) 23. A method, apparatus or system described in the present disclosure.

(591) A second set of clauses show example embodiments of techniques discussed in the previous section (e.g., item 1.h).

(592) 1. A method of processing video data (e.g., method **700** as shown in FIG. 7), comprising: performing **710** a conversion between a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first field in an adaptation parameter set that is based on a second field is derived using information included in a picture header or a slice header if a picture or a slice referring to the adaptation parameter set.

(593) 2. The method of clause 1, wherein the second field indicates a chroma format identifier of the video.

(594) 3. The method of clause 1 or 2, wherein the second field indicates a chroma sampling relative to a luma sampling.

(595) 4. The method of clause 1, wherein the second field indicates a presence of a chroma component in the video.

(596) 5. The method of clause 1, wherein the format rule further specifies that the adaptation parameter set is determined by an index included in the picture header or the slice header.

(597) 6. The method of clause 1, wherein the second field is a syntax element included in a sequence parameter set.

(598) 7. The method of clause 1, wherein the second field is based on information included in a sequence parameter set.

(599) 8. The method of clause 7, wherein the sequence parameter set is determined by an indication included in a picture parameter set that is further determined by another indication included in the picture header or the slice header.

(600) 9. The method of clause 1, wherein the value is checked after parsing the adaptation parameter set and at least one of the picture header or the slice header.

(601) 10. The method of clause 1, wherein the first field corresponds to a chroma present flag specifying a presence of chroma related syntax elements.

(602) 11. The method of clause 1, wherein the format rule specifies that a syntax element specifying whether chroma related syntax elements are included in an adaptation parameter set syntax structure.

(603) 12. The method of clause 1, wherein the first field corresponds to a chroma present flag indicating whether an APS (Adaptation Parameter Set) NAL (Network Abstraction Layer) unit includes chroma related syntax elements or not.

(604) 13. The method of clause 1, wherein the format rule further specifies that the value of the first field is determined based on a value of the second field.

(605) 14. The method of clause 13, wherein in response to the value of the second field indicating that the chroma component is not included in the video, the value of the first field indicates that the APS (adaptation parameter set) NAL (Network Abstraction Layer) unit does not include chroma related syntax elements.

(606) 15. The method of clause 13, wherein in response to the value of the second field indicating that the chroma component is included in the video, the value of the first field indicates that the APS (adaptation parameter set) NAL (Network Abstraction Layer) unit includes chroma related syntax elements.

(607) 16. The method of clause 1, wherein the format rule further specifies that the value of the first field is dependent on an APS (adaptation parameter set) identifier in the picture header or the slice header.

(608) 17. The method of clause 16, wherein the format rule specifies, in case that the second field has a certain value, to set a requirement for the value of the first field of an APS (Adaptation Parameter Set) NAL (Network Abstraction Layer) unit having an APS parameter type indicating ALF (adaptive loop filter) type and the APS identifier equal to information included in the picture header or the slice header.

(609) 18. The method of clause 17, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is equal to a second certain value.

(610) 19. The method of clause 18, wherein the first certain value and the second certain value are equal to 0.

(611) 20. The method of clause 17, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is greater than a second certain value.

(612) 21. The method of clause 20, wherein the first certain value is equal to 1 and the second certain value is equal to 0.

(613) 22. The method of clause 16, wherein the format rule specifies, in case that the second field has a certain value, to set a requirement for the value of the first field of an APS (Adaptation Parameter Set) NAL (Network Abstraction Layer) unit having an APS parameter type indicating SCALING type and the APS identifier equal to information included in the picture header or the slice header.

(614) 23. The method of clause 22, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is equal to a second certain value.

(615) 24. The method of clause 23, wherein the first certain value and the second certain value is equal to 0.

(616) 25. The method of clause 22, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is greater than a second certain value.

(617) 26. The method of clause 25, wherein the first certain value is equal to 1 and the second certain value is equal to 0.

(618) 27. The method of clause 16, wherein the format rule specifies, in case that the second field has a certain value, to set a requirement for the value of the first field of an APS (Adaptation Parameter Set) NAL (Network Abstraction Layer) unit having an APS parameter type indicating LMCS (luma mapping with chroma scaling) type and the APS identifier equal to information included in the picture header or the slice header.

(619) 28. The method of clause 1, wherein the format rule further specifies that the value of the first field is independent of a presence of an identifier

of the adaptation parameter set in the picture header or the slice header.

(620) 29. The method of clause 28, wherein the format rule further specifies, in case that the second field has a certain value, to set a requirement for the value of the first field of an APS (Adaptation Parameter Set) NAL (Network Abstraction Layer) unit having an APS parameter type indicating SCALING type and/or ALF (adaptive loop filtering) type and/or LMCS (luma mapping with chroma scaling) type.

(621) 30. The method of clause 29, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is equal to a second certain value.

(622) 31. The method of clause 29, wherein the first certain value and the second certain value is equal to 0.

(623) 32. The method of clause 29, wherein the format rule specifies to set the requirement for the value of the first field to be equal to a first certain value in case that the value of the second field is greater than a second certain value.

(624) 33. The method of clause 32, wherein the third certain value is equal to 1 and the second certain value is equal to 0.

(625) 34. The method of any of clauses 1 to 33, wherein the conversion includes encoding the video into the bitstream.

(626) 35. The method of any of clauses 1 to 33, wherein the conversion includes decoding the video from the bitstream.

(627) 36. The method of clauses 1 to 33, wherein the conversion includes generating the bitstream from the video, and the method further comprises: storing the bitstream in a non-transitory computer-readable recording medium.

(628) 37. A video processing apparatus comprising a processor configured to implement a method recited in any one or more of clauses 1 to 36.

(629) 38. A method of storing a bitstream of a video, comprising, a method recited in any one of clauses 1 to 36, and further including storing the bitstream to a non-transitory computer-readable recording medium.

(630) 39. A computer readable medium storing program code that, when executed, causes a processor to implement a method recited in any one or more of clauses 1 to 36.

(631) 40. A computer readable medium that stores a bitstream generated according to any of the above described methods.

(632) 41. A video processing apparatus for storing a bitstream representation, wherein the video processing apparatus is configured to implement a method recited in any one or more of clauses 1 to 36.

(633) A third set of clauses show example embodiments of techniques discussed in the previous section (e.g., items 11-13).

(634) 1. A method of video processing (e.g., method **800** as shown in FIG. **8A**), comprising: performing **802** a conversion between a video region of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on syntax elements at i) a picture parameter set level and ii) a picture level or a slice level.

(635) 2. The method of clause 1, wherein the video region is a slice.

(636) 3. The method of clause 1, wherein the format rule specifies that the applicability of the deblocking filter for slices referring to a picture parameter set is based on a first syntax element in a picture parameter set and a second syntax element at the picture level or the slice level.

(637) 4. The method of clause 3, wherein the format rule specifies that a value of the first syntax element specifies whether the deblocking filter is applied for the slices referring to a picture parameter set unless indicated otherwise at the picture level or the slice level.

(638) 5. The method of clause 4, wherein the value of the first syntax element equal to 0 specifies that the deblocking filter is enabled for pictures referring to the picture parameter set unless overridden for a picture or a slice by information present a picture header or a slice header.

(639) 6. The method of clause 4, wherein the value of the first syntax element equal to 1 specifies that the deblocking filter is disabled for pictures referring to the picture parameter set unless overridden for a picture or a slice by information present a picture header or a slice header.

(640) 7. The method of clause 1, wherein the format rule specifies, in case that a third syntax element indicating a presence of deblocking parameters in a slice header is not present, to infer a value of the third syntax element to be equal to a certain value.

(641) 8. The method of clause 1, wherein the format rule specifies that whether to include a second syntax element indicating the applicability of the deblocking filter in the picture level or the slice level is dependent on (1) whether the deblocking filter is disabled for pictures referring to a picture parameter set and/or (2) a presence of deblocking parameters in a picture header or a slice header that indicates an allowability of deblocking behavior to be overridden.

(642) 9. The method of clause 8, wherein the format rule specifies that whether to include the second syntax element in the picture header is dependent on (1) whether the deblocking filter is disabled for pictures referring to the picture parameter set and/or (2) a value of a deblocking override flag that indicates the presence of the deblocking parameters in the picture header.

(643) 10. The method of clause 9, wherein the format rule specifies, in case that i) a deblocking filter flag indicating whether the deblocking filter is disabled for pictures referring to the picture parameter set and ii) the deblocking override flag at a picture header level have a certain value, to omit the second syntax element from the picture header.

(644) 11. The method of clause 9, wherein the format rule specifies, in case that i) a deblocking filter flag indicating whether the deblocking filter is disabled for pictures referring to the picture parameter set, ii) the deblocking override flag at a picture header level, and iii) a presence flag indicating a presence of the deblocking override flag have a certain value, to omit the second syntax element from the picture header.

(645) 12. The method of clause 9, wherein the format rule specifies, in case that the second syntax element indicating the applicability of the deblocking filter is not present, to infer a value of the second syntax element.

(646) 13. The method of clause 12, wherein the format rule specifies to infer the value of the second syntax element to be equal to a certain value indicating the deblocking filter is enabled in the picture, in case that i) the deblocking override flag at a PPS (picture parameter set) level indicating that overriding of a deblocking operation is allowed for slices referring to the PPS, ii) a deblocking filter flag at the PPS level indicating that the deblocking filter is disabled for pictures referring to the picture parameter set, and iii) a presence flag at a picture header level indicating that the deblocking operation is to be overridden at the picture header level, and otherwise to infer the value of the second syntax element to be equal to a value of the deblocking filter flag at the PPS level.

(647) 14. The method of clause 12, wherein the format rule specifies to infer the value of the second syntax element to be equal to a certain value indicating the deblocking filter is enabled in the picture, in case that i) a deblocking filter flag at a PPS (picture parameter set) level indicating that the deblocking filter is disabled for pictures referring to the picture parameter set, and ii) a presence flag at a picture header level indicating that a deblocking operation is to be overridden at the picture header level, and otherwise to infer the value of the second syntax element to be equal to a value of the deblocking filter flag at the PPS level.

(648) 15. The method of clause 8, wherein the format rule specifies that whether to include the second syntax element in the slice header is dependent on (1) whether the deblocking filter is disabled for pictures referring to the picture parameter set and/or (2) a value of a deblocking override flag that indicates the presence of the deblocking parameters in the slice header.

(649) 16. The method of clause 15, wherein the format rule specifies, in case that i) a deblocking filter flag indicating whether the deblocking filter is disabled for pictures referring to the picture parameter set, ii) the deblocking override flag at a slice header level, and iii) a presence flag indicating a presence of the deblocking override flag have a certain value, to omit the second syntax element from the slice header.

(650) 17. The method of clause 15, wherein the format rule specifies, in case that i) a deblocking filter flag indicating whether the deblocking filter is disabled for pictures referring to the picture parameter set and ii) the deblocking override flag have a certain value, to omit the second syntax element from the slice header.

(651) 18. The method of clause 15, wherein the format rule specifies, in case that the second syntax indicating the applicability of the deblocking filter is not present, to infer a value of the second syntax element.

(652) 19. The method of clause 18, wherein the format rule specifies to infer the value of the second syntax element to be equal to a certain value

indicating that the deblocking filter is enabled in the picture, in case that i) the deblocking override flag at a PPS (picture parameter set) level indicating that overriding of a deblocking operation is allowed for slices referring to the PPS, ii) a deblocking filter flag at the PPS level indicating whether the deblocking filter is disabled for pictures referring to the picture parameter set, and iii) a presence flag at a slice header level indicating that the deblocking operation is to be overridden at the slice header level, and otherwise to infer the value of the second syntax element to be equal to a value of the deblocking filter flag at the PPS level.

(653) 20. The method of clause 18, wherein the format rule specifies to infer the value of the second syntax element to be equal to a certain value indicating that the deblocking filter is enabled in the picture, in case that i) a deblocking filter flag at a PPS (picture parameter set) level indicating that the deblocking filter is disabled for pictures referring to the picture parameter set, and ii) a presence flag at a slice header level indicating that a deblocking operation is to be overridden at the slice header level and otherwise to infer the value of the second syntax element to be equal to a value of the deblocking filter flag at the PPS level.

(654) 21. A method of video processing (e.g., method **810** as shown in FIG. 8B), comprising: performing **812** a conversion between a current video block of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies to omit a first syntax element indicating whether the current video block is coded in a skip mode based on at least one of a second syntax element indicating an applicability of a first prediction mode or a block size of the current video block, wherein the first prediction is derived from blocks of sample values of a same slice of the current video block as determined by block vectors.

(655) 22. The method of clause 21, wherein the second syntax element is at a sequence parameter set.

(656) 23. The method of clause 21, wherein the first prediction mode is an intra block copy prediction mode.

(657) 24. The method of clause 1, wherein that the format rule specifies to omit the first syntax element in case that the second syntax element indicates that the first prediction mode is enabled and the block size is not smaller than 64×64 .

(658) 25. A method of video processing (e.g., method **820** as shown in FIG. 8C), comprising: performing **822** a conversion between a video comprising one or more pictures and a bitstream of the video according to a format rule, wherein the format rule specifies a presence and/or a value of an indication indicating whether to partition a picture into a video region based on a number of coding tree blocks in the picture.

(659) 26. The method of clause 25, wherein the format rule specifies that the indication is not included in case that the number of coding tree blocks in the picture is equal to 1 or smaller than 2.

(660) 27. The method of clause 25, wherein the format rule specifies that the indication has the value equal to 0 in case that the number of coding tree blocks in the picture is equal to 1 or smaller than 2.

(661) 28. The method of any of clauses 1 to 27, wherein the conversion includes encoding the video into the bitstream.

(662) 29. The method of any of clauses 1 to 27, wherein the conversion includes decoding the video from the bitstream.

(663) 30. The method of clauses 1 to 27, wherein the conversion includes generating the bitstream from the video, and the method further comprises: storing the bitstream in a non-transitory computer-readable recording medium.

(664) 31. A video processing apparatus comprising a processor configured to implement a method recited in any one or more of clauses 1 to 30.

(665) 32. A method of storing a bitstream of a video, comprising, a method recited in any one of clauses 1 to 30, and further including storing the bitstream to a non-transitory computer-readable recording medium.

(666) 33. A computer readable medium storing program code that, when executed, causes a processor to implement a method recited in any one or more of clauses 1 to 30.

(667) 34. A computer readable medium that stores a bitstream generated according to any of the above described methods.

(668) 35. A video processing apparatus for storing a bitstream representation, wherein the video processing apparatus is configured to implement a method recited in any one or more of clauses 1 to 30.

(669) In the present disclosure, the term “video processing” may refer to video encoding, video decoding, video compression or video decompression. For example, video compression algorithms may be applied during conversion from pixel representation of a video to a corresponding bitstream representation or vice versa. The bitstream representation of a current video block may, for example, correspond to bits that are either co-located or spread in different places within the bitstream, as is defined by the syntax. For example, a macroblock may be encoded in terms of transformed and coded error residual values and also using bits in headers and other fields in the bitstream. Furthermore, during conversion, a decoder may parse a bitstream with the knowledge that some fields may be present, or absent, based on the determination, as is described in the above solutions. Similarly, an encoder may determine that certain syntax fields are or are not to be included and generate the coded representation accordingly by including or excluding the syntax fields from the coded representation.

(670) The disclosed and other solutions, examples, embodiments, modules and the functional operations described in this disclosure can be implemented in digital electronic circuitry, or in computer software, firmware, or hardware, including the structures disclosed in this disclosure and their structural equivalents, or in combinations of one or more of them. The disclosed and other embodiments can be implemented as one or more computer program products, i.e., one or more modules of computer program instructions encoded on a computer readable medium for execution by, or to control the operation of, data processing apparatus. The computer readable medium can be a machine-readable storage device, a machine-readable storage substrate, a memory device, a composition of matter effecting a machine-readable propagated signal, or a combination of one or more of them. The term “data processing apparatus” encompasses all apparatus, devices, and machines for processing data, including by way of example a programmable processor, a computer, or multiple processors or computers. The apparatus can include, in addition to hardware, code that creates an execution environment for the computer program in question, e.g., code that constitutes processor firmware, a protocol stack, a database management system, an operating system, or a combination of one or more of them. A propagated signal is an artificially generated signal, e.g., a machine-generated electrical, optical, or electromagnetic signal, that is generated to encode information for transmission to suitable receiver apparatus.

(671) A computer program (also known as a program, software, software application, script, or code) can be written in any form of programming language, including compiled or interpreted languages, and it can be deployed in any form, including as a stand-alone program or as a module, component, subroutine, or other unit suitable for use in a computing environment. A computer program does not necessarily correspond to a file in a file system. A program can be stored in a portion of a file that holds other programs or data (e.g., one or more scripts stored in a markup language document), in a single file dedicated to the program in question, or in multiple coordinated files (e.g., files that store one or more modules, subprograms, or portions of code). A computer program can be deployed to be executed on one computer or on multiple computers that are located at one site or distributed across multiple sites and interconnected by a communication network.

(672) The processes and logic flows described in this disclosure can be performed by one or more programmable processors executing one or more computer programs to perform functions by operating on input data and generating output. The processes and logic flows can also be performed by, and apparatus can also be implemented as, special purpose logic circuitry, e.g., a field programmable gate array (FPGA) or an application specific integrated circuit (ASIC).

(673) Processors suitable for the execution of a computer program include, by way of example, both general and special purpose microprocessors, and any one or more processors of any kind of digital computer. Generally, a processor will receive instructions and data from a read only memory or a random-access memory or both. The essential elements of a computer are a processor for performing instructions and one or more memory devices for storing instructions and data. Generally, a computer will also include, or be operatively coupled to receive data from or transfer data to, or both, one or more mass storage devices for storing data, e.g., magnetic, magneto optical disks, or optical disks. However, a computer need not have such devices. Computer readable media suitable for storing computer program instructions and data include all forms of non-volatile memory, media and

memory devices, including by way of example semiconductor memory devices, e.g., erasable programmable read-only memory (EPROM), electrically erasable programmable read-only memory (EEPROM), and flash memory devices; magnetic disks, e.g., internal hard disks or removable disks; magneto optical disks; and compact disc, read-only memory (CD ROM) and digital versatile disc read-only memory (DVD-ROM) disks. The processor and the memory can be supplemented by, or incorporated in, special purpose logic circuitry.

(674) While the present disclosure contains many specifics, these should not be construed as limitations on the scope of any subject matter or of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular techniques. Certain features that are described in the present disclosure in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially claimed as such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

(675) Similarly, while operations are depicted in the drawings in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. Moreover, the separation of various system components in the embodiments described in the present disclosure should not be understood as requiring such separation in all embodiments.

(676) Only a few implementations and examples are described and other implementations, enhancements and variations can be made based on what is described and illustrated in the present disclosure.

Claims

1. A method of video processing, comprising: performing a conversion between a video region of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first syntax element in an adaptation parameter set (APS) is based on a chroma format index and APS identifier information included in a picture header (PH) or a slice header (SH) in response to a picture or a slice referring to the APS, wherein the first syntax element specifies whether chroma-related APS syntax elements are allowed to be present in the APS or not, and wherein the constraint is checked after parsing the APS and the PH or SH; wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on i) a first disabled control syntax element in a picture parameter set and ii) a second disabled control syntax element in the picture header or a third disabled control syntax element in the slice header, wherein a value of the first disabled control syntax element equal to 0 specifies that the deblocking filter is enabled for slices in pictures referring to the picture parameter set unless overridden for the picture or the slice by information present in the picture header or the slice header, wherein the value of the first disabled control syntax element equal to 1 specifies that the deblocking filter is disabled for slices in pictures referring to the picture parameter set unless overridden for a picture or a slice by information present in the picture header or the slice header, and wherein the format rule specifies that whether to include the third disabled control syntax element in the slice header is dependent on (1) the value of the first disabled control syntax element and (2) a value of a fifth syntax element indicating an allowability of a presence of deblocking parameters in the slice header.
2. The method of claim 1, wherein when the chroma format index is equal to 0, a value of the first syntax element of an APS network abstraction layer (NAL) unit having APS parameters type equal to ALF_APS and APS identifier equal to APS identifier information included in the PH or SH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to LMCS_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to SCALING_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, and wherein the value of the first syntax element equal to 0 indicates that the chroma-related APS syntax elements are not allowed to be present in the APS.
3. The method of claim 1, wherein the video region is a slice.
4. The method of claim 1, wherein the format rule specifies that whether to include the second disabled control syntax element in the picture header is dependent on (1) the value of the first disabled control syntax element and (2) a value of a fourth syntax element indicating an allowability of a presence of deblocking parameters in the picture header.
5. The method of claim 4, wherein the format rule specifies: when i) the value of the first disabled control syntax element is equal to 1, and ii) the value of the fourth syntax element is equal to 1 which indicates that the deblocking parameters are allowed to be present in the picture header, the second disabled control syntax element is omitted from the picture header, and when iii) the value of the first disabled control syntax element is equal to 0, and iv) the value of the fourth syntax element is equal to 1, the second disabled control syntax element is included in the picture header.
6. The method of claim 5, wherein the format rule specifies when the second disabled control syntax element is not present, a value of the second disabled control syntax element is inferred to be equal to a certain value indicating the deblocking filter is enabled in the picture when i) the value of the first disabled control syntax element is equal to 1, and ii) the value of the fourth syntax element is equal to 1, and otherwise, the value of the second disabled control syntax element is inferred to be equal to the value of the first disabled control syntax element.
7. The method of claim 1, wherein the format rule specifies: when i) the value of the first disabled control syntax element is equal to 1, and ii) the value of the fifth syntax element is equal to 1 which indicates that the deblocking parameters are allowed to be present in the slice header, the third disabled control syntax element is omitted from the slice header, and when iii) the value of the first disabled control syntax element is equal to 0, and iv) the value of the fifth syntax element is equal to 1, the third disabled control syntax element is included in the slice header.
8. The method of claim 7, wherein the format rule specifies when the third disabled control syntax element is not present, a value of the third disabled control syntax element is inferred to be equal to a certain value indicating the deblocking filter is enabled in the slice when i) the value of the first disabled control syntax element is equal to 1, and ii) the value of the fifth syntax element is equal to 1.
9. The method of claim 1, wherein the format rule specifies that when the fifth syntax element is not present, the value of the fifth syntax element is inferred to be equal to 0, which indicates that the deblocking parameters are not present in the slice header.
10. The method of claim 1, wherein the conversion includes encoding the video into the bitstream.
11. The method of claim 1, wherein the conversion includes decoding the video from the bitstream.
12. An apparatus for processing video data comprising a processor and a non-transitory memory with instructions thereon, wherein the instructions upon execution by the processor, cause the processor to: perform a conversion between a video region of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first syntax element in an adaptation parameter set (APS) is based on a chroma format index and APS identifier information included in a picture header (PH) or a slice header (SH) in response to a picture or a slice referring to the APS, wherein the first syntax element specifies whether chroma-related APS syntax elements are allowed to be present in the APS or not, and wherein the constraint is checked after parsing the APS and the PH or SH, wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on i) a first disabled control syntax element in a picture parameter set and ii) a second disabled control syntax element in the picture header or a third disabled control syntax element in the slice header, wherein a value of the first disabled control syntax element equal to 0 specifies that the deblocking filter is enabled for slices in pictures referring to the picture parameter set unless overridden for the picture or the slice by information present in the picture header or the slice header, wherein the value of the first disabled control syntax element equal to 1 specifies that the deblocking filter is disabled for slices in pictures referring to the picture parameter set unless overridden for a picture or a slice by information present in the picture header or the slice header, and wherein the format rule specifies that

whether to include the third disabled control syntax element in the slice header is dependent on (1) the value of the first disabled control syntax element and (2) a value of a fifth syntax element indicating an allowability of a presence of deblocking parameters in the slice header.

13. The apparatus of claim 12, wherein when the chroma format index is equal to 0, a value of the first syntax element of an APS network abstraction layer (NAL) unit having APS parameters type equal to ALF_APS and APS identifier equal to APS identifier information included in the PH or SH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to LMCS_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to SCALING_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, and wherein the value of the first syntax element equal to 0 indicates that the chroma-related APS syntax elements are not allowed to be present in the APS.

14. A non-transitory computer-readable storage medium storing instructions that cause a processor to: perform a conversion between a video region of a video and a bitstream of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first syntax element in an adaptation parameter set (APS) is based on a chroma format index and APS identifier information included in a picture header (PH) or a slice header (SH) in response to a picture or a slice referring to the APS, wherein the first syntax element specifies whether chroma-related APS syntax elements are allowed to be present in the APS or not, and wherein the constraint is checked after parsing the APS and the PH or SH, wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on i) a first disabled control syntax element in a picture parameter set and ii) a second disabled control syntax element in the picture header or a third disabled control syntax element in the slice header, wherein a value of the first disabled control syntax element equal to 0 specifies that the deblocking filter is enabled for slices in pictures referring to the picture parameter set unless overridden for the picture or the slice by information present in the picture header or the slice header, wherein the value of the first disabled control syntax element equal to 1 specifies that the deblocking filter is disabled for slices in pictures referring to the picture parameter set unless overridden for a picture or a slice by information present in the picture header or the slice header, and wherein the format rule specifies that whether to include the third disabled control syntax element in the slice header is dependent on (1) the value of the first disabled control syntax element and (2) a value of a fifth syntax element indicating an allowability of a presence of deblocking parameters in the slice header.

15. The non-transitory computer-readable storage medium of claim 14, wherein when the chroma format index is equal to 0, a value of the first syntax element of an APS network abstraction layer (NAL) unit having APS parameters type equal to ALF_APS and APS identifier equal to APS identifier information included in the PH or SH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to LMCS_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to SCALING_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, and wherein the value of the first syntax element equal to 0 indicates that the chroma-related APS syntax elements are not allowed to be present in the APS.

16. A non-transitory computer-readable recording medium storing a bitstream of a video which is generated by a method performed by a video processing apparatus, wherein the method comprises: generating the bitstream for a video region of the video according to a format rule, and wherein the format rule specifies that a constraint on a value of a first syntax element in an adaptation parameter set (APS) is based on a chroma format index and APS identifier information included in a picture header (PH) or a slice header (SH) in response to a picture or a slice referring to the APS, wherein the first syntax element specifies whether chroma-related APS syntax elements are allowed to be present in the APS or not, and wherein the constraint is checked after parsing the APS and the PH or SH, wherein the format rule specifies that an applicability of a deblocking filter for the video region is determined based on i) a first disabled control syntax element in a picture parameter set and ii) a second disabled control syntax element in the picture header or a third disabled control syntax element in the slice header, wherein a value of the first disabled control syntax element equal to 0 specifies that the deblocking filter is enabled for slices in pictures referring to the picture parameter set unless overridden for the picture or the slice by information present in the picture header or the slice header, wherein the value of the first disabled control syntax element equal to 1 specifies that the deblocking filter is disabled for slices in pictures referring to the picture parameter set unless overridden for a picture or a slice by information present in the picture header or the slice header, and wherein the format rule specifies that whether to include the third disabled control syntax element in the slice header is dependent on (1) the value of the first disabled control syntax element and (2) a value of a fifth syntax element indicating an allowability of a presence of deblocking parameters in the slice header.

17. The non-transitory computer-readable recording medium of claim 16, wherein when the chroma format index is equal to 0, a value of the first syntax element of an APS network abstraction layer (NAL) unit having APS parameters type equal to ALF_APS and APS identifier equal to APS identifier information included in the PH or SH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to LMCS_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, wherein when the chroma format index is equal to 0, the value of the first syntax element of the APS NAL unit having APS parameters type equal to SCALING_APS and APS identifier equal to APS identifier information included in the PH is equal to 0, and wherein the value of the first syntax element equal to 0 indicates that the chroma-related APS syntax elements are not allowed to be present in the APS.
